

Master's Thesis

Development of a Source Library for Transfer Learning: Leveraging Clustering and Contrastive Learning Techniques

Sneha Banerjee
Matriculation Number: 430069

Monday 5th August, 2024

1st Examiner:	Prof. Dr. Christian Bauckhage, University of Bonn
2nd Examiner:	Prof. Dr.-Ing. Alexander Verl, ISW, University of Stuttgart
Supervisor:	Daniel Schiller

Declaration of Originality

Name	Sneha Banerjee
Matriculation number	430069
Address	Allmandring 8A,70569 Stuttgart
Title	<i>Development of a Source Library for Transfer Learning: Leveraging Clustering and Contrastive Learning Techniques</i>

I now declare,

- that I wrote this work independently,
- that no sources other than those stated are used and that all statements taken from other works—directly or figuratively—are marked as such,
- that the work submitted was not the subject of any other examination procedure, either in its entirety or in substantial parts,
- that I have not published the work in whole or in part, and
- that my work does not violate any rights of third parties and that I exempt the University against any claims of third parties.

Abstract

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Task and Goal	2
1.3	Approach	2
1.4	Contribution	3
2	Related Work	4
3	Fundamentals	6
3.1	Bin Picking	6
3.1.1	Definition and Application	6
3.1.2	Gripper Types	6
3.1.3	Model-Based Approaches	8
3.1.4	Model-Free Approaches	9
3.1.5	Robot System	10
3.2	Deep learning-based 3D Image processing	11
3.2.1	Convolution Neural Network	11
3.2.2	Autoencoders	12
3.2.3	Siamese Network	14
3.3	Clustering Algorithms	16
3.3.1	Density-based Clustering	17
3.3.2	Distribution-based Clustering	17
3.3.3	Hierarchical-based Clustering	18
3.3.4	Graph theory-based Clustering	18
3.4	Evaluation metrics for Clustering Algorithms	18
3.4.1	Silhouette Score	19
3.4.2	Calinski-Harabasz Index	19
3.4.3	David-Bouldin index	20
3.4.4	Density Based Cluster Validation Index	21
3.5	Transfer learning	25
3.5.1	Definition	25
3.5.2	Categories	26
3.5.3	Relevant Algorithms	27
3.6	Optimal Transport Problem	27
3.7	Sinkhorn-Knopp Algorithm	27
3.8	Dimensionality reduction	27
3.8.1	Linear Discriminant Analysis	28
3.8.2	Principal Component Analysis	28
4	Method	32
4.1	Architecture	32
4.2	Encoder Backbone	33
4.2.1	PointNet Autoencoder	33
4.2.2	Dynamic Graph Convolutional Neural Network	35
4.3	Point-level Clustering Module	38
4.3.1	Class Probability prediction	39
4.3.2	Label reassigning	39
4.4	Instance-level Contrasting Module	41
4.5	Loss function	42
4.6	Dimensionality reduction	42
4.6.1	T-distributed Stochastic Neighbor Embedding	42
4.7	Clustering Algorithm	44

4.8	Important aspects of the Studies	48
4.8.1	Focus and Restrictions	48
4.9	Suitable Algorithm and Adjustments	48
4.10	Whatever new Concept we propose	50
4.11	Source and Target Objects	50
4.12	Data Generation	50
4.13	Important Aspects	50
5	Application of algorithms	51
6	Experiments	52
6.1	Baseline algorithm	52
6.1.1	Basic Study without dimensionality reduction	52
6.1.2	Basic Study with t-distributed Stochastic Neighbor Embedding (t-SNE)	52
6.1.3	Basic Study with increasing datapoints	53
6.2	Conclu approach with PointNet as Backbone	54
6.2.1	Evaluation of increasing datapoints	54
6.2.2	Evaluation of different dimensionality reduction techniques	55
6.2.3	Evaluation of dimension of feature space representations	57
6.2.4	Evaluation of the number of semantic subgroups	61
6.2.5	Evaluation of the batch size for training	66
6.2.6	Evaluation of the effect of early stopping	68
6.2.7	Evaluation of the effect of learning rate	70
6.2.8	Evaluation of the effect of decay factor of the learning rate	71
6.3	Conclu approach with Dynamic Graph Convolutional Neural Network (DGCNN) as Back- bone	72
6.3.1	Evaluation of dimension of feature space representations	72
7	Implementation in a Bin Picking Cell	76
8	Conclusions	77
8.1	Discussions	77
8.2	Limitations	77
8.3	Future Scope	77
9	Summary	78
	List of Acronyms	i
	List of Figures	iii
	List of Tables	v
	List of Symbols	vii
	Bibliography	viii

1 Introduction

1.1 Motivation

Robot-based bin picking process automation offers enormous potential to completely transform by boosting productivity, efficiency, and cost-effectiveness. However, robot's capacity to adapt and learn from a variety of dynamic situations is frequently the deciding factor in the success of bin picking applications. Transfer learning is a potent machine learning technique that could help with this problem by allowing robots to swiftly adapt to new tasks and situations by using the knowledge from pre-trained models. Transfer learning seeks to use information from one domain or task to enhance performance on a related domain or task. Conventional machine learning paradigms involve training models on a given dataset for a certain problem, starting from scratch. This involves a huge computational overhead and time and cost associated with it. But with transfer learning, even when the datasets or domains are distinct, the information obtained from one task can be helpful for completing a related task.

The increasing requirement for effective and economical solutions in a variety of fields, including robotics, computer vision, natural language processing, and healthcare, gives rise to the necessity of a source library in transfer learning. Below is comprehensive overview of why transfer learning requires a source library:

1. **Acceleration of Research and development** It can take a lot of time and resource to create transfer learning solutions from scratch. A source library offers a foundation of proven models, datasets and algorithms, which speeds up research and development. Researchers have the ability to rapidly iterate, explore diverse methodologies, and expand on previous findings to propel the frontiers of transfer learning.
2. **Adaptability to Diverse Domains** Transfer learning can be used in a variety of fields, such as robotics, time series analysis, image recognition, and natural language processing. A source library provides frameworks that are adaptable to specific tasks and datasets, as well as modular components that can be integrated with existing networks to meet the varied needs of diverse domains.
3. **Reproducibility and Accessibility** For transfer learning applications, a source library offers a centralized collection of models, datasets, algorithms, and tools. Through easy to access resources, a source library fosters collaboration, reproducibility, and transparency for academics, developers, and other involved stakeholders.
4. **Minimization of Redundant Efforts** Creating transfer learning solutions frequently requires researchers to start from scratch when creating comparable algorithms or models. By providing pre-built components and reference implementations, a source library can reduce duplication of effort and free up developers to work on new concepts and applications.
5. **Standardization and Best Practices** To ensure consistency and dependability across implementations, a source library might provide best practices and standards for transfer learning approaches. Researchers can more efficiently compare results, assess performance, and build upon the work each other's work by embracing standard frameworks, interfaces and evaluation criteria.
6. **Encouraging Non-Experts** Complex transfer learning can necessitate domain-specific knowledge, machine learning proficiency, and data preprocessing skills. Technical difficulties are abstracted away by a source library, enabling non-experts to apply transfer learning techniques to their applications. users with limited technical knowledge can efficiently implement transfer learning due to the availability of tools and instructions.

In conclusion, a transfer learning source library acts as a catalyst for new research, expands access to cutting edge methods, and quickens the creation of useful applications in a variety of fields. An ecosystem of collaboration and a centralized repository of information are what enables academics and researchers in machine learning and artificial intelligence to take on challenging tasks and discover new avenues for advancement.

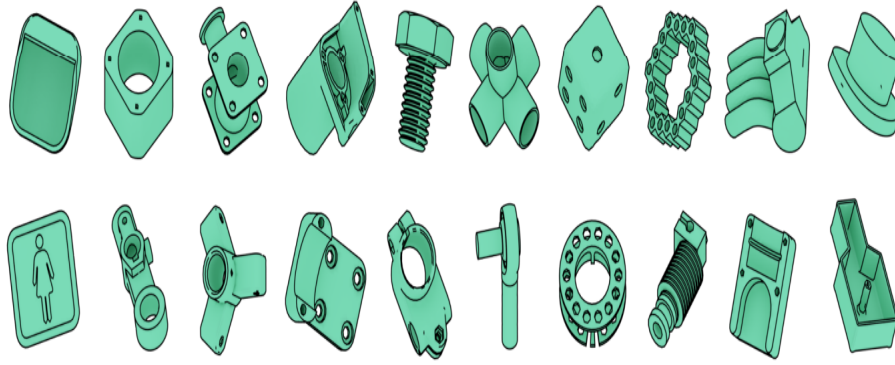


Figure 1.2.1: Some random objects from the ABC dataset.[39]

1.2 Task and Goal

Bin picking is fundamentally a pose estimation problem, which makes it a computer vision task. The solution to the bin picking problem was greatly influenced by the advancements in deep learning techniques for computer vision in recent years. At the moment, there are two available methods: model-free solutions and model-based solutions. The model-free methods rely on an image to find appropriate grasp positions, whereas the model-based solutions use Computer-Aided Design (CAD) models of items to detect them in an image[75]. The bin picking application at Fraunhofer Institute for Manufacturing Engineering and Automation (IPA) utilizes an advanced model-based method initially based on PQ-Net++ from [37]. The dataset used for this task is the ABC dataset[39] which is a collection of one million CAD models without any ground truth labels. It contains a large variety of objects, a handful of which are shown in Fig. 1.2.1

The goal of this work is to develop a source library consisting of "good" source data which can be used as the data from the source domain in the further steps in a transfer learning pipeline. Therefore, the focus is to identify which features prevalent in an object make it a "good" source data. A good collection of source data would be the objects that approximately capture the overall variance of the entire 1 million ABC dataset. Thus, it is trivial to think that more objects in the source library could better capture the overall variance of the dataset. But since the bin application is a model-based approach, it would be required to formulate the CAD models of all these objects. Thus, having a large number of objects in the source library would entail high computational time for the generation of all the object models. Therefore, it is crucial to determine a permissible number of objects in the source library to facilitate its usage down the pipeline. To further elaborate on this, if two objects are very similar to one-another than in spite of having a large similarity which just one other object, it loses the generalization capability of being nearly similar to a lot of other objects in the dataset and would therefore require a large amount of objects in the source library to capture the entire variance in the dataset. Thus, there is a trade-off between being very similar to a very small subset of objects and being approximately similar to a larger subset of objects in the dataset. Moreover, it is not universally defined which features in an object qualify as an important feature and how it contributes in deciding whether an object is capable of becoming a good source data.

1.3 Approach

An end-to-end autoencoder proposed by [50] was used to get the feature representations of the objects in the dataset[39]. The autoencoder learns both the local features and the global features of an object. Once the feature representation of the objects are obtained, they are grouped into clusters utilizing multiple clustering algorithms. A suitable clustering algorithm was utilized to get the representing members of the clusters. A high-level overview of the entire method is shown in Fig. 1.3.1. The clustering algorithm would not just consider the Euclidean distance between the datapoints but also other intrinsic features of the dataset such as the density because real-world datasets are not necessarily globular in structure. A suitable evaluation metric such as [52] was used to compare the quality of the clustering algorithms which was robust to the

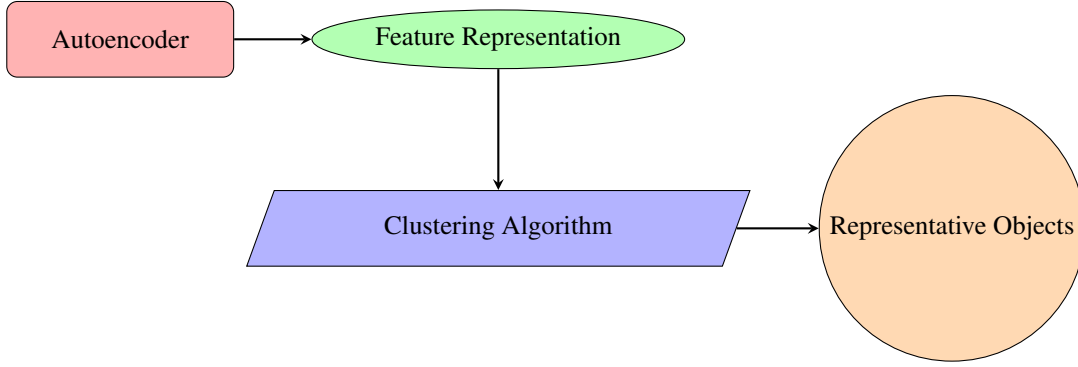


Figure 1.3.1: Overview of the entire pipeline for development of source library.

size and shape of individual clusters as well as the outliers and noise of the dataset. Since, computing the Density Based Clustering Validation (DBCVC) index on the entire dataset was extremely computationally expensive, the evaluation was performed on random batches of the subset. Moreover, the DBCVC is sensitive to the number of features in the dataset and fails to provide any tangible result in the presence of a large number of features (e.g. over 150). Thus, a dimensionality reduction was necessary for the evaluation of the clustering algorithm. Once, the clustering algorithm with best results in the evaluation metrics is achieved the entire dataset is categorized into a number of clusters, where the medoid of the cluster is chosen as the representative member of the cluster. The similarity of the test dataset with the representative members of the datasets shows the generalization quality of the source library. The approximate time for each step of the work is shown in Tab. 1.

Step	Time
Training the model	
Dimensionality reduction	
Clustering Algorithm	
Similarity evaluation	

Table 1: Overview of the approximate time taken for the creation of the source library

1.4 Contribution

Although density-based clustering algorithm like Density-Based Spatial Clustering of Applications with Noise (DBSCAN) and Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN) shows promising results when the clusters in real-world datasets aren't globular in nature, yet the current existing implementations in the literature doesn't put any restrictions on the number of clusters returned by the clusters. In case of very large datasets like the ABC dataset[39], these algorithms return a large number of clusters (over multiple thousands). The model generation of such thousands of objects aren't feasible or efficient in business situations such as in IPA. Thus, novel contribution in this work is stated as follows:

- The idea of multi-level HDBSCAN algorithm is proposed in this work to limit the number of clusters to a feasible number but also retain the properties of a density based clustering algorithm.
- An extensive experimentation on the optimal hyperparameter values during model training has been carried on for the development of a high performing network used for the generation of feature representations of the objects in the dataset.

2 Related Work

This chapter provides a quick overview of the literature on unsupervised learning approaches for point clouds as well as variety of local feature extraction techniques on point clouds.

Deep learning based approaches Since deep learning methods can directly consume point clouds, feature learning for 3D objects has advanced significantly in the recent years. By independently learning every point in the point cloud and combining point characteristics with permutation invariant operations, PointNet [65] and Deep Sets [93] were the pioneers in the field to handle unordered and unstructured 3D points. Both PointNet[65] and Deep Sets[93] were important developments in deep learning for handling unordered data; PointNet was designed particularly for 3D point cloud analysis, while Deep Sets provides a broader framework for feature learning that is permutation invariant and could be extrapolated to graphs and sequences. Although these methods have shown promising results in a number of applications, further research is required to resolve their shortcomings in capturing local features and enhance their scalability and performance in a variety of applications and larger datasets. By adding a hierarchical feature learning mechanism that operates directly in a metric space, PointNet++ [66] expands the capabilities of PointNet. Its contributions include the ability to learn features at different scales, the hierarchical organization of point clouds into nested partitions, and sampling/grouping procedures that record intricate spatial relationships while preserving permutation invariance. But because of the increased computational complexity brought forth by hierarchical architecture, training and inference periods may be prolonged. Its applicability in some circumstances may be limited by its scalability, specially with very large point clouds. PointNet++ improves point cloud feature learning overall, but its practical application necessitates careful consideration of processing resources and scalability.[84] designed a DGCNN architecture specifically for point cloud data. Its main contribution is to enable more efficient feature learning by dynamically building graph structures from point clouds based on local geometric correlations. But on the other hand, dynamic graph creation adds more computational complexity and might necessitate precise hyperparameter tweaking. While PointCNN[45] achieved state-of-the-art performance on various point cloud tasks, the X-transformation introduces additional computational overhead and may limit scalability, specially with large-scale datasets. PointCNN[45] proposes a Convolutional Neural Network (CNN) specifically designed for processing point clouds. Its major contribution is the introduction of X-transformation, a permutation-invariant operation that reorients point clouds into a canonical layout, allowing application of standard convolutional operations. PointConv [90], a deep learning architecture that conducts convolutional operations directly on 3D point clouds was presented in "Deep convolutional Networks on 3D Point Clouds". Its contribution is the construction of a new convolutional operator that is permutation invariant and preserves local geometric structures of point clouds. However, the convolutional procedures on irregular point clouds may cause PointConv to suffer from increased computational complexity, which could affect the efficiency of training and inference.[48] presented a CNN that was specifically designed to handle tasks related to point cloud analysis. The incorporation of relation-aware convolutional procedures, which capture both local and global geometric relationships inside point clouds, is its primary contribution. To obtain optimal performance, careful parameter tuning may be necessary due to the dependence on relation-aware features, which may result in an increase in computing overhead. Position-adaptive feature learning was the primary innovation of PConv[91], which is the dynamic assembly of convolutional kernels based on local geometric structures. by using this method, the network performed better in segmentation and classification tasks by improving its capacity to detect intricate geometric patterns and variances within point clouds. Nevertheless, computational complexity may provide PConv with difficulties, especially during training and inference, and more research is needed to determine whether or not it can scale to large-size datasets. All things considered, PConv was a major step forward in point cloud feature learning, but its practical application necessitates careful attention to scalability and processing resources. The main contribution of AdaptConv [95] was the inclusion of adaptive graph convolutional operations, which dynamically modify convolutional kernels receptive fields according to the point clouds local geometric characteristics. Because of the flexibility, the network can better execute tasks like segmentation and classification by capturing local and global geometric data. Convolutional operations adaptive nature, however, might add more computing complexity and necessitate precise parameter tweaking. Moreover, the quality of local geometric information and intricacy of the point cloud may have a significant influence on the adaptive graph convolutional approach's

efficacy, which may affect how well it performs in various datasets and applications. A unique method for encoding point clouds in 3D space was proposed in [76], which uses quadratic terms to more precisely reflect local surface geometry. The main contribution is the addition of quadratic components to the conventional point-to-point representation, which improves the model’s ability to detect surface curvature and form fluctuations. The suggested approach enhances the discriminative power of deep learning models for tasks like point cloud segmentation and classification by adding higher-order geometric information to the representation. Quadratic terms, on the other hand, might add more computational complexity, which could affect the speed of inference and training. Furthermore, the quality and density of raw point cloud data may have an impact on how successful the suggested representation is, necessitating careful thought and preprocessing. All things considered, [76] presented improvements in encoding intricate geometric details; however, due to its computational complexity and data sensitivity issues, more research is needed before it can be used in real-world scenarios. [18] described how perturbation learning can be used to improve point cloud data resolution. This method’s primary advantage is its capacity to produce high resolution point clouds with enhanced geometric details, which is advantageous for applications like object detection and 3D reconstruction. The technique can efficiently capture surface details and fine-grained features by utilizing perturbation learning, which improves the overall quality of the unsampled point clouds. Furthermore, the method is adaptable and does not require specific surface representations, making it suitable for wide range of point cloud datasets. The computational complexity of training the perturbation model, which can rise in tandem with the volume and intricacy of the input point cloud, is one of the possible disadvantage, though. Additionally, the quality of the perturbation model that is trained and the availability of the adequate training data may determine how effective the method is, which may restrict its application to specific situations. Overall, [18] presented encouraging advances in point cloud processing, nonetheless, for practical implementation, it is imperative to overcome the issues associated to computing and data. [74] presented a novel method for geometric feature preservation in Light Detection and Ranging (LiDAR) point cloud compression. This framework’s ability to produce lossless compression-which guarantees that no information across many layers of the point cloud by utilizing a layer-wise geometry aggregation mechanism. This allows for efficient modelling and compression of intricate 3D structures. Furthermore, the framework is flexible and can be adjusted to suit various compression needs and LiDAR datasets. A possible disadvantage, is the computational complexity involved in analyzing and combining geometric data over several layers; this complexity may rise as input point cloud data becomes more complicated and large. Additionally, even though the technique produces lossless compression, it might not always outperform lossy compression techniques in terms of compression ratios, which might be important to take into account in situations when storage is limited.[94] offered a cutting-edge method for employing transformer networks to refine votes in order to improve 3D object detection performance. The method’s main contribution is how it uses transformer architectures to combine and fine-tune votes from various point cloud regions in order to improve object detection. The approach improved accuracy and robustness in 3D object detection tasks by capturing global context information and spatial links between object proposals through the incorporation of transformer-based refinement. Furthermore, the approach is flexible and may be combined with current 3D object detection frameworks, offering a flexible way to improve detection efficiency. While these approaches were remarkably successful, in order to do feature learning, they need supervised data. Point cloud models are difficult to implement in novel real-world scenarios where labelled data is hard to come by due to their reliance on annotation. Therefore, it’s critical to devise strategies for lowering the quality of annotated samples needed to complete deep learning-based point cloud interpretation tasks with the necessary performance.[50]

3 Fundamentals

3.1 Bin Picking

3.1.1 Definition and Application

Bin picking is a robotics and automation technique used in manufacturing and logistics, automotive industry, particularly in warehouses and assembly lines. It involves locating, grabbing and manipulating objects from a bin or container utilizing robotic devices equipped with sensors, cameras and algorithms. Thus, it is a crucial area of interest for research in computer vision and robotics. There are usually multiple steps in the process: sensing and perception, planning and path calculation, grasping and manipulation, and ultimately transportation and placement which will be further elaborated in the following passages.

Sensing and Perception To determine the kind position, orientation and the kind of objects inside a bin or container, the robot uses its sensors and cameras to scan it. For the purpose of identifying the items, especially ones that are partially occluded or organized randomly or has entanglement in between them, advanced computer vision algorithms are used for the purpose.

Planning and Path Calculation Following the identification of the items in the bin, the robots control system determines the best course for the arm or gripper to take in order to reach and retrieve the intended item in the bin. Preventing collisions with other items in the container and maximizing efficiency are the two crucial factors in this part of the process.

Grasping and Manipulation After positioning the arm of the robot in a suitable location, the robot arm employs its gripper to firmly hold the object of interest. To guarantee that the item is raised without damage and slippage, this stage calls for very precise control.

Transportation and Placement The robot moves the item to the intended spot, like a shipping container, assembly line or conveyor belt, after it has been picked. Before settings the object down, the robot occasionally needs to realign or orient it.

There are numerous benefits of bin picking in manufacturing and logistics.

- **Enhanced Efficiency** Businesses can greatly boost productivity and throughput, particularly in high-volume operations, by automating the process of selecting goods from bins.
- **Flexibility** Bin picking applications are appropriate for a variety of manufacturing situations because they can handle a wide range of items based on their shapes, size and materials.
- **Savings** Automating manual labor-intensive jobs, such as bin picking, can save labor costs and lower the chance of accidents that come with physical handling.
- **Accuracy** Robots can pick and put objects with high degree of repeatability and accuracy because of its advanced sensing and control systems, which lowers mistake rates and improves product quality.

All things considered, bin picking applications are essential to contemporary manufacturing and logistic operations, helping businesses to increase productivity and streamline workflows.

3.1.2 Gripper Types

Grippers are crucial parts of robotic systems used in bin picking applications that allow them to grab and move objects out of the bins or containers. There are different kinds of grippers, and each is appropriate for a particular purpose depending on the shape, size, weight, and substance of the objects being handled. The following are a few typical gripper kinds used for bin picking.

Vacuum Grippers These grippers cling to smooth or flat surfaces using suction as shown in Fig. 3.1.1. When handling huge, flat good like sheets of plastic, glass, or metal, they are especially useful. By using

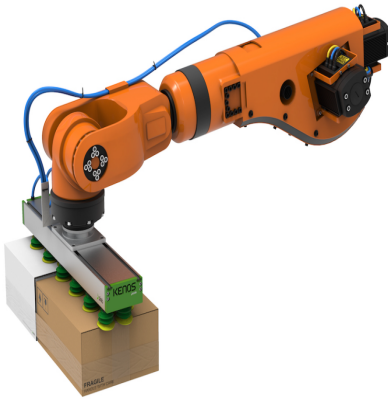


Figure 3.1.1: Vacuum Grippers[63]



Figure 3.1.2: Mechanical Grippers[23]



Figure 3.1.3: Magnetic Grippers[71]



Figure 3.1.4: Adaptive Grippers[53]

interchangeable suction cups or pads, vacuum grippers can be made to handle objects of various sizes and shapes. The primary advantage of these type of grippers is the need for one-surface accessibility. This makes handling cluttered bins easier. The drawback however is that a vacuum gripper can still generate traction even if the intended grasping position is not met. This results in a possibly erroneous placement and an imprecise position of the gripper.[75]

Mechanical Grippers To grasp objects, mechanical grippers use mechanical jaws, fingers, or claws. They may be tailored with various jaw configurations to meet particular object geometries and are appropriate for a broad variety of shapes and sizes. Mechanical grippers are useful for picking up objects with uneven shapes, such as bottles, crates, and components, as they offer a solid hold as shown in Fig. 3.1.2. But because of the tough and rigid mechanical nature of the gripper, it might cause damage while handling delicate and sensitive objects.

Magnetic Grippers These devices draw in and hold onto ferromagnetic materials like steel and iron using electromagnetic forces as shown in fig. 3.1.3. As such, the benefits and drawbacks are similar to those of vacuum grippers. They are appropriate for applications where cleanliness or delicate surfaces are a concern since they are perfect for handling metallic objects without the need for direct contact. Therefore, it is possible that the gripper might grab multiple objects at a time instead of a single object which is not desirable. Moreover, because of its magnetic properties, it can induce magnetic behavior to the objects being handled which is not desirable and can have serious implication in certain applications. [75]

Adaptive Grippers These grippers can handle a broad range of sizes and forms without the need for exact

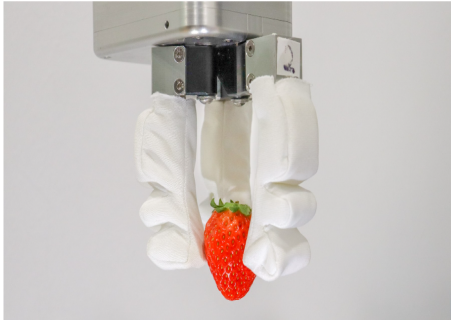


Figure 3.1.5: Soft Grippers[55]

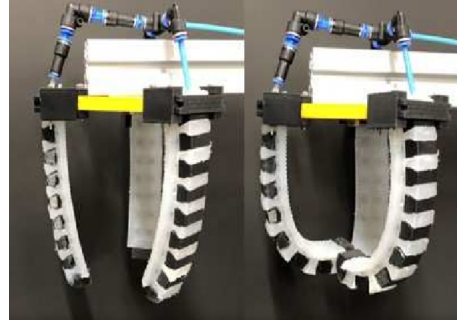


Figure 3.1.6: Hybrid Grippers with soft and rigid structures[60]

placement since they are made to conform to the shape of the object being held as shown in Fig. 3.1.6. These grippers frequently modify their shape to fit the curves of the object by using inflated bladders or flexible polymers. However, to facilitate its adaptive property, it demands for complex control algorithms for precise grasping.

Soft Grippers Designed to softly grab delicate or unusually shaped objects without causing damage, soft grippers are created from responsive materials like silicone or rubber as shown in Fig. 3.1.5. They are frequently employed in tasks like picking up food or electrical components, where dexterity and delicate handling are crucial. However, they have lower gripping force as compared rigid grippers, thus limiting suitability for heavier objects.

Hybrid Grippers To improve performance and versatility, hybrid grippers integrate many grasping techniques such as vacuum suction and mechanical clamping. These grippers are frequently employed in intricate bin picking scenarios where a solitary gripping technique would not be adequate. It also requires more calibration and maintenance due to the integration of multiple systems. It also leads to greater power consumption and computational overhead for executing hybrid gripping mechanism. Thus, the characteristics of the objects being handled, the capabilities of the robotic system, and the particular requirements of the application all play a role in the gripper selection. Stakeholders can maximize the effectiveness, dependability, and precision of their bin picking processes by choosing the right kind of gripper. Next, we look into the different approaches in bin-picking applications.

3.1.3 Model-Based Approaches

Model-based bin picking approaches use a digital model, or representation, of the items in the bin as a guide for organizing and carrying out the bin picking procedure. To enable accurate perception and manipulation, these methods make use of CAD data or 3-D models of the items. Fig. 3.1.7 gives an overview of model-based bin picking approach, an elaboration of which is provided below.

Object Modelling Creating precise digital models of the objects to be picked is the first stage in a model-based bin-picking strategy. CAD software or 3D point cloud generated from real objects through scanning methods like laser scanning or photogrammetry can be used for this. The model ought to be accurately represent the objects surface features, size, and geometry.[46]

Bin Perception The robots sensors, including depth cameras or laser scanners, record information about the contents of the bin once the object models are accessible. Based on the shape, size and orientation of each object in the bin, this data is processed to identify and segment each of them. In order to match and align the perceived objects with their corresponding models, the digital models are used as reference templates.[46]

Pose Estimation It is the process of figuring out each recognized object's exact location and orientation with

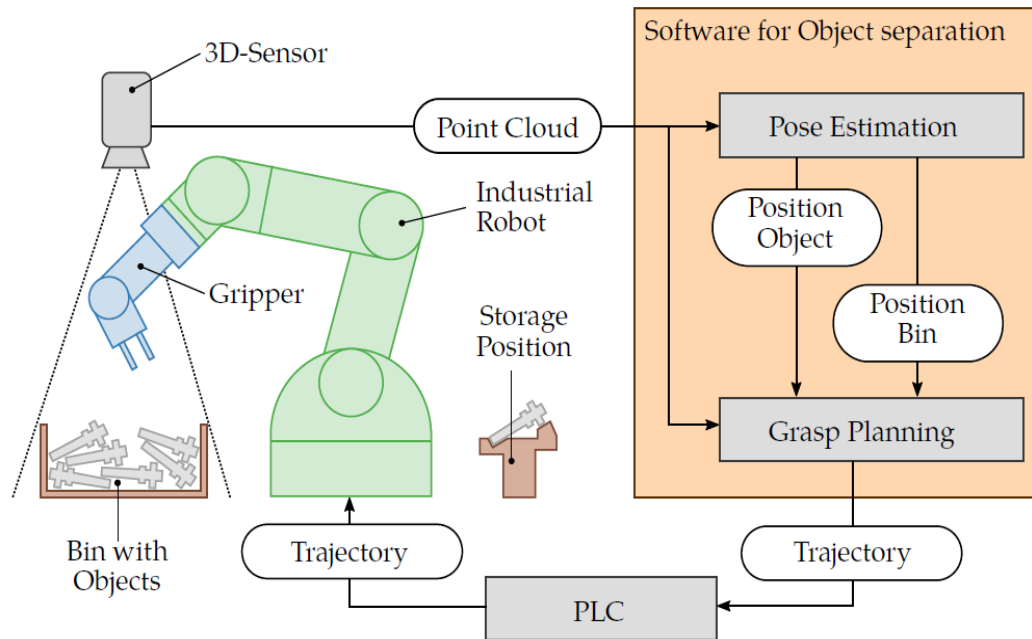


Figure 3.1.7: Overview of model-based bin picking approach.[75]

respect to the gripper or end effector of the robot. Planning precise gripping trajectories and guaranteeing successful object manipulation depend on this phase. Pose estimation methods include geometric fitting algorithms, point cloud registration, and feature mapping.[46]

Grasping Planning After determining the objects posture, the robot arranges its grasping techniques to retrieve the objects from the bin. In order for the robot's arm or gripper to approach, grasp, and safely lift each object, collision-free motion paths must be created. To enable successful gripping, planning algorithms take into account variables including gripper kinematics, object geometry, and workspace limitations. [46]

Feedback Control In order to monitor and modify the robot's movements in real-time while it is grasping, feedbacks from the sensors such as force/torque sensors or vision systems is utilized. By using feedback control approaches, the robot may adjust its grip in response to unforeseen disruptions or changes in the surroundings, guaranteeing dependable and durable picking performance.[46]

Validation and Verification The system compares the actual results with the predicted result based on model-based planning to determine whether the grasp was successful after selecting each object. Verifying grasp stability, item alignment, and adherence to quality standards are a few examples of validation tasks. The robot can rectify any mistakes by trying to pick the object again or modifying its approach.[46]

3.1.4 Model-Free Approaches

The goal of model-free bin picking approach is to pick objects from a bin without using CAD data or predefined object models. To identify and handle objects based solely on sensor data, these methods use advanced machine learning algorithms and complex perception techniques. Model-free approaches can be broadly categorized into two groups: deep-learning based approach[6], reinforcement learning based approach[44] and hybrid perception-based approach[33].

Deep Learning-Based Approach Without explicitly modelling the objects, deep learning techniques like CNNs are trained on massive amount of sensor data to learn object recognition and grasping policies on

their own. Labelled object instances and corresponding grasping actions are collected from large datasets of sensor data (e.g. Red Green Blue (RGB) images, depth maps). Using end-to-end optimization, CNNs are trained on those data to acquire grasping policies and object representations. based on sensor input, robotic systems with trained models are able to grasp objects and identify them in real-time. To enhance grip execution and boost performance, feedback from sensors (such as force/torque sensors) are employed.[6]

Reinforcement Learning-Based Approach Without explicit object models, robots can acquire grasping policies through trial and error interactions with the environment by using Reinforcement Learning (RL) techniques. By performing grasping movements and getting feedback (like reward signals) when a task is completed successfully, the robot interacts with its surroundings(e.g. a bin).RL algorithms learn grasping policies by optimizing the total rewards earned from the accomplished grasps and interactions with the surroundings. In order to maximize the performance, RL agents strike a balance between exploring novel grasping tactics and utilizing previously learnt rules. robotic systems are equipped with capability to perform bin-picking tasks in real-time.[44]

Hybrid Perception-Based Approach Robots can learn grasping rules from data by combining perception and action primitives in hybrid techniques, which also incorporate domain knowledge or limitations. Using deep learning models or conventional computer vision approaches, sensor data (e.g. RGB images) is processed to extract object representation or features. Machine learning algorithms like imitation learning, reinforcement learning, or a mix of the two are used to learn grasping policies. Using the learnt policies and the sensor data, perception and action modules work together to optimize the grasping performance. The learnt policies are improved over time by adjusting the grip execution based real-time data from sensors.[33]

3.1.5 Robot System

For pick and place tasks, delta robots and parallel kinematics are the usual system. For bin-picking applications, however, serial kinematics and articulated robots are the more popular options. The advantages include, a much bigger workspace, more degrees of freedom for the end-effector or gripper, allowing a wider range of gripping poses and less limitations due to internal collisions. The robot system's main task is to match various positions. The following are the three crucial ones for bin picking systems.

- The position of the object $\mathbf{W}_{P_O}$ characterizes the object's position in relation to the world coordinates.
- The position of the gripper $\mathbf{W}_{P_G}$ characterizes the end-effector's position in relation to the world coordinates.
- The grasping position $\mathbf{G}_{P_O}$ characterizes the position of the object in relation to the position of the gripper. This is particularly crucial for the object's precise placement.

These three positions constitute the closed transformation chain. In other words, the computation of the third pose is easy if the first two are known. These three poses are shown in Fig. 3.1.8 A robot can place its end-effector using one of the two movement techniques: linear (LIN) motion and point-to-point (PTP) motion. Every robot motor moves synchronously when in PTP motion. Generally speaking, figuring out the trajectory of the Tool Center Point (TCP) is difficult when in this mode. The benefit of this motion, however, is its potential to achieve maximum speed. Because of this, this mode is employed to travel great distances between certain poses. The TCP is a crucial point in the LIN motion. In this motion, the TCP travels linearly from one place to another in Cartesian space. Its speed is much slower than in PTP, which is a drawback for confined workspaces. Inverse kinematics calculation and singularity avoidance are the two components of the LIN mode operation. These are articulated robot configurations where the division in the inverse kinematics is close to 0, resulting in high speed rotations of individual axes. these configurations are typically detected by robot controller, which will stop the robot.[75]

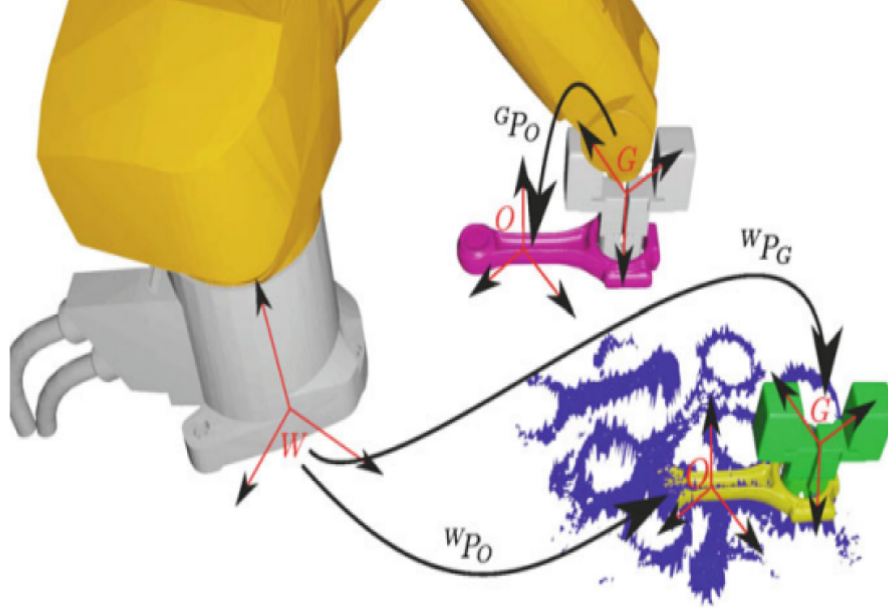


Figure 3.1.8: Important positions the workspace in a bin-picking application. W is the world coordinate system with its origin at the centre of the base of the robot. O is the object coordinate system with its origin at the center of the object. G is the gripper coordinate system with its origin at the center of the gripper. $\mathbf{W}_{P_O}$ is the object's position in relation to the world coordinate system. $\mathbf{W}_{P_G}$ is the end-effector's position in relation to the world coordinate system. $\mathbf{G}_{P_O}$ is the position of the object in relation to the position of the gripper.[8]

3.2 Deep learning-based 3D Image processing

3.2.1 Convolution Neural Network

Inspired from the visual cortex of humans, (CNN or ConvNet) is a type of deep neural network designed for processing data which appear in a grid like manner like images, videos, etc. It was introduced by Yann LeCun et. al in [42] in 1998. It gets its name because of the usage of a special kind of linear mathematical operation called the convolution instead of using matrix multiplication as prevalent in the pre-existing neural networks. The key components of a CNN are - Convolution layer, activation function, pooling layer, loss function, output layer. The principle building block of a CNN is the convolution layer. It consists of a number of learnable filters (kernels) which can be visualised like a cubic block. The success of CNNs can be attributed to three major concepts: sparse interactions, parameter sharing and equivariant representations.

Sparse interactions In traditional fully connected network, matrix multiplication is performed which involves a parameter matrix. The interaction between input and output units are captured by a distinct parameter in the parameter matrix. But CNNs have sparse interactions, i.e. only a subset of units or neurons in a layer is connected to a local region in the preceding layer. This is done by using kernels that are significantly smaller than the input. This implies, less number of parameters need to be stored which reduces the memory consumption and also it is computationally efficient because it has to perform fewer operations. For example, if there are m inputs and n outputs, the fully connected network would need to store $m \times n$ parameters and have a runtime of $O(m \times n)$ time complexity per input. On the other hand, if we restrict the number of connections for each unit to be k , then we would have $k \times n$ and have a runtime of $O(k \times n)$ time complexity per input, where k is quite some fold lesser in magnitude as compared to m . Moreover, since convolution layers are stacked upon one another in a deep convolution network, units in the deeper layers have a larger receptive field, because of its indirect interaction with a larger region in the

input.

Parameter sharing Another important focus behind using the convolution layer was to reduce the number of parameters in a Fully Connected Network (FCN). For example in a 1024×1024 image, a FCN would have over 1 million hidden units, which means it would have over 1 trillion trainable parameters. But the pixels in an image are only locally correlated. So, CNNs make use of the kernels to limit the focus on smaller regions on the image at a time known as the receptive field. This significantly reduces the number of trainable parameters, thus reducing the memory consumption. [12]. These layers perform a convolution operation between the input (eg. image) I and the kernel K to produce an output S known as the feature map as in Eq. 1.

Equivariant Representations A function is said to be equivariant if the input to a function is changed, then the output changes in the similar way. Because of the parameter sharing mechanism, convolutions operations are translation equivariant. When the kernels are applied on an input image, the convolution layers generate a 2D map of where the particular feature occurs in the particular image. Furthermore, a pixel is related to its neighboring pixels to form a meaningful context (create a feature e.g. an edge in an image) but it is not limited to where it can occur throughout the image. Thus it creates multiple filters each of which look for the same feature throughout the image. But it is also to be kept in mind that convolution is not equivariant to other geometric and affine transformations like rotation, scaling, etc. Since convolution operations are commutative in nature, Eq. 1 can also be written as Eq. 2. Typically, Eq. 2 is easier to incorporate into a machine learning library since values for both 'm' and 'n' varies within a small range.[24]

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(m,n)K(i-m,j-n) \quad (1)$$

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i-m,j-n)K(m,n) \quad (2)$$

Activation function The next step in a CNN is to apply an activation function. the purpose of using a non linear activation function is to capture the non-linearity in the data. Moreover if non-linearity is not used in between the multiple layers of a neural network, the network is effectively only one layer deep which is not capable even to capture the non-linearity in real world datasets. Rectified Linear Unit (ReLU) is the most commonly used non-linear activation function used in CNNs because unlike sigmoid function or tanh function it does not penalise "too correct" data points. Another remarkable benefit of using ReLU is its ability to propagate gradient through deep networks with a constant factor. Also it is more memory efficient to use ReLU as it doesn't require to store the ReLU outputs separately as compared to tanh outputs.

Pooling The third step of a CNN is to use the pooling layer. The main purpose of this operation is to make the detection of the features robust to the exact location of the eye (i.e. invariant to small translations). The different types of pooling operations are - max pooling and average pooling. It also helps in reducing the dimensionality of the input without losing too much information. This is done to make the computations faster down the deeper layers of the network. If the pooling operations are performed after every k pixels, then the next layer processes inputs that are k times lesser. Since the number of parameters in a layer are dependent on the size of the input to the layer, it significantly reduces the computational overhead on using pooling operations.

Complete CNN

3.2.2 Autoencoders

Autoencoders are a special type of feedforward neural network in which the output tries to reconstruct the input. It is predominantly used in unsupervised learning for the tasks of dimensionality reduction, learning feature representations, and for data compression. It tries to encode the input data into a more compact representation with lower dimensions called the "code"[24] or "latent space". The idea is that this latent space representation should capture the most vital aspects of the input data. This is done by the first part of the network called the encoder. The second part of the network, the decoder then tries to decode this compressed representation of the input data to reconstruct the original input data as accurately

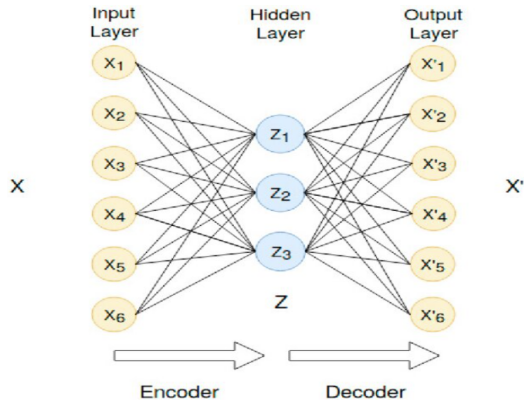


Figure 3.2.1: Architecture of an undercomplete autoencoder.[51]

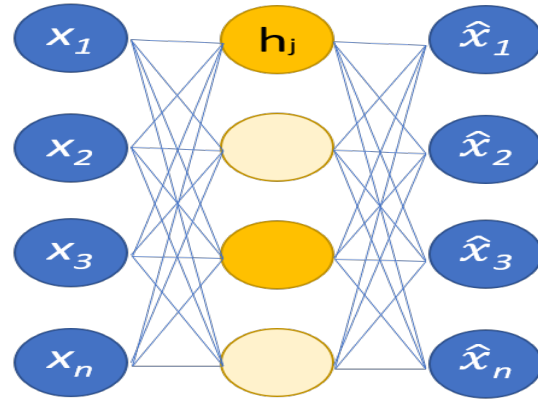


Figure 3.2.2: Architecture of a single layer sparse encoder. The hidden nodes in bright yellow are active, while the ones in light yellow are set to zero, hence inactive[3]

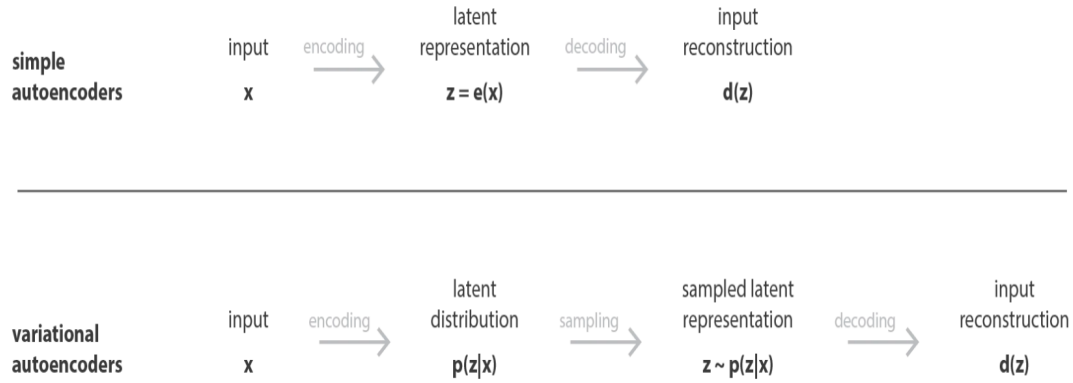


Figure 3.2.3: Difference between a vanilla autoencoder and a Variational Autoencoders (VAE).[79]

as possible. during the training of an autoencoder, the goal is to minimise this reconstruction loss, i.e. the difference between the original input data and the output of the decoder. There are different types of autoencoders - undercomplete autoencoders, convolutional autoencoders, regularized autoencoders, and variational autoencoders,

Undercomplete autoencoders ensures that the dimension of the latent space representation is less than the dimension of the original input data as show in Fig.3.2.1. Because, the output of the decoder to be the exact copy of the input is of no use. Rather, if the dimension of the code is less than that of the input, then it ensures that the autoencoder learns those representative features of the input data which are most salient[24]. Convolutional autoencoders are an extension of traditional autoencoders where convolution layers are used as building blocks in both the encoder and the decoder part of the autoencoder. After training the network, the encoder part is used for ectracting the features of the data and the decoder part is used for the reconstruction of the input data.

Regularized autoencoders are a special type of autoencoders that employs one or more regularisation techniques to prevent the model from overfitting. The different types of regularisations used are - L1 or L2 Regularisation which adds a penalty to the loss function depending on L1 or L2 norm of the model weights As a result of this, the model is capable of learning sparse representation of the training data where many weights are set to a very small value or zero as shown in Fig3.2.2. Dropout is also used as a regulariza-

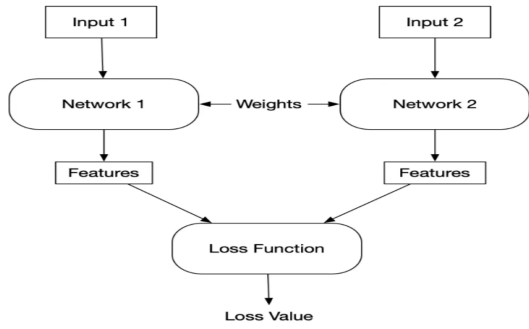


Figure 3.2.4: A generic siamese network.[1]

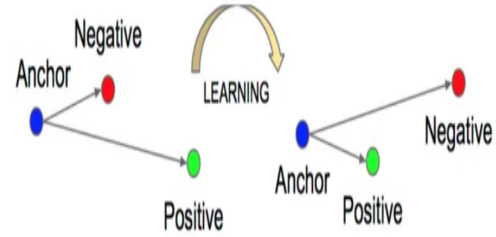


Figure 3.2.5: Before (left) and after (right) minimizing triplet loss function[78]

tion technique where a percentage of the model's nodes are set to zero during training so that the model doesn't rely heavily on any the weight of any particular node, thus preventing overfitting and improving generalisation on unseen data. The previously mentioned regularisation techniques give rise to a variation of regularised autoencoders called the sparse autpencoders[54]. Sometimes noise is also added to the training data or the hidden layers of the model to increase the robustness the model giving rise to denoising autoencoders[81]. One more regularisation technique if to apply a contractive regularization term based on Frobenius norm of the Jacobian matrix of the model's hidden layer activations with respect to the input data[68, 3]. This ensures that a small neighborhood of the input data corresponds to a small neighborhood in the latent space representation, which means small perturbations in the input data leads to small or zero variation in the latent space representation[68, 3]. By doing so, it makes the model more robust to small changes in the input data, thus preventing overfitting.

VAEs are a distinct type of autoencoders which produces latent space representations that are continuous, which allows random sampling and interpolation for the generation of new datapoints. The difference between a vanilla autoencoder and a VAE is shown in Fig.3.2.3. Instead of producing a single vector for the latent space representation, it generates two vectors: a vector of means μ and a vector of standard deviations γ for all datapoints. An encoding is then sampled from a distribution with mean μ and standard deviation γ . Thus even for the same input, i.e. when the mean and the standard deviation are the same, the sampled encoding would vary because of the involvement of the sampling procedure. The mean vector controls the position where the encoding of the input should have it's centre and the standard deviation controls the area over which the sampled encoding is allowed to vary from the mean. As a result of this, the decoder learns to decode not just the encoding of a single point but also the points in the neighborhood and hence a continuous latent space representation is obtained. In order to ensure that the latent space representation satisfies that the nearby encoding are similar to each other on a local scale while also facilitating interpolation on a global scale VAEs jointly optimizes the reconstruction loss and the Kullback–Leibler divergence (KL)[41] loss.[36, 31]

3.2.3 Siamese Network

Traditional deep learning models have shown ground breaking results in a multitude of applications like image recognition and classification, web scraping, speech recognition and caption generation. But it has a limitation that the models tend to perform well if the training and testing data points are from the same distribution. If the model is to be made capable to predict datapoints belonging to an unknown class, we need to re-train the model with an abundant number of training samples from that class and then predict it with unseen samples of that class. This technique can often be computationally expensive and can increase expotentially if the number of classes increases. Also generating a huge number of samples for a new class of objects can be tedious and quite challenging in multiple scenarios. The Siamese network[38, 7] is one such way to mitigate the above mentioned problem. The idea behind siamese networks is that two identical copies of the neural neural network with shared weights process two distinct datapoints as shown in Fig.3.2.4. Because the networks have shared parameters, it ensures that two very similar datapoints

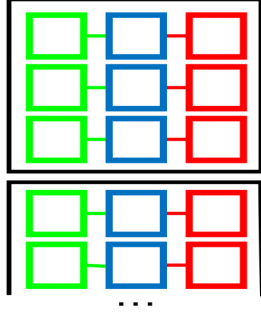


Figure 3.2.6: Minibatches in offline triplet mining. The green boxes are positive datapoints, the blue boxes are anchor datapoints and the red boxes are negative datapoints.

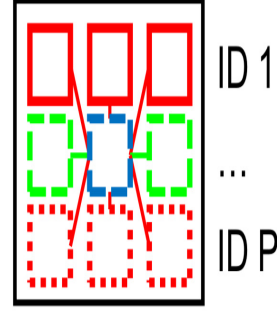


Figure 3.2.7: Triplets constructed only within the minibatch. There are P object classes in the minibatch. The green boxes are positive datapoints, the blue box is the anchor datapoint and the red boxes are negative datapoints.

cannot have too different locations in the feature maps as both the networks evaluate the same function. In siamese networks, the model learns a similarity function, the loss function, which plays the pivotal role in determining how similar or dissimilar the datapoints are to one another. The two main loss functions used in siamese networks are contrastive loss function[7] and triplet loss function[4]. Since the goal of a siamese network is not to perform classification task on datapoints, rather to compute the similarity between them, contrastive loss functions are more suited for this task as compared to cross-entropy loss functions, as it is for classification tasks.

Contrastive loss function evaluates how capable a siamese network is in deciding the similarity between two given datapoints as is given by Eq. 3[1]

$$\mathcal{L}_{\text{con}}(Y, D_w) = (1 - Y) \frac{1}{2} (D_w)^2 + (Y) \frac{1}{2} \max(0, m - D_w)^2 \quad (3)$$

where $\mathcal{L}_{\text{con}}$ is the calculated contrastive loss, D_w is the Euclidean distance between the outputs of the twin networks[1]. Y takes the value 0 or 1. If the two datapoints belong to the same class then Y takes the value 0, otherwise it takes the value 1[1]. $\max()$ is a function to choose the higher of the two values, 0 or $m - D_w$ where m is the margin and has a value greater than 0[1]. The usage of a margin in this equation ensures that the pair of datapoints which are dissimilar beyond this margin value are excluded while calculating the loss function[1]. The euclidean distance between the embeddings of the two datapoints are given by the Eq. 4[1]

$$D_w(X_1, X_2) = \sqrt{\{G_w(X_1) - G_w(X_2)\}^2} \quad (4)$$

where $G_w(X_i)$ is the output of the twin network such that $i \in \{1, 2\}$ as shown in Fig.3.2.4.

Triplet loss function[86] trains the siamese network in a way that a triplet of datapoints are presented to the siamese network. by applying triplet loss the model tries to group the datapoints similar to the anchor datapoint, close to one another while increasing the distance between the anchor datapoint and the datapoints dissimilar to it as shown in Fig.3.2.5. In order to do so, a positive value called margin is used which increases the distance between dissimilar datapoints and also eliminates the output of any trivial solution. Euclidean distance or cosine distance is used as the distance metric while calculating the triplet loss and shown in Eq. 5[28, 1]

$$\mathcal{L}_{\text{tri}}(a, p, n) = \sum_{\substack{a, p, n \\ y_a = y_p \neq y_n}} \max(0, d(a, p) - d(a, n) + \text{margin}) \quad (5)$$

where y_x is the class label for the datapoint x for $x \in \{a, p, n\}$, a is the anchor datapoint, p is the positive example having the same class label as the anchor datapoint and n is the negative example having a different

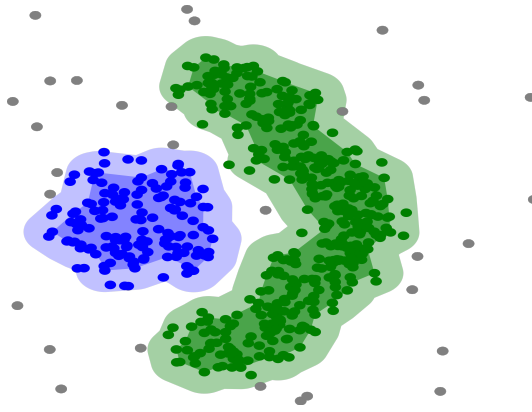


Figure 3.3.1: DBSCAN can find non-linearly separable non-convex clusters.[16]

class label as compared to the anchor datapoint. but there are some practical problems related to the use of triplet loss function. The number of possible triplets grow cubically with the increase in the size of the dataset. Also most of the triplets are often uninformative. Thus mining "hard triplets" or the ones crucial for the learning of the model becomes a challenging task. As per the definition of the triplet loss function, the triplets can be of three types: easy triplets, hard triplets and medium hard triplets. The triplets which satisfy the condition $d(a, p) + margin < d(a, n)$ results to the loss value being 0, hence are called the easy triplets.[78] For the triplet for which the negative datapoint is closer to the anchor as compared to the positive datapoint i.e. $d(a, n) < d(a, p)$ are the examples of hard triplets.[78] And the medium hard triplets are the ones where the negative datapoints are not closer to the anchor datapoint as compared to the positive datapoint but still results in positive loss i.e. $d(a, p) < d(a, n) < d(a, p) + margin$. [78] Thus the medium hard triplets are the most essential for the model training as they constitute for the most useful information. Two triplet mining procedures are used to mitigate the issue of excessive computational overhead with increase in dataset size. They are offline triplet mining and online triplet mining. In offline hard triplet mining, at first the dataset is manually processed to find hard triplets and then they are used for training the model. But manually finding the hard triplets can be quite tedious. Online triplet mining is a better approach as compared to the one mentioned above. The model is trained in minibatches of the dataset as shown in Fig.3.2.6. Using only the triplets that were mined from the dataset could be a wasteful design choice in such scenario. So in a minibatch, it contains way more potential triplets than the ones that were mined. So each member of another triplet becomes an additional negative datapoint example. But both hard positive and hard negative datapoints are required for training. An even better design to mitigate that would be to choose let's say K datapoints from P different classes in the dataset, where the K positive examples serve as hard positives while the rest of the datapoints in the minibatch serve as hard negatives as shown in Fig.3.2.7[28].

Thus siamese networks are very robust to the class imbalance in dataset which is very common in real datasets. Since it depends on very few datapoints of a particular object class, it has good generalisation capabilities for unseen datapoints in the future. They are also very robust to small perturbations in the datapoints due to difference in lighting conditions or orientation or background, as they keep similar objects close to each other even if the datapoints are slightly different. Since the siamese networks places similar datapoints together while increasing the distance between dissimilar datapoints, they are often useful for learning semantic similarity within the datapoints. However despite its benefit, training a siamese network takes longer as compared to traditional neural networks because it involves learning from quadratic pair of datapoints.[1]

3.3 Clustering Algorithms

When the dataset does not have any labelled data, then it is said to be unsupervised learning. In this case, there is no ground truth available to measure the correctness of the outputs generated by the Machine Learn-

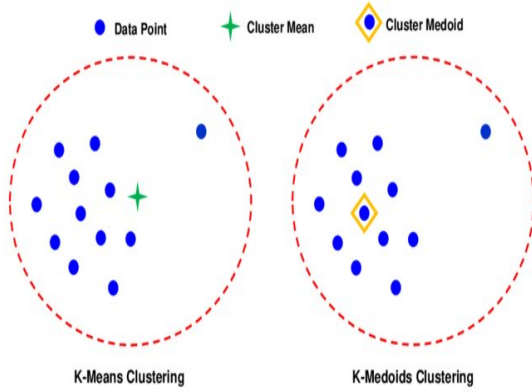


Figure 3.3.2: Difference between K-Means and K-Medoids.[20]

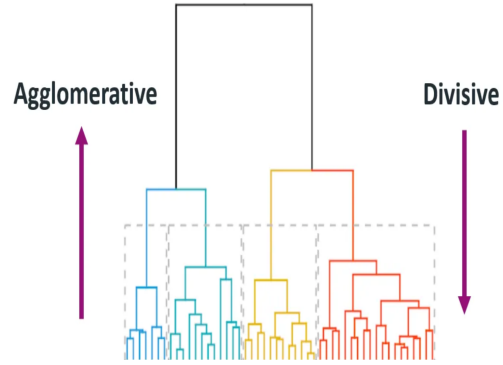


Figure 3.3.3: A dendrogram in hierarchical clustering.[29]

ing (ML) models. The primary focus of unsupervised learning is to find hidden and interesting pattern in the data. Unsupervised learning is of utmost importance in the Artificial Intelligence (AI) world as several real world datasets do not have available annotations, which requires a lot of human effort. Unsupervised learning algorithms can be broadly categorized into the following domains-clustering, dimensionality reduction, and association analysis. Clustering algorithms aim at grouping unlabelled data into groups or clusters based on how similar or dissimilar the datapoints are to one another. It can reveal underlying hidden pattern in the data and is used in applications like image segmentation, fraud detection, etc. Dimensionality reduction reduces the number of irrelevant features or dimensions of the dataset. The inclusion of more features does help in the better representation of the dataset but it also significantly increases the memory consumption and complexity to work with it. Also it is often difficult to visualise real-life datasets with too many features. Association analysis is a rule-based unsupervised learning method that reveals the relationship between attributes in the dataset. It is used in applications like market analysis, intrusion detection, etc. Clustering algorithms can be broadly classified into the following categories: density-based, distribution-based, graph-theory based and hierarchical-based.

3.3.1 Density-based Clustering

In density-based clustering, the algorithm looks for areas of high concentration of the datapoints and groups them as a cluster. The benefit of this algorithm is that the shape a cluster can be is not limited and hence doesn't necessarily have to be convex in nature as show in Fig. 3.3.1. Moreover, these algorithms are also very robust to outliers as they do not force the outliers to belong to any category but are rather ignored as it is unlikely that outliers can form an area of high concentration of datapoints. I have carried on the experiments on this thesis on two such density-based clustering algorithms called DBSCAN[21] and HDBSCAN algorithms[49]. The available scikit-learn implementations were used for this purpose[61]. The DBSCAN[21] algorithm does not require the users to define the number of clusters to be generated which doesn't force dissimilar datapoints to belong to the same cluster. However, this algorithm was still sensitive to two parameters ϵ , the maximum distance between the datapoints to be considered in the same cluster and the minimum number of samples in the cluster which the user needs to define [61]. But finding an optimum value for these parameters often require domain expertise and are dependent on the data. The HDBSCAN[49] algorithm mitigates this issue and thus does not require the user to define these two parameters. It is a hierarchical density-based clustering that performs the DBSCAN algorithm over multiple ϵ values to find the most stable result.

3.3.2 Distribution-based Clustering

In distribution-based algorithms, a datapoint is said to be a member of the cluster depending on the probability of it's membership to the cluster. The more the distance of a point increases from the centre of

the cluster, the less is its probability of belonging to that cluster. Centroid-based algorithms group the datapoints based on some initial cluster centres. Once all the datapoints are softly assigned to some cluster membership, the cluster centres are recalculated and this process is iterated until convergence. These algorithms are sensitive to the initial parameters like the cluster centres chosen in the first step. Another major disadvantage of these clustering algorithms is that they always form spherical clusters. The user also needs to define the number of clusters the dataset is to be grouped in, which makes it sensitive to outliers. However, these algorithms can be executed very fast and we have used two such algorithms in our experiments- K-Means[34] and K-Medoids[35]. In K-Means, the mean of the datapoints of the clusters is assigned as the cluster-centroid. It might not be an actual datapoint in the dataset, rather a blurred, noisy average of a datapoint in the cluster. On the contrary, the K-Medoids algorithm assigns an actual datapoint of the dataset, that is most centrally located as the cluster centroid as shown in Fig.3.3.2. Thus K-Medoids is more robust to outliers and noises as compared to K-Means.

3.3.3 Hierarchical-based Clustering

Hierarchical-based clustering algorithm that form a hierarchy of clusters. Datapoints in a cluster are more similar to each other as compared to other groups. This hierarchy of clusters is visualised by a hierarchy tree called dendograms. Hierarchical clustering algorithms can be of two types - agglomerative and divisive. In agglomerative clustering, each datapoint is considered as a separate cluster in the first step. Then these clusters are merged into one another until only one cluster remains. Thus, at the end, the last level cluster consists of all the datapoints in the dataset. The divisive method is the reverse procedure of the agglomerative method. In the beginning, all the datapoints are considered to be in a single cluster and gradually the cluster is broken into smaller clusters, until each cluster consists of only one datapoint. A visual representation of a dendogram has been shown in Fig.3.3.3

3.3.4 Graph theory-based Clustering

Spectral clustering is a powerful machine learning tool which divides datapoints into meaningful groups based on their similarity. The concept of spectral clustering has its root in the graph theory. It tries to determine the node communities in a graph by looking at the edges that connect them. It is specially useful when the dataset cannot be linearly-separated. The affinity matrix of the datapoints, which gauges the pairwise similarity between points, is the basis for spectral clustering. It works by creating a graph representation of the data, in which datapoints are represented by nodes and the degree of similarity between the points indicated by edges. After that the eigenvectors of a matrix are obtained from this graph which is used to create the embeddings of the datapoints in a lower dimensional space. The embedded points are then divided into clusters by using a common clustering algorithm, usually the distributed based clustering algorithm K-means, where each entry in the embedding matrix is the representation of the datapoints in the lower dimensional space. Spectral clustering has shown promising results with datasets that have complex structures and non-linear separability, which makes it a useful tool in variety of fields, such as social network analysis, image segmentation, and dimensionality reduction.[83]. A comparison between the results of k-means and spectral clustering in the presence of non-globular clusters is shown in fig. 3.3.4

3.4 Evaluation metrics for Clustering Algorithms

The ground truth or labels are available in supervised learning. So to determine the quality of the clustering algorithm we can calculate the level of accuracy by comparing the labels predicted by a model to the ground truth. But most real world datasets do not have ground truth. Therefore, it makes it challenging in unsupervised learning paradigm to determine the quality of the clustering algorithm. But a good clustering algorithm is still expected to have little difference between the objects in a cluster i.e. small within-cluster variance and more difference between objects from different clusters i.e. large between-cluster variance. Thus the evaluation metrics used in the literature could be categorised into two broad groups: extrinsic measures and intrinsic measures, depending on the availability of the ground truth. Extrinsic measures, when the ground truth is available uses metrics like Rand Index, Mutual Information, V-measure and Fowlkes-Mallows Scores. Intrinsic measures, when the ground truth is not available mostly uses Silhouette Score, Calinski Harabaz Index and David Bouldin index.

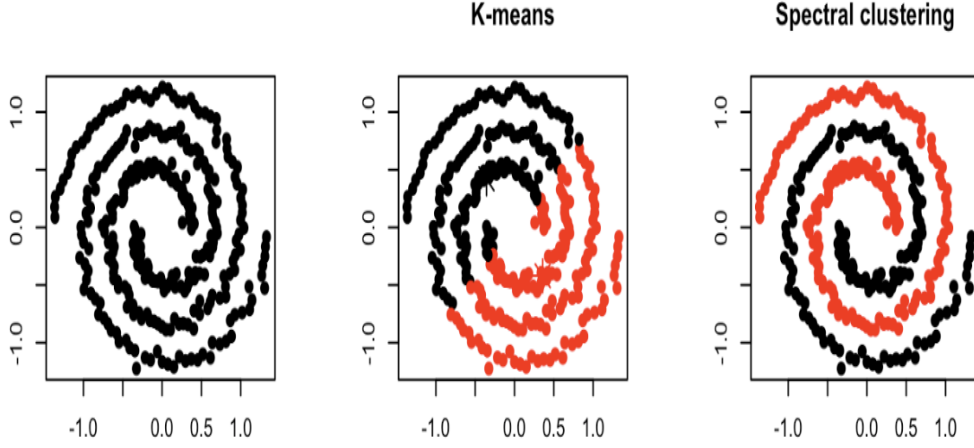


Figure 3.3.4: A comparison between the results of K-means clustering and Spectral clustering. The image in the left shows the points that need to be clustered. [57]

3.4.1 Silhouette Score

Silhouette Score is used to calculate the distance between points within a cluster as compared to its distance from the points in the neighbouring cluster and is given by Eq. 6

$$S_c = \frac{(n_c - i_c)}{\max(n_c, i_c)} \quad (6)$$

where S_c is the Silhouette Coefficient, n_c is the average of the nearest-cluster distance of each datapoint and i_c is the average distance between the datapoint within the cluster. The Silhouette coefficient lies within the range $[-1, 1]$, where the more closer the coefficient is to 1, the better is the clustering results. When the Silhouette Coefficient is close to 0, it implies that the datapoints are on or very close to the decision boundary between adjacent clusters, whereas negative values imply that the datapoints have been wrongly classified. But the drawbacks of using this metric is that it tends to favour dense and convex clusters. Since it takes into account, the average of the distance within a cluster as well as between adjacent clusters, it doesn't perform well if different clusters have a significant difference in their respective sizes, which is quite a common phenomenon in real world datasets. The chosen distance metric for the calculation of the Silhouette score also has a significant effect on it and can lead to very different results.

3.4.2 Calinski-Harabasz Index

Calinski-Harabasz Index (C-H Index) or the Variance Ratio Criterion is used to measure the dispersion between the datapoints within a cluster as compared to in-between clusters as given by Eq. 7

$$S_{CH} = \frac{tr(B_k)}{tr(W_k)} \times \frac{(n_E - k)}{k - 1} \quad (7)$$

where S_{CH} is the C-H Index, $tr(B_k)$ is the trace of the in-between cluster dispersion matrix as in Eq. 8 and $tr(W_k)$ is the trace of the within-cluster dispersion matrix as in Eq. 9

$$B_k = \sum_{q=1}^k n_q (c_q - c_o)(c_q - c_o)^T \quad (8)$$

$$W_k = \sum_{q=1}^k \sum_{x \in C_q} (x - c_q)(x - c_q)^T \quad (9)$$

where c_q is the centroid of the cluster q and c_o is the overall centroid of the datapoints. B_k calculates the quality of the separation between two clusters, i.e., higher the value of B_k , better it is. W_k calculates the distance of a datapoint (s) from the centroid of the cluster, and thus measures the cohesiveness or compactness of the datapoints within a cluster, i.e, smaller the value of W_k , better it is. Distribution-based clustering algorithms like K-Means tries to minimise the value of W_k . Thus according to the equation of C-H Index in Eq. 7, the higher the C-H index, the better is the clustering results. Because sum-of squared Euclidean distance is used to calculate the distance of the datapoints from their respective cluster centroids, C-H Index tends to favour convex clusters, which is a major drawback of using this metric as in Silhouette Coefficient.[9]

3.4.3 David-Bouldin index

David-Bouldin index is another metric used for evaluating clustering algorithms, which uses only the intrinsic features of the dataset and doesn't require any ground truth labels. David and Bouldin proposed R , a measure to calculate cluster similarity that satisfied the following conditions[80]:

- $R(S_i, S_j, M_{ij}) \geq 0$
- $R(S_i, S_j, M_{ij}) = R(S_j, S_i, M_{ij})$
- $R(S_i, S_j, M_{ij}) = 0$ iff $S_i = S_j$
- if $S_i = S_j$ and $M_{ij} < M_{ik}$, then $R(S_i, S_j, M_{ij}) > R(S_i, S_k, M_{ik})$
- if $M_{ij} = M_{ik}$ and $S_j > S_i$, then $R(S_i, S_j, M_{ij}) > R(S_i, S_k, M_{ik})$

where M_{ij} is the distance between the centroids of the clusters i and j and S_i and S_j are their respective dispersion measures. The conditions enforce that the cluster similarity measure R is non-negative and symmetrical in nature. It becomes 0 if and only if the dispersions of the two clusters disappear. If the distance M_{ij} increases even if S_i, S_j are constant, then the cluster similarity measure R decreases and *vice versa*. Also if M_{ij} remains constant and S_i, S_j increases, then R increases. Thus the David-Bouldin index was defined as the average of the similarities of each cluster as in Eq. 10, using the same notations as the authors.

$$\bar{R} = \frac{1}{C} \sum_{i=1}^C R_i \quad (10)$$

where C is the total number of clusters and R_i is the $\max_{j \neq i} R_{i,j}$. Thus, $\bar{R}$ or the David-Bouldin index is the average of the similarity measures R of each cluster with its most similar cluster. Thus, David-Bouldin index has a lower bound of 0, while a higher value means bad clustering. In other words, the less is the average similarity, the better is the clustering results. Because of this, it acts as a good measure to decide on the number of clusters that actually exists in the dataset. Thus, this index can be used to determine the number of clusters in distribution-based clustering algorithms like K-means. $R_{i,j}$ is more formally defined as in Eq. 11.

$$R_{i,j} = \frac{S_i + S_j}{M_{ij}} \quad (11)$$

The dispersion measure of a cluster i is defined as in Eq. 12

$$S_i = \left\{ \frac{1}{T_i} \sum_{j=1}^{T_i} \|X_j - A_i\|^q \right\}^{\frac{1}{q}} \quad (12)$$

where T_i are the number of datapoints in the cluster i , and X_j are the respective vector representations of the datapoints in the cluster i . For $q = 1$, The dispersion measure S_i is the average Euclidean distance of the vector representation of the datapoints from the centroid of the cluster i , while for $q = 2$, it is the standard deviation of the metric about the datapoints in the cluster with respect to its centroid [80]. It is different as compared to the distance defined between the centroids of the clusters which are defined as in Eq. 13

$$M_{i,j} = \left\{ \sum_{k=1}^N \|a_{ki} - a_{kj}\|^p \right\}^{\frac{1}{p}} \quad (13)$$

where a_{ki} is the k -th component of the vector representation of a_i which is the centroid of the cluster i . $M_{i,j}$ is the Minkowski metric of the centroids of the clusters i and j , i.e., when $p = 1$, it is the Manhattan distance and when $p = 2$ it is the euclidean distance between the centroids. When both p and q are 2, then $R_{i,j}$ is the Fisher similarity measure between the two clusters[80]. But this metric too is not without its limitations. It is not robust to noise and outliers, thus giving an incorrect sense of good clustering. This index too is biased towards spherical clusters with similar sizes, thus making it inappropriate for many real world applications. Moreover, since it is a metric of the clustering results on a global level and gives no information about the individual clusters, even a single "too good" or "too bad" cluster can heavily influence the final result.

3.4.4 Density Based Cluster Validation Index

Thus, we could see how even the evaluation metrics using just the intrinsic features of the dataset are still not efficient in the presence of noise, outliers or non-convex clusters. As pointed out by Jain and Dubes, "without a strong effort in this direction, cluster analysis will remain a black art accessible only to those true believers who have experience and great courage" [32]. The fact that this statement holds even today, 35 years after he said it is even more astonishing. To tackle the problem of inefficient evaluation metrics for non-convex clusters or where the different clusters have a difference in size Moulavi *et. al.* proposed the DBCV Index [52]. The authors have utilised the theory of Hartigan's model of density-contour trees[26]. The least dense region within a cluster and the most dense region outside a cluster is calculated with it which is analogous to computing the within cluster and between cluster density connectedness. As per the definition of Density Contour Trees[26] by Hartigan, density based clusters are regions of high density which are separated by low density regions. Having such model as a backdrop, it is expected that a good clustering algorithm which generate clusters such that the least dense region within a cluster is still more dense as compared to regions surrounding a cluster. Traditional intrinsic measure based evaluation metrics uses distance between the datapoints to compute cluster-variance and then incorporates it with the separation between the clusters to eventually compute the quality of the clustering algorithm. But this was not the target for DBCV. So they proposed a metric which was defined by densities instead of distances. DBCV wanted to take into account both the density and the shape of the cluster. So at first, the authors defined the concept of "all-points-core-distance" ($a_{ptscoredist}$). It was the inverse of the density of each object with respect to all other objects inside its cluster as given in Eq. 14.

$$a_{ptscoredist}(\mathbf{o}) = \left\{ \frac{\sum_{i=2}^{n_i} \left(\frac{1}{KNN(\mathbf{o}, i)} \right)^d}{n_i - 1} \right\}^{-\frac{1}{d}} \quad (14)$$

Using the above-mentioned equation, they defined a symmetric reachability distance which was then utilized to generate the Minimum Spanning Trees (MST) inside each clusters. Because the MSTs are built on the transformed space of symmetric reachability distances, it is capable of capturing both the density and shape of the cluster. On using these MSTs, they were able to find the least dense region within each of the clusters and the most dense regions between the pair of clusters. As per the definitions provided by the authors, $\mathbf{O} = o_1, o_2, \dots, o_n$ is the dataset containing n objects in the $\mathbb{R}^d$ feature space. $\mathbf{Dist}$ is an $n \times n$ matrix of pairwise distance $d(o_p, o_q)$ between the objects of the dataset, such that $o_p, o_q \in \mathbf{O}$. $KNN(\mathbf{o}, i)$ is the distance between the object $\mathbf{o}$ and its i^{th} nearest neighbor. $C = (C_i, N) 1 \leq i \leq l$ is the results returned by the clustering algorithms having l clusters and a set of N noise objects. n_i is the size of the i^{th} cluster and n_N is the number of objects in that particular cluster. To get an estimate of the density of an object within a cluster, traditional approaches use the notion of taking the inverse of the threshold distance required to find K objects within this threshold[26, 10]. But a drawback of this method would be that one would need to decide the value of the parameter K , which is not desirable. Moreover, with this approach, the density of the object is dependent to its distance from only one other object i.e. the k^{th} nearest neighbor. This approach is not as robust as compared to considering more objects in its neighborhood as done in Gaussian kernel density estimate. Thus in order to propose a more robust density estimate, the authors proposed the use of a new parameterless core distance which can be elucidated as the inverse of a density estimate and thus can be used in defining the Mutual Reachability Distance (MRD). So now instead of using just one point in the cluster, all the points are taken into account such that the objects closer to it contribute to the

density more as compared to the objects further away. We can see in Eq. 14, that the inverse of the KNN distances raised to the power of dimensions has been calculated such that closer objects gets a higher weight as compared to further objects. This effect can further be increased by using squared Euclidean distance instead of Euclidean distances. Also, as per the definition of MRD[43], the core distance of an object is compared to the distances of the object to other objects in the cluster. Hence, the core distance needs to be comparable to these distances. With the help of the following proof the authors have could rightfully conclude that for each object $\mathbf{o}$ in the cluster, the "all-points-core-distance" $a_{ptscoredist}(\mathbf{o})$ lies between the second and the last nearest neighbor distance of the object[52] as in Eq. 15.

Proof. We have $\forall 2 \leq i \leq n$,

$$KNN(\mathbf{o}, 2) \leq KNN(\mathbf{o}, i) \implies \left(\frac{1}{KNN(\mathbf{o}, i)} \right)^d \leq \left\{ \frac{1}{KNN(\mathbf{o}, 2)} \right\}^d$$

Therefore,

$$\begin{aligned} \sum_{i=2}^n \left(\frac{1}{KNN(\mathbf{o}, i)} \right)^d &\leq (n-1) \left(\frac{1}{KNN(\mathbf{o}, 2)} \right)^d \\ \left((n-1) \frac{1}{KNN(\mathbf{o}, 2)} \right)^{-\frac{1}{d}} &\leq \left(\frac{\sum_{i=2}^n \left(\frac{1}{KNN(\mathbf{o}, i)} \right)^d}{n-1} \right)^{-\frac{1}{d}} \\ KNN(\mathbf{o}, 2) &\leq a_{ptscoredist}(\mathbf{o}) \end{aligned}$$

The upper bound of the equation can be proved similarly giving rise to

$$a_{ptscoredist}(\mathbf{o}) \leq KNN(\mathbf{o}, n)$$

□

$$KNN(\mathbf{o}, 2) \leq a_{ptscoredist}(\mathbf{o}) \leq KNN(\mathbf{o}, n) \quad (15)$$

Moreover, the core distance is approximated to the distance of the object to its k^{th} nearest neighbor such that k is not too large. This means, that only a small neighborhood of the object is to be considered. Furthermore, let n objects be uniformly distributed random variables in a d -dimensional unit hypersphere. Then the core distance of object $\mathbf{o}$ is given by Eq. 16 which can be proved as following.

$$a_{ptscoredist}(\mathbf{o}) = \ln(n)^{-\frac{1}{d}} \quad (16)$$

Proof. First the authors proved the following:

$$\forall 1 \leq i \leq j \leq n, \frac{KNN(\mathbf{o}, i)}{KNN(\mathbf{o}, j)} \approx \left(\frac{i-1}{j-1} \right)^{\frac{1}{d}} \quad (17)$$

where $KNN(\mathbf{o}, i)$ is the distance between the object $\mathbf{o}$ and its i_{th} nearest neighbor in the hypersphere. Now, the minimum volume of the hypersphere containing the i_{th} nearest neighbor of the object when the object is at the center of the hypersphere is given by Eq. 18

$$V_i \approx \frac{\pi^{\frac{d}{2}}}{\Gamma(\frac{d}{2} + 1)} r_i^d \quad (18)$$

where V_i is the minimum volume of the hypersphere with radius r_i that contains $KNN(\mathbf{o}, i)$. So, we get $KNN(\mathbf{o}, i) = r_i$. So in case of a unit hypersphere, $KNN(\mathbf{o}, i) \approx 1$. Also, because the objects are uniformly

distributed, the ratio of the volume of the hyperspheres which contains the i_{th} and j_{th} nearest neighbors to the object $\mathbf{o}$ is $\frac{V_i}{V_j} \approx \frac{i-1}{j-1}$

$$\frac{V_i}{V_j} \approx \left(\frac{r_i}{r_j} \right)^d \quad (19)$$

$$\frac{r_i}{r_j} = \frac{KNN(\mathbf{o}, i)}{KNN(\mathbf{o}, j)} \approx \left(\frac{i-1}{j-1} \right)^{\frac{1}{d}} \quad (20)$$

From Eq. 17, we have,

$$\frac{I}{KNN(\mathbf{o}, i)} = \left(\frac{n-I}{i-I} \right)^{\frac{I}{d}} \frac{I}{KNN(\mathbf{o}, n)}$$

Therefore,

$$a_{ptscoredist}(\mathbf{o}) = \left(\frac{\sum_{i=2}^n \left(\frac{n-I}{i-I} \right)^{\frac{I}{d}} \frac{I}{KNN(\mathbf{o}, n)}}{n-I} \right)^d^{-\frac{1}{d}}$$

Since we have $KNN(\mathbf{o}, n) \approx 1$, therefore,

$$\begin{aligned} a_{ptscoredist}(\mathbf{o}) &= \left(\frac{\sum_{i=2}^n \left(\frac{n-I}{i-I} \right)^{\frac{I}{d}}}{n-I} \right)^d^{-\frac{1}{d}} \\ &= \left(\frac{\sum_{i=2}^n \left(\frac{n-I}{i-I} \right)}{n-I} \right)^{-\frac{I}{d}} \\ &= \left((n-I) \frac{\sum_{i=2}^n \left(\frac{1}{i-I} \right)}{n-I} \right)^{-\frac{I}{d}} \\ &= \left(\sum_{i=2}^n \left(\frac{1}{i-I} \right) \right)^{-\frac{I}{d}} \\ &= (\ln(n-I) + \gamma + \epsilon)^{-\frac{I}{d}} \\ &\approx \ln(n)^{-\frac{I}{d}} \end{aligned}$$

where $\sum_{i=1}^n \left(\frac{1}{i} \right)$ is a harmonic series. Furthermore, from Eq. 17,

$$\begin{aligned} \frac{KNN(\mathbf{o}, \frac{n}{\ln(n)})}{KNN(\mathbf{o}, n)} &\approx \left(\frac{\frac{n}{\ln(n)} - I}{n-I} \right)^{\frac{I}{d}} \\ &\approx \left(\frac{\frac{n}{\ln(n)}}{n} \right)^{\frac{I}{d}} \\ \frac{KNN(\mathbf{o}, \frac{n}{\ln(n)})}{KNN(\mathbf{o}, n)} &\approx \ln(n)^{-\frac{I}{d}} \end{aligned}$$

Since $KNN(\mathbf{o}, n) \approx 1$,

$$KNN\left(\mathbf{o}, \frac{n}{\ln(n)}\right) \approx \ln(n)^{-\frac{I}{d}}$$

Therefore, $a_{ptscoredist}(\mathbf{o}) \approx \ln(n)^{-\frac{1}{d}} \approx KNN(\mathbf{o}, j)$. where j is the closest natural number to $\frac{n}{\ln(n)}$. $\square$

This proves that the core distance of the object $\mathbf{o}$ is approximately equal to the same nearest neighbor distance and is independent of the dimension of the data space, if the objects are uniformly distributed. If the dataset doesn't have a uniform distribution, which is the case for most real world datasets, only the first property holds i.e. the density of the object doesn't depend on just one other object but on all objects in the cluster.

Now the $a_{ptscoredist}$ was used to compute the symmetric reachability distance which I would delve into deeper in the following paragraphs. As per the authors, the MRD between two objects $\mathbf{o}_i$ and $\mathbf{o}_j$ as in Eq. 21.

$$d_{mreach}(\mathbf{o}_i, \mathbf{o}_j) = \max\{a_{ptscoredist}(\mathbf{o}_i), a_{ptscoredist}(\mathbf{o}_j), d(\mathbf{o}_i, \mathbf{o}_j)\} \quad (21)$$

where $d(\mathbf{o}_i, \mathbf{o}_j)$ is the Euclidean distance between object $\mathbf{o}_i$ and $\mathbf{o}_j$. Once the MRD was computed, it could now be used to generate the mutual reachability graph where the objects are the different vertices and the graph and the MRD calculated above are the edge weights between the pair of vertices. Now with the set of objects $\mathbf{O}$ and the mutual reachability graph G , we get the MST of graph G as MST_{MRD} . So if we now quickly overview the process of DBCV, for a single cluster C_i having objects O , we first calculate the $a_{ptscoredist}$ of each object in the cluster. This is then used to compute the MRD between all pair of objects in the cluster. Depending on the MRD, the MST is generated. This process is then performed for all the clusters generated by the clustering algorithm, eventually giving rise to l MSTs if the clustering algorithm returns l clusters in total. Thus, we can notice that the number of MSTs increase linearly with the increase in the number of clusters. Also, the size of the tree depends on the number of objects in the clusters and thus can blow up exponentially in case of very large datasets having a lot of elements in each cluster.

From the MSTs generated above, the authors have crafted a new measure to validate the quality of the clustering algorithm depending on the density sparseness and density separation as explained below. The Density Sparseness of a Cluster (DSC) C_i can be visualized as the least dense region within the cluster and thus can be obtained from the maximum edge weight of the MST_{MRD} of that particular cluster. On the other hand, Density Separation of a Pair of Clusters (DSPC) C_i and C_j where $1 \leq i, j \leq l, i \neq j$ can be visualized as the region of highest density in between adjacent clusters and can be formulated as the minimum MRD between the objects of a cluster to the objects of it's neighboring cluster. The DSC and the DSPC together then give the validity index of a cluster as in Eq. 22.

$$V_C(C_i) = \frac{\min_{1 \leq j \leq l, i \neq j} (DSPC(C_i, C_j)) - DSC(C_i)}{\max \left(\min_{1 \leq j \leq l, i \neq j} (DSPC(C_i, C_j)) - DSC(C_i) \right)} \quad (22)$$

Once we have the validity index of each cluster, the final DBCV score of the clustering algorithm can be defined as the weighted average of the validity index of all the clusters in C generated by the clustering algorithm as in Eq. 23. It is to be noted that even if noisy elements aren't explicitly present in the formula as a cluster, but it does however implicitly affects the overall score as the weighted average takes into consideration both the cardinality of individual clusters and well as the total number of elements in the dataset, which includes noise and outliers.

$$DBCVC(C) = \sum_{i=1}^l \frac{|C_i|}{|O|} V_C(C_i) \quad (23)$$

If a cluster has better density compactness within the cluster as compared to density separation in between adjacent clusters, then the validity index of the cluster would take a positive value. On the other hand, if the density outside the cluster separating it from adjacent clusters is more than density compactedness inside the cluster then it would take a negative value. Thus, the DBCV score can vary in the range $[-1, 1]$ where greater DBCV score indicates to a better density-based clustering algorithm.

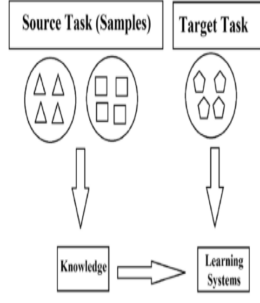


Figure 3.5.1: Model trained for a task in the similar domain is re-used for a new task.[30]

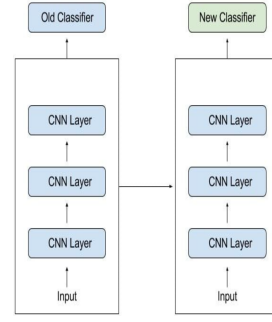


Figure 3.5.2: A model trained on a different task is adapted to be used for a new task.[89]

3.5 Transfer learning

Transfer learning is an optimization in machine learning techniques where a model trained for a particular task let's say task 1 is used for another task, task 2. It uses the knowledge learnt by the model in the previous task to be adapted for the new task. Transfer learning has its benefits in a multitude of applications because real world problems doesn't necessarily always have an abundant amount of labelled data which can be used to train a model from scratch[87]. It can be used for tasks which intend to solve different but related problems[87]. In other words, if the domain of the data for task 2 is similar to that of task 1, transfer learning can be used as shown in Fig.3.5.1. Several approaches have been adapted to use transfer learning. Let's say we want to solve task T_1 , but we don't have adequate data to train a deep neural network for it. But we have ample data to solve a similar task T_2 . So we can train a model for task T_2 and then use it for task T_1 . We might need to re-train only the later layers or all the layers of the model, but that heavily depends on the problem at hand. Another approach could be to use an existing pre-trained network. There are a lot of available open-source models which can be re-trained and re-used depending on the problem. Some of the popular models that are used as pre-trained models in transfer learning are Oxford VGG Model[73], Google Inception model[77], Microsoft ResNet model[27]. Another approach could be to use transfer learning for the purpose of feature extraction. Transfer learning has found a lot of application in the domain of computer vision and natural language processing because it requires a lot of data for training such complex models from scratch. In the domain of computer vision, we know that the early layers of a deep neural network focuses on learning the low level features like the detection of edges, while the middle layers focus on learning the mid level features like the detection of shapes. In transfer learning, these pre-trained early and middle layers could be used verbatim and only the latter layers which focus on the high level features could be re-trained for the new task as shown in fig3.5.2. Transfer learning has numerous advantages like significant reduction in the amount of training time[47], improvement in performance for some neural networks[59] and less reliance on a huge amount of data[19, 40, 96].

3.5.1 Definition

According to [87, 58], transfer learning can be formally defined as the following. A domain $\mathcal{D}$ of the input data, with the feature space $\mathcal{X}$ and a marginal probability distribution $P(X)$ such that $X = \{x_1, \dots, x_n\} \in \mathcal{X}$, where n is the number of feature vectors in X . In the given input data domain $\mathcal{D}$, the task $\mathcal{T}$ has a label space $\mathcal{Y}$ and a predictive function $f(\cdot)$ which is learned from the feature vector and label pairs (x_i, y_i) such that $x_i \in \mathcal{X}$ and $y_i \in \mathcal{Y}$ and $f(x)$ is the learner that predicts the label value for x . As per the above definition, the source domain is defined as $\mathcal{D}_S$ and the corresponding source task as $\mathcal{T}_S$. The target domain is defined as $\mathcal{D}_T$ and the corresponding target task as $\mathcal{T}_T$. Transfer learning aims at improving the target predictive function $f_T(\cdot)$ by using the related information from $\mathcal{D}_S$ and $\mathcal{T}_S$ where $\mathcal{D}_S \neq \mathcal{D}_T$ or $\mathcal{T}_S \neq \mathcal{T}_T$. Since $\mathcal{D}_S = \{\mathcal{X}_S, P(\mathcal{X}_S)\}$ and $\mathcal{D}_T = \{\mathcal{X}_T, P(\mathcal{X}_T)\}$, here the condition $\mathcal{D}_S \neq \mathcal{D}_T$ means that $\mathcal{X}_S \neq \mathcal{X}_T$ and/or $P(\mathcal{X}_S) \neq P(\mathcal{X}_T)$. The scenario where $\mathcal{X}_S \neq \mathcal{X}_T$ in the context of transfer learning is defined as heterogeneous transfer learning. The scenario where $\mathcal{X}_S = \mathcal{X}_T$ is defined as homogeneous transfer learning. The scenario where $P(\mathcal{X}_S) \neq P(\mathcal{X}_T)$ means the source and the target data are from different domains, i.e., they belong

Transfer learning Settings	Related areas	Source Domain Labels	Target Domain Labels	Tasks
Inductive Transfer Learning	Multi-task learning	Available	Available	Regression, Classification
Transfer Learning	Self-taught Learning	Unavailable	Available	
Transductive Transfer Learning	Domain Adaptation, Simple Selection bias, Co-variate Shift	Available	Unavailable	
Unsupervised Transfer Learning		Unavailable	Unavailable	Clustering, Dimensionality Reduction

Table 2: Different settings of transfer learning [58]

to different marginal distributions. Transfer learning algorithms are not expected to give optimal results in this case. Referring back to the definition of transfer learning, when $\mathcal{T}_S \neq \mathcal{T}_T$ and it was defined that $\mathcal{T} = \{(Y), P(Y|X)\}$. this scenario is possible when $\mathcal{Y}_S \neq \mathcal{Y}_T$ and/or $P(Y_S|X_S) \neq P(Y_T|X_T)$. The scenario where $P(Y_S|X_S) \neq P(Y_T|X_T)$ implies that the conditional probability distributions between the source and target domains are different. The case $\mathcal{Y}_S \neq \mathcal{Y}_T$ mean the label space of the source and target domain are different.[87]

3.5.2 Categories

As per the definition of transfer learning mentioned above, it can be broadly categorized into 3 groups: inductive transfer learning, transductive transfer learning and unsupervised transfer learning as mentioned in [58]. Depending on the availability of labels of the source and target tasks, these 3 categories are possible as shown in Table 2.

Regardless of whether the source and target domains are the same or not, the target task in an inductive transfer learning situation is distinct from the source task. In this instance, in order to generate an objective predictive function $f_T(\cdot)$ for usage in the target domain, some labeled data from the target domain are needed. Furthermore, inductive transfer learning can be broken down into two groups depending on the availability of source and target domain labels. When a lot of labeled data is available for the source domain, it is similar to multi-task learning. But the difference lies in the fact that the goal of inductive transfer learning is to attain good results in the target task only by using the knowledge obtained from the source task while multi-task learning endeavors to learn both the target and source tasks concurrently[58]. If the labels of the source data are not available, then it is similar to the self-taught learning as proposed by [67]. In such a scenario, the label space of the source data and the target data can vary which means that the knowledge obtained from the source domain cannot be utilised directly. This implies that basically the labels of the source domain are not available.[58]

On the other hand, in transductive transfer learning, the target task and the source task are same, but the source domain and the target domain are different. Depending on the similarity of the feature space and the marginal probability distribution between the source and the target domain, transductive transfer learning too can be classified into two groups. One scenario can be that the feature space of the source domain and that of the target domain are different i.e. $\mathcal{X}_S \neq \mathcal{X}_T$. Again, the feature space of the two domain can be same, but the input data's marginal probability distributions are different i.e. $P(X_S) \neq P(X_T)$. Similar assumptions underlie the latter instance of the transductive transfer learning, which is related to sample selection bias [92], covariate shift [72], and domain adaptation for knowledge transfer in text categorization [15, 92].[58] Lastly, in unsupervised transfer learning, the target task is different from the source task as it was in inductive transfer learning, but the labels of neither the source domain nor the target domain is available. the goal of unsupervised transfer learning is to solve unsupervised learning problems in the target domain like density estimation, dimensionality reduction and clustering[14, 85]. The categorisation of transfer learning has been summarised in Fig.3.5.3

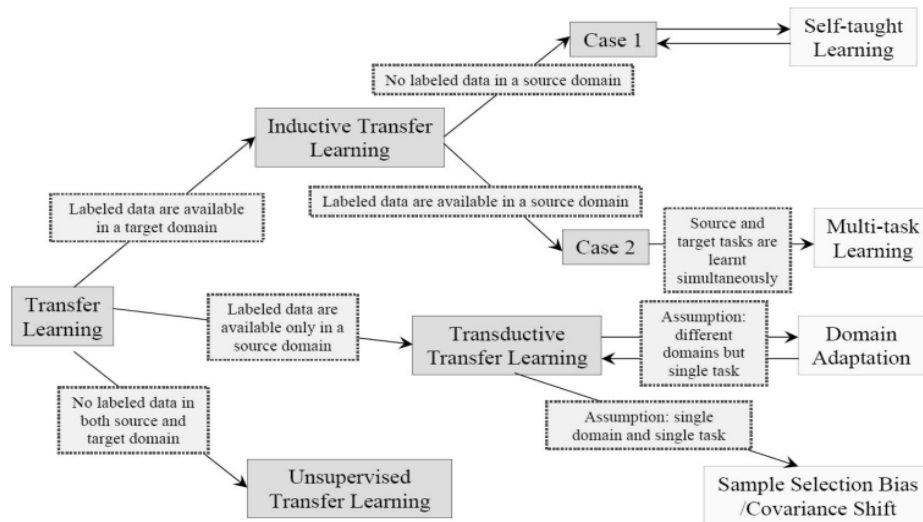


Figure 3.5.3: Overview of the different types of transfer learning.[58]

3.5.3 Relevant Algorithms

3.6 Optimal Transport Problem

3.7 Sinkhorn-Knopp Algorithm

3.8 Dimensionality reduction

Reducing the number of features (i.e., dimensions) in a dataset without sacrificing its relevant attributes is known as dimensionality reduction.[17] This is equivalent to eliminating features that are superfluous or redundant, or just noisy data, in order to build a model with fewer variables. Dimensionality reduction refers to a variety of preprocessing techniques for feature selection and data compression. Although dimensionality reduction techniques operate differently, they all use variable extraction or combination to convert high-dimensional spaces into low-dimensional spaces.

Dimensions, often known as features or input variables, are predictor variables in machine learning that control a model's output. Any dataset having a significant number of predictor variables is referred to as high-dimensional data. Machine learning algorithms face several practical challenges when dealing with high-dimensional datasets, including longer calculation times and larger data storage requirements. Perhaps the greatest worry, though, is that predictive models are becoming less accurate. Models in machine learning that are trained on high-dimensional datasets frequently have poor generalization.

Curse of dimensionality

The inverse link between growing model dimensions and declining generalizability is known as the "curse of dimensionality." The model space rises with the amount of model input variables. Conversely, sparse data results from a constant amount of data points. This indicates that much of the feature space of the model is empty, or devoid of observable data points. Data points get so different from one another as data sparsity rises that prediction models are less successful in finding explanatory patterns.[24] Models may overfit on training data in order to explain patterns in sparse data sufficiently. Thus, poor generalizability may result from increases in dimensionality. By causing multicollinearity, high dimensionality might further impede model interpretability.[5] The effect of the number of model parameters on the performance of the model is shown in Fig. 3.8.1 There is a chance that certain variables are unnecessary or associated as the number of model variables rises. The curse of dimensionality can be mitigated by increasing the collection of data by reducing data sparsity. However, the number of data points required to counteract the curse of dimensionality grows exponentially with the number of dimensions in a model. It's not always possible, of

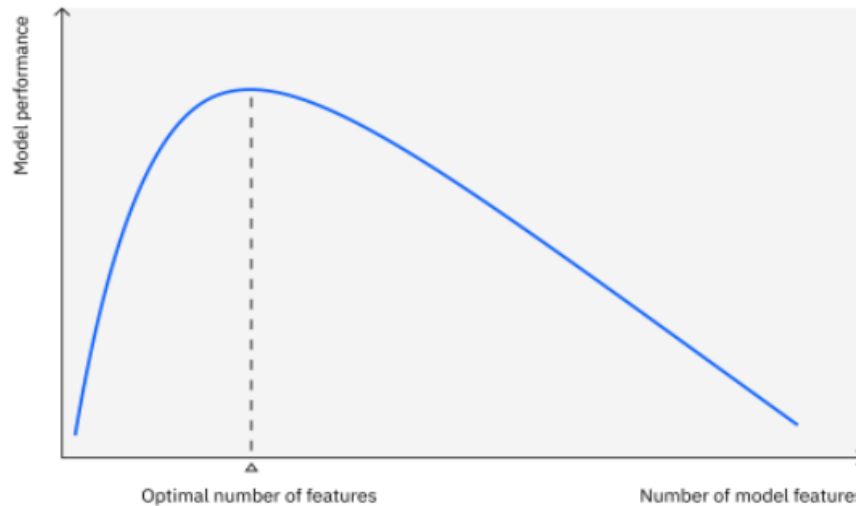


Figure 3.8.1: The effect of the number of model parameters on the model performance.[88]

course, to gather enough data. Therefore, dimensionality reduction is necessary to enhance data analysis. Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA), and t-SNE are dimensionality reduction algorithms that improve machine learning models.

3.8.1 Linear Discriminant Analysis

In supervised machine learning, LDA is a method used to resolve multi-class classification problems. LDA reduces the dimensionality of the data to enable the separation of several classes with numerous features. A generative model framework is used in LDA, which is often referred to as Normal Discriminant Analysis (NDA) or Discriminant Function Analysis (DFA). In other words, LDA algorithms estimate the distribution of data for each class and classify additional data points by applying Bayes' theorem [5]. Conditional probabilities, or the likelihood of an event given the occurrence of another event, are computed by Bayes. Bayes is used by LDA algorithms to determine the likelihood that a particular data item will be included in a specific output. Finding a linear feature combination that distinguishes any number of categories of objects or events is how LDA operates. To facilitate easier classification, LDA projects data with at least two dimensions into a single dimension as shown in Fig. 3.8.2. As a result, the method is occasionally called "dimensionality reduction." Unlike logistic regression, for example, which is restricted to binary classification, LDA's adaptability guarantees that it may be applied to multi-class data classification issues. For this reason, LDA is frequently used to improve the performance of other machine learning classification methods, such as Support Vector Machines (SVM), random forests, and decision trees.

3.8.2 Principal Component Analysis

Similar to LDA, PCA is also a dimension reduction method. Unlike LDA, PCA is not restricted to problems involving supervised learning. In other words, PCA can reduce dimensions for unsupervised learning tasks without taking class labels or categories into account. Mathematician Karl Pearson first presented the PCA approach in 1901. It functions under the requirement that the variance of the data in the space of fewer dimensions should be maximal even when data in a space with more dimensions is transformed to data in a lower dimensional space as shown in Fig. 3.8.3. In an unsupervised learning paradigm, it can find the correlation between different features in the dataset. A linear combination of the variables in the original dataset gives rise to the principal components. By combining the variables in this manner, the majority of the information found in the original variables is condensed into the first components, and these new variables, or principal components, are left uncorrelated. The premise is that ten-dimensional data sets provide ten principal components. PCA then attempts to assign the maximum amount of information to the first principal component, the maximum amount of information left to the second principal component, and so

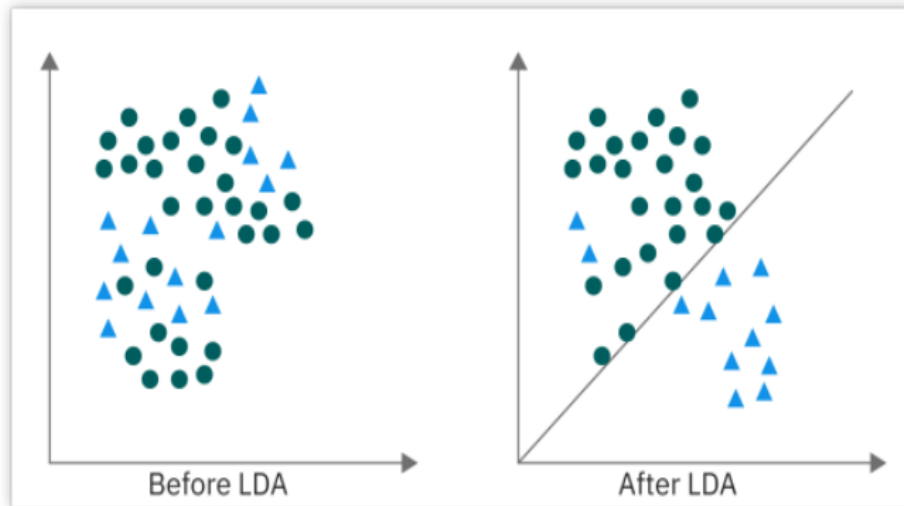


Figure 3.8.2: Projection of the data before and after applying LDA[88]

on, until the result is somewhat like the Fig. 3.8.4. By removing the components with little information and using the ones that remain as the new variables, the dimensionality can be minimized without compromising much information when the data is organized using principle components. It's crucial to understand that because the primary components are created as linear combinations of the original variables, they are less interpretable and have no true significance. From a geometric perspective, the principal components are the data's directions that account for the greatest amount of variance, or the lines that encompass the majority of the data's information. Here, variation and information are related in that a line's variance indicates how dispersed the data points are along it; the more dispersion a line has, the more data it contains. To put all of this into perspective, consider principle components as new vectors that offer the optimal viewpoint for examining and assessing the data, making the variations between the observations easier to observe.

Working principle for PCA

1. **Normalization** This step's objective is to normalize the continuous original variable's range to ensure each one makes an equal contribution to the analysis. More precisely, the reason PCA is extremely sensitive to the variances of the original variables is the reason normalization must be done first. In other words, if the ranges of the initial variables differ significantly, the variables with greater ranges will take precedence over the variables with smaller ranges (for instance, a variable ranging from 0 to 100 will take precedence over a variable ranging from 0 to 1), which will produce biased results. Therefore, this issue can be avoided by translating the data to similar scales. The mathematical procedure for achieving this is to take each variable's value, subtract the mean, then divide the result by the standard deviation as shown in Eq. 24.

$$z = \frac{\text{value} - \text{mean}}{\text{standard deviation}} \quad (24)$$

All of these variables will be converted to the same scale after standardization is complete.

2. **Computation of covariance matrix** This step's objective is to determine whether there is a correlation between the variables in the dataset being analyzed and how they differ from the mean in relation to one another. Because variables might occasionally include redundant information due to strong correlations, the matrix of covariance is calculated to find these correlations. The correlation matrix of a 3-D data with variables a, b, c are given by the covariance matrix shown below.

$$\begin{bmatrix} \text{Cov}(a,a) & \text{Cov}(a,b) & \text{Cov}(a,c) \\ \text{Cov}(b,a) & \text{Cov}(b,b) & \text{Cov}(b,c) \\ \text{Cov}(c,a) & \text{Cov}(c,b) & \text{Cov}(c,c) \end{bmatrix}$$

The covariance matrix has all possible pairs of initial variables

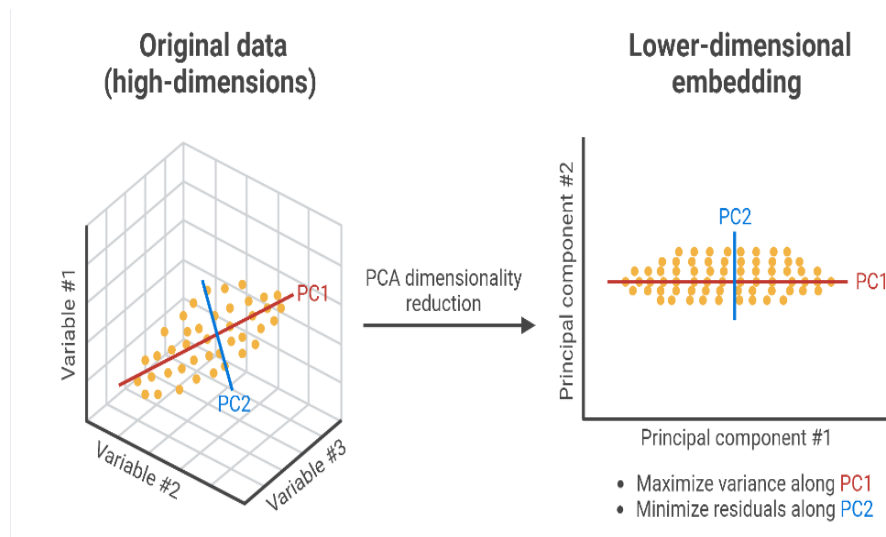


Figure 3.8.3: Projection of the data from higher dimensional space to lower dimensional space on applying PCA[64]

as entries, and is a $p \times p$ symmetric matrix (where p is the number of dimensions). We essentially have the variances of each starting variable along the major diagonal (top left to bottom right), as the variance of a variable with itself is its covariance ($\text{Cov}(a,a)=\text{Var}(a)$). Furthermore, because the covariance is commutative ($\text{Cov}(a,b)=\text{Cov}(b,a)$), the covariance matrix's entries are symmetric with regard to the main diagonal, meaning that the top and lower triangular sections are equal. When two variables increase or decrease at the same time, i.e. they are correlated, then the value in the covariance matrix is positive. Whereas, if the two variables doesn't follow the same trend, i.e. one increases while the other decreases, then the variables are uncorrelated.

3. **Computation of eigenvectors and eigenvalues** The two necessary mathematical concepts that are essential for the computation of the principal components are eigenvectors and eigenvalues. Every eigenvector has an associated eigenvalue with it. The number of eigenvectors or eigenvalues for the data is equivalent to the dimension of the data. The eigenvector gives the direction of the axis along which most of the variance of the original dataset is preserved or in other words the direction of the principal components. The eigenvalue associated with the eigenvector is the coefficient of the eigenvector, i.e. the amount of variance that particular principal component is responsible for in the total variance. The principal components are obtained by arranging the eigenvectors from highest to lowest of their corresponding eigenvalues.
4. **Creation of feature vector** Creating the feature vector involves selecting which elements to keep and which to remove. Generally speaking, components with low eigenvalues won't be as important. A plot is generated to display the cumulative proportion of variation as well as the percentage of total variance explained. Typically, the number of PCA components we wish to include is indicated by the point at which the Y axis of eigenvalues or total variance explained forms a "elbow".
5. **Data transformation along the axis of principal components** With the exception of normalization, none of the data is altered in the preceding steps; instead, the input data set is always expressed in terms of the original axes, or the original variables. The primary components are chosen and the feature vector is created. The goal of this final step is to reorient the data from the original axis to the ones that the principal components, thus the name Principal Components Analysis represent by utilizing the feature vector that was created using the eigenvectors of the covariance matrix. To accomplish this, multiply the feature vector's transpose by the original data set's transpose as shown in Eq. 25

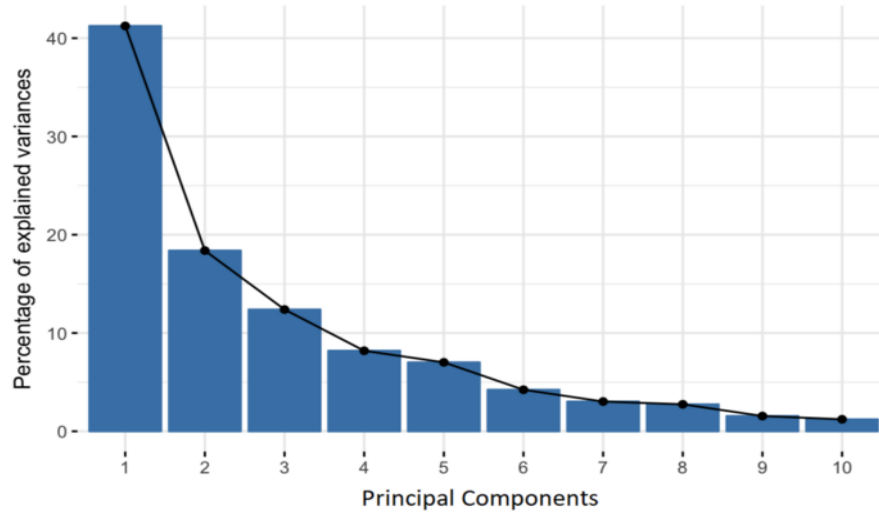


Figure 3.8.4: The percentge of explained variance for each principal component

6.

$$TransformedDataSet = FeatureVector^T \times NormalizedOriginaldataSet^T \quad (25)$$

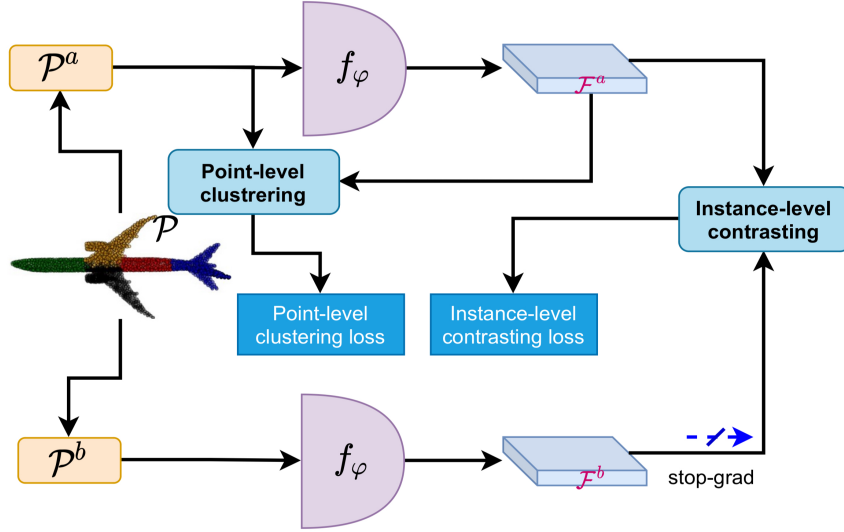


Figure 4.1.1: The Conclu model pipeline including the two modules: point -level clustering and instance-level contrasting.[50]

4 Method

This chapter gives the details of the networks and clustering algorithms that were examined. The unsupervised feature representation learning approach proposed by Guofeng Mei *et. al.*[50] was adapted to be used in our scenario. The authors proposed an unsupervised learning approach, ConClu that learns both the local features and well as the global features of the point clouds. It was suitable to be used in our scenario, as it would require no point-level annotation of the dataset, which was unavailable in our case. To do so, it performed point-level clustering of the points clouds to generate semantically related regions within the point cloud and performed instance-level contrasting to make the network robust to its global appearance which would be explained in detail in the following subsections.[50]

The primary motivation behind using an unsupervised learning methodology is that annotation of point clouds can be really challenging. To really focus on the point level features of a point cloud, one would require a high number of points in the point cloud. And labelling such high number of points in each point cloud in large real world datasets can be really expensive and severely inefficient. Unsupervised learning facilitates high quality feature representation without the need for humans in the loop. Moreover, sporadic and non-uniform distribution of the point clouds can also add further challenges in point-level annotation of the point clouds. Thus, in the absence of such annotations, it is impossible to get supervised feature representations for computer vision tasks.

4.1 Architecture

Analysing how humans perceive an object, they do not focus merely on the individual points but rather semantically related points that form a group and act as the fundamental block for the entire object as a whole. The global features focus on the entire shape of the object while the local features attains to the more detailed information in the different parts of the object. In this work, we have 3D point cloud $\mathbf{P} = \{\mathbf{p}_i \in \mathbb{R}^3 | i = 1, 2, 3, \dots, N\}$ where N is the number of points in each point cloud and $\mathbf{p}_i$ is each point in the point cloud represented in 3D cartesian coordinate system. The point cloud $\mathbf{P}$ is fed to an encoder backbone f_ϕ which generates a point-wise feature matrix $\mathbf{F} = \{f_{p_i}\}_{i=1}^N$ where f_{p_i} is the point-wise feature vector. The objective was to train a feature encoder f_ϕ , PointNet in our case, having parameters ϕ in an unsupervised learning paradigm. The entire pipeline of the process is elaborated in Fig.4.1.1. As we can see, it has two parts: the point-level clustering for learning the local features and the instance-level contrasting for learning the global features. Before we go into a detailed analysis of each module, here is

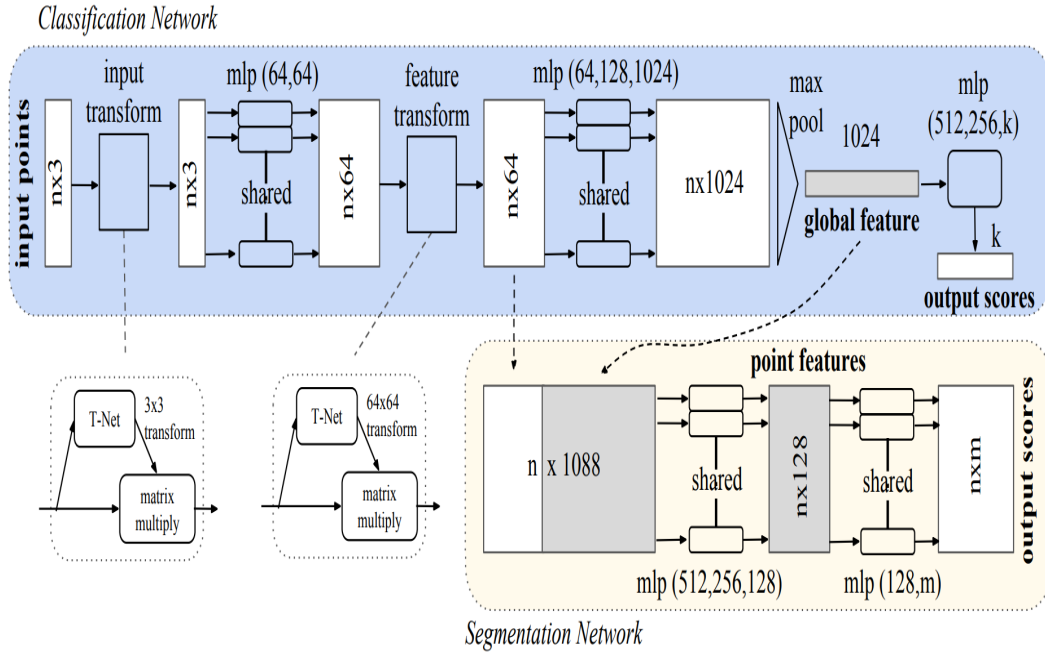


Figure 4.2.1: The PointNet Architecture.[65]

a brief overview of the entire process. The augmented views $\mathbf{P}^a$, and $\mathbf{P}^b$ are obtained for each point cloud $\mathbf{P}$ and are fed to the encoder f_ϕ such that it generates the feature matrices $\mathbf{F}^a$, and $\mathbf{F}^b$ respectively. Thus, the inputs to the point-level clustering are $\mathbf{P}^a$ and $\mathbf{F}^a$. It is to be noted here that the feature encoder f_ϕ shares the weights between the two augmented views. f_ϕ is trained in a way such that it minimises the point-level clustering and the instance-level contrasting loss together. It is also to be noted here, that the encoder receives the gradient only from the top branch. After the training is completed, both the losses are cast aside and only the encoder is needed for further tasks.

4.2 Encoder Backbone

Keeping in line with [50], two feature representation learning backbones were used for the experiments, the PointNet [65] and the DGCNN [84] backbone. The mechanism and the architecture of the feature extraction backbones have been elaborated further in the following sections.

4.2.1 PointNet Autoencoder

For the purpose of weight sharing and other kernel optimization tricks, traditional convolution networks need extremely regular formats for the input data as prevalent in RGB or depth images and 3D voxels. But point clouds are not regular data structures. It is just a set of 3D points which is permutation invariant to its constituent points. Or in other words, the order in which the individual points of a point cloud are processed is inconsequential. In [65], the authors suggested an end-to-end architecture that accepts the point clouds as input and generates the class label of the entire object or a class label for each of the individual points in the point cloud. The PointNet architecture is shown in Fig. 4.2.1 It consists of three important modules: a symmetric function to aggregate information, a module that integrates the local and the global features, an alignment network that orients the points of the point cloud and their features. A further detailed overview of each of the building blocks are given in the following passages.

Symmetry function The points in the point cloud are an unordered set of points, which means the order in which they appear is irrelevant. Therefore, to make the network invariant to permutation, the following approaches could be adopted.

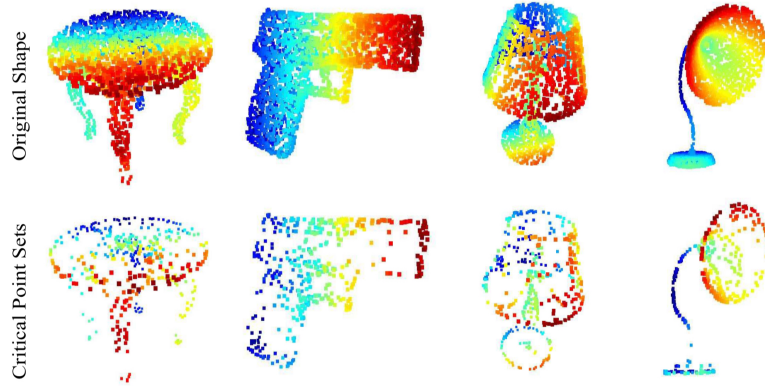


Figure 4.2.2: Critical points set and the original shape.[65]

- The input points could be sorted in canonical manner.
- To treat the input points like sequential data and train a Recurrent Neural Network (RNN). But in order to counter the effect, the RNN needs to be trained with all possible set of permutations
- The data from each point could be combined using a symmetric function. Binary addition and multiplication are some examples of symmetric functions.

Sorting the points isn't an efficient solution because in high dimension space, there isn't actually any stable ordering w.r.t point perturbations. Contradiction is a simple way to demonstrate that. If such an ordering existed, it would mean there is a bijection mapping between the higher dimension space and a 1d real line. For this ordering to be stable, the bijection mapping would need to maintain spatial proximity even if the dimensions decreased in the event of point perturbations. This is not feasible in most cases. As a result, sorting only partially fixes the ordering problem, and it brings additional challenges for the network to learn a consistent bijection mapping between the input and the output. The second approach was to treat the points as a sequential data and training an RNN with different randomly permuted input sequence in order to make it invariant to the order in which the input appears. But in [82], it was proved that the order indeed matters and cannot be overlooked completely. Moreover, even if an RNN is robust to the order of the input with small length(e.g. dozens), it suffers from scalability issues. It can't be scaled to inputs with thousands of elements as it is in point clouds. Hence, the idea proposed by the authors in [65] was to devise a symmetric function on the transformed set of points to approximate the general function defined on the points as in Eq. 26

$$f(\{x_1, \dots, x_n\}) \sim g(h(x_1), \dots, h(x_n)) \quad (26)$$

where $f : 2_{\mathbb{R}_N \rightarrow \mathbb{R}}, \mathbb{R}^N \rightarrow \mathbb{R}^K$ and $g : \underbrace{\mathbb{R}^K \times \dots \times \mathbb{R}^K}_n \rightarrow \mathbb{R}$ is the symmetric function. Here h is a multi-layer

perceptron network and g is a combination of a single variable function and a max pooling function. Even if this module is fairly simple, it has some of very important implications. If C_S is the set of critical points, the points which contributed to the max pooled feature, the PointNet architecture was seen to be quite robust to the non-critical points. In other words, discarding the non-critical points didn't lead to any loss of information about the global shape of the object as shown in Fig. 4.2.2.

If needed we can add theoretical analysis here

Local and Global Information Aggregation The output of the symmetry function is a vector $[f_1, \dots, f_K]$ which is the global feature representation of the point cloud. But, to get point-wise segmentation, information about both the local features and the global features are necessary. The local features are learnt by the network in the *Segmentation Network* module as shown in Fig. 4.2.1. Once the global feature vector of the point cloud is calculated, it is concatenated with each of the point features and is fed into the network. This enables the network to learn new point-level features when it is informed about both the local and the global information. Because of this module, the network is able to predict the class probability of individual point based on both the local geometry of the object as well as the semantic region it belongs to on a global-level.[65]

Joint Alignment Network The network should be invariant to certain geometric transformations like rigid body transformations (e.g. rotation, translation). A trivial solution to this issue is aligning every input set to a canonical order prior to feature extraction. But this would require an additional layers for aligning all input sets. On the contrary, the authors in [65] utilized a "mini-network" (*T-net*) as shown in Fig. 4.2.1. This predicts a transformation matrix which is then applied to the coordinates of the points in the point cloud. The mini-network too takes after the main network and is made up of point independent feature extraction module, max pooling layers and the fully connected layers. To align the point-features from different point cloud sets, another alignment network could be introduced. It would predict the transformation matrix required for the alignment. But that would significantly increase the difficulty of optimization since the transformation matrix has much higher dimensions as compared to the spatial transformation matrix. To circumvent this issue, a regularization term was used in the softmax training loss. The constraint enforced in this case was that the feature transformation matrix is to be approximated by an orthogonal matrix as given in Eq. 27

$$L_{reg} = \|I - AA^T\|_F^2 \quad (27)$$

where $\|\cdot\|_F$ is the Frobenius-norm, I is the Identity matrix, and A is the feature alignment matrix predicted by the mini-network. The feature transformation matrix has been approximated by an orthogonal matrix because orthogonal transformations ensure that no information about the input has been lost.

4.2.2 Dynamic Graph Convolutional Neural Network

Another more general feature encoder was used in this work. It was seen that the PointNet architecture treated each point of the point cloud independently. The model was made invariant to the different possible permutations by applying a symmetric function to aggregate the features. But a limitation to this method is that it doesn't take into consideration the topology of the points. It ignores the connectedness in between the points and thus cannot learn the local features. This limitation was addressed by Y. Wang *et al.* in [84]. The authors proposed the EdgeConv operation which was able to learn the local features of the point cloud while still meeting the requirement of being permutation invariant. EdgeConv generated the edge features which were capable of capturing the topological information of the points i.e. the connection between a point and its neighbors. In other words, the EdgeConv operation computes a local graph for each of the points in the point cloud which was capable of learning the edge embeddings for the edges in the graph. So the model group points based on both the Euclidean distance between the points and also how they were semantically related. The EdgeConv is a plug-and-play module that would be used combined into any existing model architecture to further enhance its performance. The approach has been further elaborated in the following passages.

Unlike PointNet, the DGCNN network captures the topological information by generating a local neighborhood graph and then applying an operation analogous to the convolution operation on the edges connecting a pair of points. It is in line to the methodology used in graph neural networks. But unlike a traditional graph CNN, the local neighborhood graph is not fixed in this case. On the contrary, it is updated after every layer in the network. This is because the k -nearest neighbors of a point doesn't remain the same after every layer of the network. This happens as the k -nearest neighbors are computed not based on the Euclidean distance between the points, but rather of the proximity of the points based on the learnt embeddings. In other words, the proximity of the points in the feature space are different from the proximity of the input points in the Euclidean space. This is responsible for the non-local flow of information through the point cloud.

Edge Convolution In this work, we have 3D point cloud $P = \{p_i \in \mathbb{R}^F | i = 1, 2, 3, \dots, N\}$ where N is the number of points in each point cloud and p_i is each point in the point cloud represented in 3D cartesian coordinate system where $p_i = (x_i, y_i, z_i)$. In this work, $F = 3$, but it is also possible to include additional information about the coordinates such as surface normals, color, etc. like most traditional deep neural networks, each layer acts upon the outputs of the previous layer. So generally speaking, F is the dimension of the features of a particular layer in the network. The architecture of the model used in this work is shown in Fig. 4.2.3. The top branch is used for the classification of the point cloud and the bottom branch is for the point-wise segmentation task. The classification task requires the n points in point cloud as

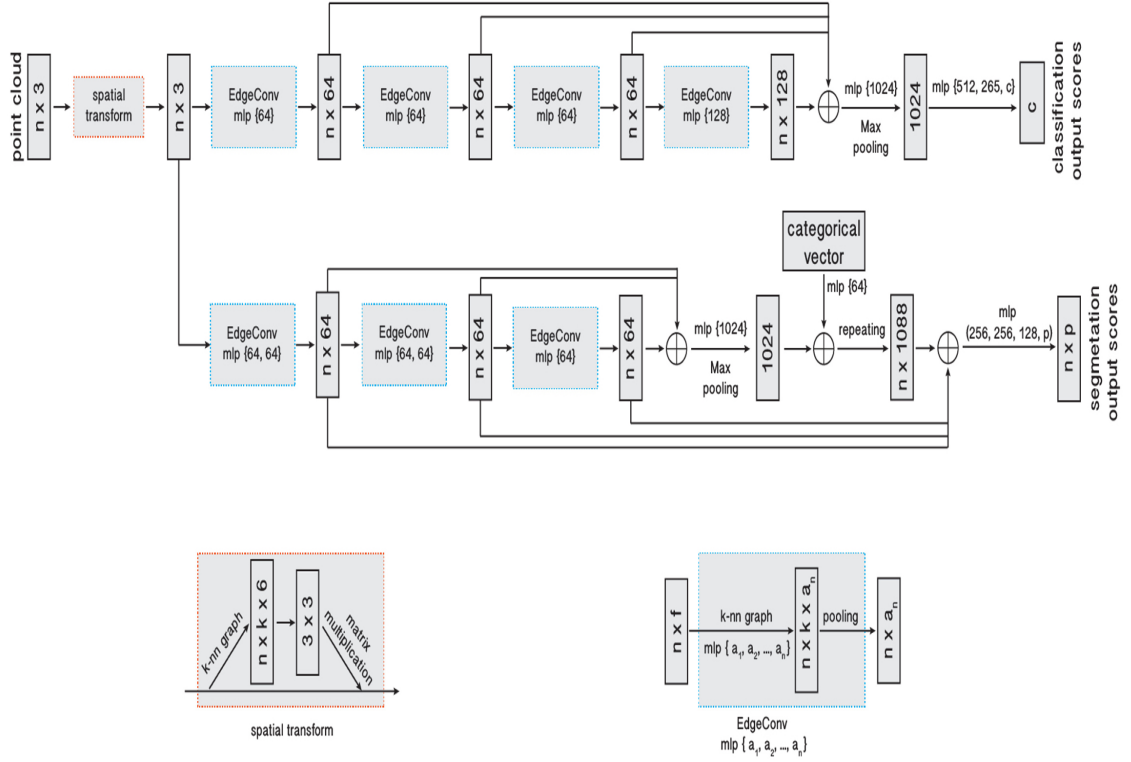


Figure 4.2.3: The DGCNN Architecture.[84]

input. k is pre-defined as the number of nearest neighbors that are to be considered for generating the edge features. The EdgeConv layer computes an edge feature set of size k for each point p_i in the point cloud. An aggregation operation is performed on each of these sets to get the EdgeConv response for the respective points. A global aggregation function is applied on the output of the last EdgeConv layer to generate a 1D global feature descriptor. This is used to compute the classification score for c classes. The point-wise segmentation branch extends the classification branch by concatenating the 1D global descriptor with all EdgeConv responses (acts as local descriptors) for each point in the point cloud. Thus, it generates a point-wise classification score for p semantic classes that it can belong to. $\oplus$ is the concatenation operation. The spatial transform block is the module used to align the input point cloud to a canonical space. It is done by applying an estimated 3×3 transformation matrix. The coordinates of each of the points in the point cloud are concatenated with the difference in the coordinates with the k neighboring points giving rise to a tensor. This tensor is used in computing above-mentioned transformation matrix. The input point cloud of shape $n \times f$, here $f = 3$ is fed to the EdgeConv block which computes the edge features for each of the points in the point cloud. An Multi Layer Perceptron (MLP) is used for this purpose where the number of neurons in a layer is defined as $\{a_1, a_2, \dots, a_n\}$. the output of this block is a tensor with shape $n \times a_n$ after performing a pooling operation on the neighboring edge features.

A directed graph $G = (V, E)$ which captures the topology of the point cloud where $V = \{p_i | i = \{1, 2, \dots, n\}\}$ is the set of vertices and $E \subset V \times V$ is the set of edges. The graph G is the k-Nearest Neighbor (k-NN) graph of $\mathcal{P}$ in $\mathbb{R}^F$. the graph has self-loops which means, each point in the point cloud has an edge to itself. The edge features computed by the EdgeConv block was defined as $e_{ij} = h_\theta(p_i, p_j)$ where $h_\theta : \mathbb{R}^F \times \mathbb{R}^F \rightarrow \mathbb{R}^{F'}$ is a non-linear function with a set of trainable parameters θ . The EdgeConv operation is defined as a symmetric aggregation operation $\square$ which is applied on a channel wise manner. Summation (Σ) or max are some of the examples of aggregation functions. It is applied on the edge features associated with all the edges that originate from each connected vertex. Therefore, the output of the EdgeConv layer for i -th vertex

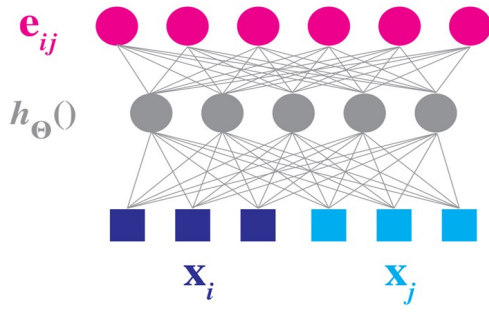


Figure 4.2.4: The Computation of edge feature e_{ij} from a pair of points p_i and p_j . Here, h_θ is a fully connected layer and the weights associated with it are the trainable parameters θ . [84]

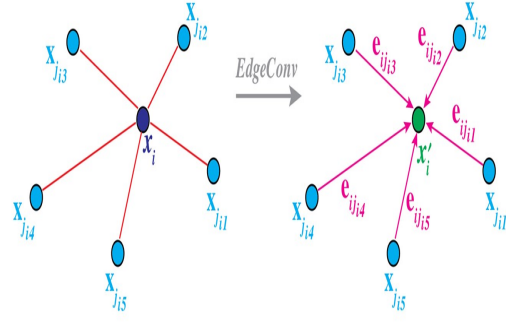


Figure 4.2.5: The EdgeConv operation. The output of EdgeConv is computed by aggregating the features associated with all the edges that originate from each connected vertex. [84]

is given by Eq. 28

$$p'_i = \square_{j:(i,j) \in E} h_\theta(p_i, p_j) \quad (28)$$

drawing similarity with convolution on images, p_i is the central pixel and $\{p_j : (i, j) \in E\}$ is a patch around the central pixel as shown in Fig. 4.2.5. Therefore, for a point cloud with F dimensions and n points in it, the EdgeConv layer generates a point cloud with F' dimensions and n points in it.

Deciding the non-linear function h_θ and the symmetric aggregation function $\square$ plays a pivotal role in the deciding the properties of the EdgeConv layer. To further illustrate on this point, if $p_1, \dots, p_n$ are the points in the point cloud and the graph G encapsulates the topological information of patches of fixed size around each central pixel, then if $\theta_m \cdot p_j$ is chosen as the non-linear function and sum (Σ) is chosen as the aggregation operation, then it would behave as a standard convolution operation as in eq. 29

$$p'_{im} = \sum_{j:(i,j) \in E} \theta_m \cdot p_j \quad (29)$$

where $\theta = (\theta_1, \theta_2, \dots, \theta_M)$ are the weights of M different filters used in the convolution operation. Each entry in θ_m has the dimensionality as p_i and $\cdot$ is the Euclidean inner product. Another variation of h could be

$$h_\theta(p_i, p_j) = h_\theta(p_i). \quad (30)$$

Here only the global embedding is encoding is taken into consideration and the non-linear function is unaware of the local features. This happens in PointNet and thus it could be considered as a special case of EdgeConv. Another variation of h as adapted by Atzmon *et al.* in [2] is

$$h_\theta(p_i, p_j) = h_\theta(p_j). \quad (31)$$

and

$$p'_{im} = \sum_{j \in V} (h_\theta(p_j)) g(u(p_i, p_j)). \quad (32)$$

Here g is a Gaussian kernel and u calculates the pairwise Euclidean distance between the points in the point cloud. Another variation of h could be to only capture the local features and ignore the global features based on the fact that the global shape is a collection of the local patches as shown in the following

$$h_\theta(p_i, p_j) = h_\theta(p_j - p_i). \quad (33)$$

The final variation of h which was utilized by the authors in [84] is an asymmetric edge function as shown in eq. 34

$$h_\theta(p_i, p_j) = \bar{h}_\theta(p_i, p_j - p_i). \quad (34)$$

This explicitly takes into account the global feature information as in Eq.30 captured by the coordinates of the central pixel of the patch p_i and the information of the local neighborhood as in Eq. 33 and is captured by $p_j - p_i$. the EdgeConv operator was defined as

$$e'_{ijm} = \text{ReLU}(\theta_m \cdot (p_j - p_i) + \phi_m \cdot p_i). \quad (35)$$

This operator could be implemented as a shared MLP and then performing a max-pooling operation as

$$p'_{im} = \max_{j:(i,j) \in E} e'_{ijm}, \quad (36)$$

where $\theta = (\theta_1, \theta_2, \dots, \theta_M)$ are the weights of M different filters used in the convolution operation.

Dynamic Graph Update A crucial point in this network was to regenerate the graph using k-NN in the feature space after each layer. This is the main difference of this network as compared to traditional graph CNN for which there is a fixed graph for all the layers of the network. Because the graph was updated after every layer, the architecture gets its name the DGCNN. With dynamically updating the graph after each layer, the receptive field increases to as large as diameter of the point cloud, even if the points in the point cloud are sparse. Thus after each layer a different graph is computed $G^{(l)}=(V^{(l)}, E^{(l)})$, where $G^{(l)}$ is a graph in the l -th layer with vertices $V^{(l)}$ and edges $E^{(l)}$ which can be denoted as $(i, j_{i1}), \dots, (i, j_{ik_i})$ such that $p_{j_{i1}}^{(l)}, \dots, p_{j_{ik_i}}^{(l)}$ are the k-nearest points closest to point p_i at layer l . In other words, the model constructs a different graph for each point in the point cloud after every layer where the nearest points and the edges emerging from a vertex(point in the point cloud in the case) changes at every layer. A pairwise distance matrix in the feature space is used to calculate the k-nearest points of each point in the point cloud. As a result of this, the network has the following properties,

- *Invariant to permutations.* The output of an EdgeConv layer is given by

$$p'_i = \max_{j:(i,j) \in E} h_{\theta}(p_i, p_j). \quad (37)$$

Because of the usage of a max operator which is symmetric in nature, the output of the layer p'_i is permutation invariant to the order in which p_i appears in the input point cloud. Other functions which are symmetric (eg. $\sum$) in nature can also be applied here instead of max. the global max pooling operator used to aggregate point features is a symmetric function and thus gives permutation invariant results.

- *Invariant to translation.* The EdgeConv operator satisfies "partial" translation invariance. If a translation T is applied to the points p_i and p_j of the point cloud $\mathcal{P}$, then applying translation to Eq. 35, the translated point cloud would be

$$\begin{aligned} e'_{ijm} &= \theta_m \cdot (p_j + T - (p_i + T)) + \phi_m \cdot (p_i + T), \\ &= \theta_m \cdot (p_j - p_i) + \phi_m \cdot (p_i + T). \end{aligned} \quad (38)$$

Therefore, if $\phi_m = 0$, then EdgeConv operator is completely translation invariant. But this however restricts the model to recognize an object depending on only the unordered set of patches, i.e. it disregards the orientation and the exact location of the patches. On the other hand, with both $p_j - p_i$ and p_i , the model considers both the local orientation of the patches as well as the global information about the shape of the object.

4.3 Point-level Clustering Module

Now, the model already has the feature representation from the feature encoder backbone (eg. PointNet or DGCNN). The next step is the point-level clustering module. Fig.4.3.1 gives a detailed visualisation of the point-level clustering module. We can see that it is made up of 2 entites, the "class probability prediction" function and the "label reassigning". The point-level clustering module is analogous to the semantic segmentation task. Each point p_i of the point cloud $\mathcal{P}$ is assigned to one of the J predetermined

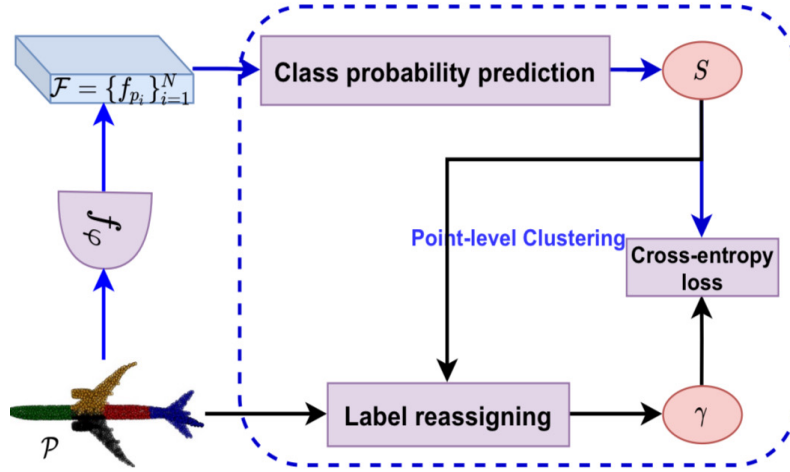


Figure 4.3.1: The architecture of the point-level clustering based unsupervised feature learning.[50]

possible subgroups or semantic categories. Essentially, it is achieved by processing each of the point cloud $\mathbf{P}$ followed by a point-level class probability prediction operator that yields a matrix for class probability $\mathbf{S} = \{s_{ij} \in [0, 1]\}_{i,j}^{N,J}$. The label reassigning operator thus takes the point cloud $\mathbf{P}$ and the class probability matrix $\mathbf{S}$ and generates a pseudo-label matrix $\gamma = \{\gamma_{ij} \in \{0, 1\}\}_{i,j}^{N,J}$. The weights for the encoder f_ϕ are learnt such that it minimizes the average cross-entropy loss between the pseudo-label γ and the predicted class probability $\mathbf{S}$. The average cross entropy loss is calculated by the formula given in Eq.39.

$$\epsilon(\gamma, \mathbf{S}) = -\frac{1}{N} \langle \gamma, \log \mathbf{S} \rangle = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^J \gamma_{ij} \log s_{ij} \quad (39)$$

But to train a network with a cross-entropy loss, the ground truth labels of the points in the point cloud are required. Since that is not available, one needs to assign the label γ automatically, which is explained in details in the following passage.

4.3.1 Class Probability prediction

As mentioned in the last passage, the output of the encoder f_ϕ is fed as input to a classification head ϕ_α and it generates a logit score for each of the feature vector as shown in Fig.4.3.2. The vector for the logit score is given by $\mathbf{g}_i = (g_{i1}, g_{i2}, \dots, g_{iJ})$, where $\mathbf{g}_i = \gamma_\alpha(f_{p_i})$. The classification head has 2 fully connected layers where each layer is a linear layer followed by a batch normalization. The leakyReLU activation function is used in all the layers except the last layer. As the output of the last layer we get N vectors for each of the N points in the point cloud where each of the output vector has J dimensions, the number of predetermined subcategories each point cloud is to be segmented into. Thus each entry of the feature vector is the probability of that point belonging to j -th category. Thus, the logit score matrix $\mathbf{G} = \{g_{ij}\}_{i,j}^{N,J}$ has the dimension $N \times J$. The prediction that the point p_i belongs to the category j is computed by the application of a row-wise softmax operation on $\mathbf{G}$ given by Eq.40

$$s_{ij} = \frac{\exp(g_{ij})}{\sum_{l=1}^J \exp(g_{il})} \quad (40)$$

4.3.2 Label reassigning

In order to calculate the labels shown in Fig.4.3.3, one needs to the representative members of each of the semantic subgroups. It can be done by selecting the cluster mean or cluster centers of each of these subgroups as the representative member of the cluster. The class probability matrix $\mathbf{S}$ can be inferred as the

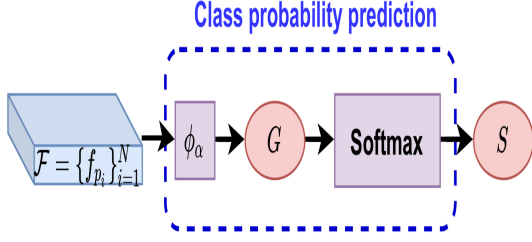


Figure 4.3.2: Architecture of class probability prediction.[50]

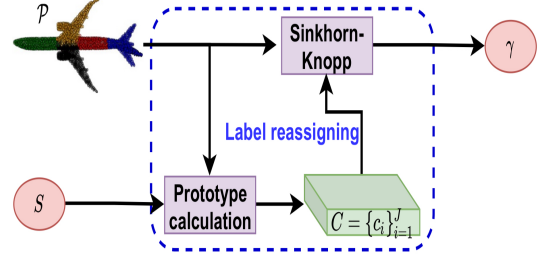


Figure 4.3.3: Architecture of label reassigning [50]

soft assignment of each point p_i in the point cloud to J semantic partitions. So the softly weighted mean of each of these partitions can be calculated as in Eq.41

$$c_j = \frac{1}{\sum_{i=1}^N s_{ij}} \sum_{i=1}^N s_{ij} \mathbf{p}_i, j = 1, 2, \dots, J \quad (41)$$

or in another way $C = \{c_j\}_{j=1}^J$ is a matrix with dimensions $J \times 3$. In a completely unsupervised learning setup, $\gamma \in [0, 1]$ can be visualized as the posterior probability of a p_i belong to the j -th partition. γ was optimized in the same method as in Eq.40. This results in a degenerate solution where each point is assigned to a single category. Thus, two assumptions were very pivotal for the reassigning of the labels. Firstly, the points in the point cloud ought to be partitioned into "equally-sized" partitions. In other words it can be formulated as $\sum_{i=1}^N \gamma_{ij} = \frac{N}{J}$. This is extremely importance in order to pretend all the points in the point cloud to be assigned to a single partition. Secondly, since the idea of using cluster centers as representative members of the partitions was inspired from the k-means algorithm, it means that if a point p_i of the point cloud belongs to the j^* -th partition, it means that the distance from p_i to c_j should be the shortest as compared to the distance of p_i to all other cluster centers. In other words $\|\mathbf{p}_i - c_{j^*}\|_2 \leq \|\mathbf{p}_i - c_j\|_2, j \neq j^*$. This is also equivalent to $\min_{\gamma} \sum_{i=1}^N \sum_{j=1}^J \gamma_{ij} \|\mathbf{p}_i - c_j\|_2^2$. This objective function is optimised to get γ . As per the rules of probability, the sum of all possible outcomes of an event is equal to 1, therefore, we get $\sum_{j=1}^J \gamma_{ij} = 1$. Thus, we can formulate the matrix of joint probability Γ as $\Gamma = \frac{\gamma}{N}$ which has the elements $\Gamma_{ij} = \frac{\gamma_{ij}}{N}$. The distance matrix $D = \{d_{ij}\}_{i,j}^{N,J}$ has dimensions $N \times J$ where each element d_{ij} is defined as $d_{ij} = \|\mathbf{p}_i - c_j\|_2^2$. In accordance to the assumptions stated above, the objective function D could be minimized with respect to Γ as in Eq.42

$$\begin{aligned} & \min_{\Gamma} \langle \Gamma, D \rangle, \\ & s.t., \Gamma^T \mathbf{1}_N = \frac{1}{J} \mathbf{1}_J, \Gamma \mathbf{1}_J = \frac{1}{N} \mathbf{1}_N \end{aligned} \quad (42)$$

where $\mathbf{1}_J$ is the vector of ones having dimension J . The above condition enforces that each of the cluster representative is selected atleast $\frac{N}{J}$ times for each point cloud and $\sum_{j=1}^J \gamma_{ij} = 1$. The objective function as defined in Eq.42 is a typical case of optimal transport problem[62]. Such problems can be solved by the Sinkhorn-Knopp algorithm[13]. In other words, it means solving the entropic regularized objective function in Eq. 43

$$\begin{aligned} & \min_{\Gamma} \langle \Gamma, D \rangle - \epsilon H(\Gamma), \\ & s.t., \Gamma^T \mathbf{1}_N = \frac{1}{J} \mathbf{1}_J, \Gamma \mathbf{1}_J = \frac{1}{N} \mathbf{1}_N, \end{aligned} \quad (43)$$

where, $H(\Gamma) = \langle \Gamma, \log \Gamma - 1 \rangle$ is the entropy of Γ and ϵ is the regularization parameter. As per [13], on solving Eq. 43, the normalized exponential matrix in Eq.44 is attained.

$$\Gamma = \text{diag}(\mu) \exp(D/\epsilon) \text{diag}(\nu) \quad (44)$$

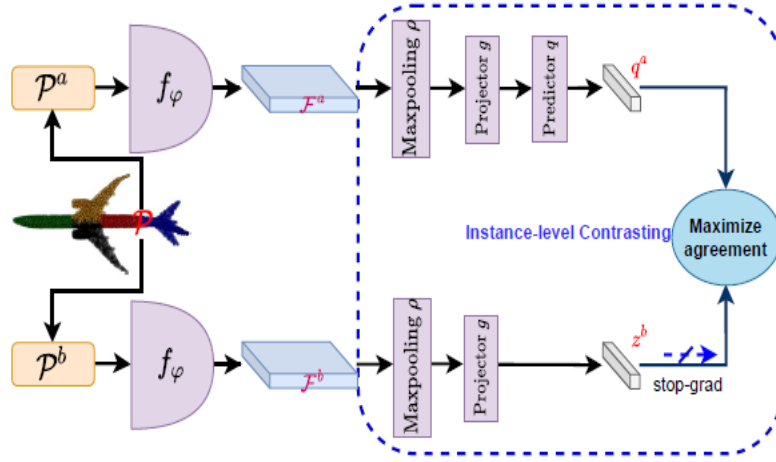


Figure 4.4.1: The architecture of the instance-level Contrasting.[50]

where μ and ν are renormalized vectors in $\mathbb{R}^N$ and $\mathbb{R}^J$ respectively. These vectors were computed on utilizing the iterative Sinkhorn-Knopp algorithm[13] with the constraints $\mu = \frac{1}{N} \mathbf{1}_N$ and $\nu = \frac{1}{J} \mathbf{1}_J$. The pseudo-code for the Sinkhorn-Knopp algorithm is elaborated in Algorithm 3. In the experiments, it was observed that 7 iterations were sufficient for a decent performance. Thus the overall point-level clustering can be encapsulated by an Expectation Maximization (EM) like algorithm. The network learns the parameters f_ϕ and ϕ_α by optimization of Eq.39 and get a label reassigning matrix γ by solving the optimization problem in Eq. 44 with respect to Γ . It is executed by performing the following steps in repetition.

decide iterations for sinkhorn

- Step 1: representation learning. Having the current probability matrix γ , the weights of the model are updated by optimizing Eq. 39 with respect to the parameters f_ϕ and ϕ_α . It is in line with the supervised case where the model uses the cross entropy loss for classification.
- Step 2: label reassigning. On having the updated set of f_ϕ and ϕ_α , Γ is calculated by using Eq. 44. The posterior probability matrix is then computed $\gamma = N \cdot \Gamma$

Thus each update step requires a single matrix-vector multiplication with has the time complexity of $O(N \times J)$ which means the computation time increases linearly with the increase in the number of points in the point cloud. Thus scalability of this method is quite high, and the process remains relatively quick even for a million samples in the dataset. Moreover, the orthogonal regularization was introduced to circumvent the issue of having the same output vector for all cluster representatives. It is computed by

$$\mathcal{L}_{orth}(C) = \|C_*^T C_* - I\|_1, \quad (45)$$

where $\|\cdot\|_1$ is the l_1 -norm or the Manhattan distance and $C_* = [\frac{c_1}{\|c_1\|_2}, \frac{c_2}{\|c_2\|_2}, \dots, \frac{c_J}{\|c_J\|_2}]$. The pseudo-code for the point-level clustering module is elaborated in 1

4.4 Instance-level Contrasting Module

This module plays a crucial role in learning the global features of the point clouds. The architecture of which is shown in Fig. 4.4.1. The network is fed with two augmented views $\mathcal{P}^a, \mathcal{P}^b$ of the same point cloud $\mathcal{P}$. The feature representation obtained as the output of the encoder backbone f_ϕ , are fed to a MaxPooling layer ρ , the output of which is fed to a projection MLP head g [11]. The encoder backbone f_ϕ and the projection head g shares the weights for both the augmented views. The output of the projection head of one of the augmented views (the top branch) is fed to the prediction head q . The purpose of this unit is to transform the output of one of the augmented views to match the other. The operation is applied to only one augmented view to make the network asymmetric [25]. In other words, the output vectors can be realized as $q^a \triangleq \mathbf{q}(g(\rho(\mathcal{F}^a)))$ and $z^b \triangleq g(\rho(\mathcal{F}^b))$ where $\mathcal{F}^a = f_\phi(\mathcal{P}^a)$ and $\mathcal{F}^b = f_\phi(\mathcal{P}^b)$. Keeping in line with

what does it mean to be asymmetric

[11], stop-gradient (stopgrad) was applied on the second branch z^b to resist the network from generating a constant mapping without using negative samples for contrastive learning. The network is constructed such that the similarity between q^a and z^b is maximized. Mathematically speaking, the mean-squared error between the l_2 -normalized prediction q^a and projection z_b is minimized as shown in Eq. 46.

$$\mathcal{D}(q^a, z^b) \triangleq \left\| \frac{q^a}{\|q^a\|_2} - \frac{z^b}{\|z^b\|_2} \right\|_2^2 = 2 - \frac{2q^{aT}z^b}{\|q^a\|_2 \cdot \|z^b\|_2}, \quad (46)$$

In other words, it is the negative cosine similarity, up to a scale of 2. Since, stopgrad is enforced on z^b , Eq. 46 is modified by the following

$$\mathcal{D}(q^a, \text{stopgrad}(z^b)) \quad (47)$$

which means, z^b acts as a constant vector in this equation. Keeping in line with [11], a global symmetrized loss was defined as in Eq. 48

$$\mathcal{L}_{global} = \mathcal{D}(q^a, \text{stopgrad}(z^b)) + \mathcal{D}(q^b, \text{stopgrad}(z^a)) \quad (48)$$

The minimum value the above equation can attain is 0. This is because, in the first term of the equation, the encoder backbone q^a gets no gradient flow from z^b . But it gets gradients from q^b in the second term. In the same way, in the second term of the equation, the encoder backbone q^b gets no gradient flow from z^a . But it gets gradients from q^a in the first term. if the stopgrad mechanism was not applied, then in spite of having a loss of zero during training, the representations learnt by the network would be futile i.e., all the point clouds would be mapped to the same feature representation. or in other words, the network would crumble to constant mapping [11].

4.5 Loss function

The objective of the network is to simultaneously minimize both the point-level clustering loss and the instance level contrasting loss. If the two augmented views of the point cloud $\mathcal{P}$ are $\mathcal{P}^a$ and $\mathcal{P}^b$, then the point-level clustering loss can be calculated by Eq. 49

$$\mathcal{L}_{local} = \varepsilon(\gamma^a, S^a) + \varepsilon(\gamma^b, S^b) + \eta(\mathcal{L}_{orth}(C^a) + \mathcal{L}_{orth}(C^b)) \quad (49)$$

decide value of eta

where $\eta > 0$ is a regularization parameter. The value of η is set to in this work. Here γ^a is the posterior class probability of the point belonging to one of the subcategories, S^a is the predicted class probability and C^a is the normalized center of the augmented view of the point cloud $\mathcal{P}^a$. In the same way, the corresponding terms for the other augmented view $\mathcal{P}^b$ are γ^b , S^b and C^b . Therefore, the total loss could be calculated as the linear combination of $\mathcal{L}^{global}$ and $\mathcal{L}^{local}$ as

$$\mathcal{L}_{total} = \mathcal{L}_{local} + \mathcal{L}_{global} \quad (50)$$

4.6 Dimensionality reduction

4.6.1 T-distributed Stochastic Neighbor Embedding

A technique for reducing dimensionality, t-SNE is mostly used for data visualization in 2D and 3D maps. This approach has become well-liked since it can identify non-linear relationships in the data. The math underlying t-SNE has been walked through in this subsection after providing an intuitive explanation of it. If the data has more than two or three features, one might want to consider looking for clusters in the data. One can use this information to better understand the data and determine how many clusters to use in clustering models like kmeans, if necessary. To further grasp what one hopes to obtain, the following example is used for illustration. It is assumed that the data is to be converted from a 2D space to a 1D dimension as shown in Fig. 4.6.1 and Fig. 4.6.2 respectively.

Every color in this illustration denotes a cluster. It is evident that the densities of each cluster differ. It can be observed that at least two of the clusters overlap when simple projection the data onto one of its dimensions is performed. This highlights that a better method is necessary for the dimension reduction task. This problem is handled by the t-SNE algorithm, and its effectiveness is described in the following steps.

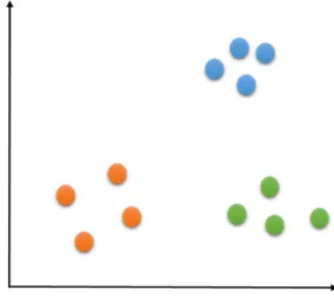


Figure 4.6.1: The original data in 2-D space

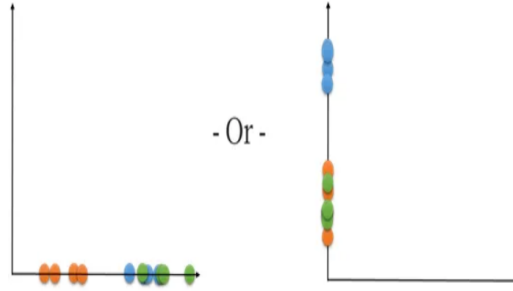


Figure 4.6.2: Data projected to 1-D space

- Determining a joint probability distribution that captures the commonalities amongst the data points.
- Constructing a dataset of the points with the target dimension and thereafter computing the joint probability distribution of those points as well.
- Gradient descent is used to transform the low-dimensional dataset into a joint probability distribution that is as close as feasible to the high-dimensional dataset.

Working principle of t-SNE

1. The likelihood of points to belong to the same neighborhood

Finding each point's Euclidian distance from every other point is the initial step in the process. Next, these distances are converted into conditional probabilities, which express how similar one pair of points is to the other. This means that it is required to determine the degree of similarity—that is, the likelihood—between any two points in the data. A Gaussian with a standard deviation of σ_i and a center at x_i represents the conditional probability of point x_j to be next to point x_i . The conditional probability is defined mathematically as shown in Eq. 51. The factors that effect σ_i are discussed later.

$$p_{j|i} = \frac{\exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{\|x_i - x_k\|^2}{2\sigma_i^2}\right)} \quad (51)$$

Since the clusters can have varying densities, which is why the division by the total of all the other points positioned at the Gaussian centered at x_i is necessary. Looking back to Fig. 4.6.1 as an example to clarify, the orange cluster has a lesser density than the blue. As a result, there will be less similarity between the orange and blue points if one uses a Gaussian to compute the commonalities between each pair of points. The normalization is performed because it is only necessary to view the clusters as such in the final output, and if any of the clusters have differing densities need not be visualized. The joint probability distribution can be computed from the conditional distributions as shown in Eq. 52.

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n} \quad (52)$$

One of the ways that the t-SNE approach differs from the previous SNE is that it uses the joint probability distribution instead of the conditional probability. The computation in the third stage of the algorithm is made simpler by the symmetric property of the pairwise similarities ($p_{ij} = p_{ji}$).

2. Transformation of the data to a lower dimension

In this step, a joint probability distribution for the points in the low-dimensional space is computed. In order to achieve that, a new dataset is created at random that has K features (K being the desired dimension) and the same amount of points as the original dataset. As the dimension reduction is performed for visualization, K will often be two or three. Returning to our example, the t-SNE



Figure 4.6.3: A random dataset created in 1D space.



Figure 4.6.4: Visualization of the dataset in 1D space after applying t-SNE.

algorithm now creates a random dataset of 1D points as shown in Fig. 4.6.3. Similar to the original dataset, the joint probability distribution for this set of points is generated using the t-distribution rather than the Gaussian distribution. The points in this case are denoted as y and the probabilities as q , and thus the joint probability distribution is computed as shown in Fig. 53

$$q_{ij} = \frac{(1 + \|y_i - y_j\|)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|)^{-1}} \quad (53)$$

The heavy tails attribute of the t-distribution is the rationale behind selecting it over the Gaussian distribution. This property helps minimize "crowding" of the points in the lower dimension by making moderate gaps between points in the high-dimensional space become extreme in the low-dimensional space. An additional benefit of employing the t-distribution is that it enhances the algorithm's optimization procedure in the third stage.

3. Modification of the dataset in the low-dimensional space for the best possible data visualization

Now, the Kullback–Leibler divergence (KL) divergence is utilized to maximize the similarity between the joint probability distribution of the data points in the low dimensional space and the higher dimensional space. The KL divergence serves as a gauge for the degree of difference between two distributions. The KL divergence is defined for distributions P and Q in the probability space of χ by Eq. 54.

$$D_{\text{KL}}(P||Q) = \sum_{x \in \chi} P(x) \log \left(\frac{P(x)}{Q(x)} \right) \quad (54)$$

In instances where the distributions are identical, the KL divergence value approaches 0, indicating an increase in similarity between the distributions. The lower dimension dataset is modified so that, in the joint probability distribution, it resembles the original data as soon as possible. To do this, gradient descent is used. The KL divergence between the joint probability distributions P from the high-dimensional space and Q from the low-dimensional space is the cost function that the gradient descent algorithm seeks to minimize as defined in Eq. 55.

$$C = \text{KL}(P||Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (55)$$

The low dimension dataset's point values are obtained from this optimization, which is then utilized for visualizing the dataset in the lower dimensional space. Referring to the example before, the clusters in the low-dimensional space appears as in Fig. 4.6.4. The t-SNE model has a number of hyperparameters that can fine-tuned depending on the use-case. A few of these have to do with gradient descent; the two most crucial ones are the number of iterations, the learning rate and perplexity. It serves to select the Gaussian expressing the conditional distribution in high-dimensional space's standard deviation, σ_i . It can be defined as the count of each point's effective neighbors. For perplexities ranging from 5 to 50, the model is fairly robust.

4.7 Clustering Algorithm

Once the feature representations were obtained, multiple clustering algorithms were utilized to group the objects in the datasets into clusters. For this, one algorithm from each type of clustering algorithms as mentioned in 3.3 has been implemented to analyse the behavior of different clustering algorithms in this

use case. Since the ultimate goal of the task in hand is to find representative members of the dataset, it is more robust to find the medoid of the cluster instead of the arithmetic mean. As mentioned in , the medoids are actual datapoints of the dataset which is more practical to use in this use case instead of using a rather blurred, noisy arithmetic mean of the clusters. So, the scikit-learn implementation of the KMedoids algorithm was one of the clustering algorithm that was analysed for the task in hand.[69]

Working principle of KMedoids algorithm

1. At first, k points are selected from the dataset which act as the initial medoids. Here, k is the number of clusters the dataset is to be partitioned into.
2. The non-medoid points in the cluster are then assigned to the cluster of the closest medoid.
3. Then the total sum of distances or the cost of each cluster is calculated by computing the distance of the non-medoid points from the medoid.
4. Each non-medoid point is then considered to be the medoid of the cluster and the total sum of distances is recomputed.
5. Lastly, it checks if the recalculated cost is greater than the previously calculated cost. If this is true, it reverses the swap and the algorithm converges. If the calculated cost is lesser, the same process repeated from step 4.

After the algorithm converges, k medoid points are obtained which are the representative members of k clusters. The Euclidean distance metric or the l_2 -norm as shown in Eq. 56 was used for this algorithm.

$$d_{eq}(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_n - q_n)^2} \quad (56)$$

where, p and q are the cartesian coordinates of the datapoints in the n -dimensional Euclidean space. Other distance metrics like the Manhattan distance or the l_1 -norm as shown in Eq. 57 could also be used for this purpose.

$$d_{man}(p, q) = |p_1 - q_1| + |p_2 - q_2| + \dots + |p_n - q_n| \quad (57)$$

Different strategies could be adopted to select the initial medoids for the algorithm. It could be "random", "heuristic", "k-medoids++" or "build". The "random" initialization method selects k elements from the dataset at random. The "heuristic" method selects the k points such that the distance it has the smallest sum of distance to all other points. The "k-medoids++" methods adopts a strategy similar to the "k-means++". It tends to choose the initial points in a way such that the initial medoids are more separated than the other methods. The "build" is a greedy initialization method that was originally used in the Partitioning Around Medoids (PAM) algorithm.[69]. Another clustering algorithm that was suggested for this task is the "multi-level HDBSCAN" clustering. The traditional implementation of the HDBSCAN algorithm available in scikit-learn can provide hierarchical density based clusters. However, this algorithm does not provide any control on the number of clusters that can be generated. Thus in the presence of very large datasets like the ABC [39] dataset, it can potentially generate more than a hundred thousand clusters and thus that many number of cluster representative members. Generating that many data for the source library in the current infrastructure present at IPA is extremely computationally expensive and thus technically infeasible. Increasing the minimum cluster size to be considered as a cluster and increasing the minimum number of samples around a point to be considered as a core point can decrease the overall number of cluster, but that would also mean that the algorithm would label more datapoints as outliers. Losing too much information about the objects as outliers can jeopardize the purpose of a source library which is expected to generalize the feature of maximum possible objects in the dataset.

Working principle of multi-level HDBSCAN algorithm

1. At first, HDBSCAN algorithm is applied on the whole dataset. After the clustering algorithm, p medoids are obtained. These p datapoints are the relevant points for the next step.

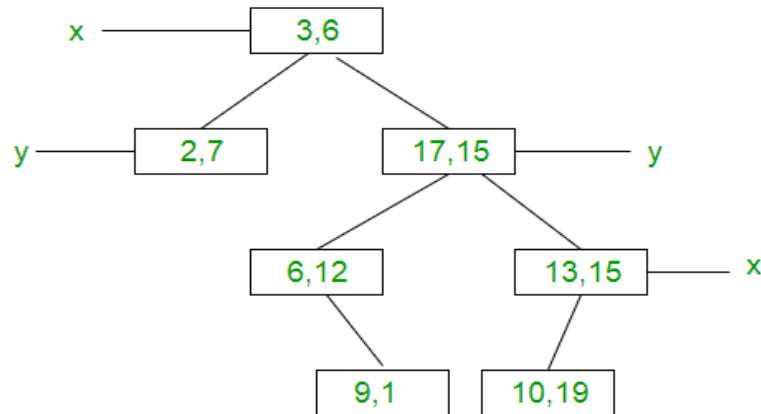


Figure 4.7.1: A demonstration of the KD-tree algorithm with 7 datapoints having 2 dimensions x and y . [70]

2. The HDBSCAN algorithm is applied again on those p datapoints. As a result of it, m clusters are obtained and n datapoints are labelled as outliers. Thus, there would be m medoids which are the representative members of each of the clusters. From this level onwards, no new datapoints would be discarded as outliers. Thus each of the datapoints labelled as outliers would be assigned to a new cluster along with all the fellow cluster member which was a part of the cluster this particular datapoint was a member of in the previous level.
3. Thus, the relevant datapoints for the next level are the m medoids and all the n outliers relabelled as medoids of new clusters. Step 2 and 3 are repeated until convergence, i.e. when $m + n \leq k$ where k is the maximum number of permissible clusters.

After the convergence of the algorithm, c medoid points are obtained which are the representative members of the c clusters where $c \leq k$. Similar to the KMedoids algorithm, the Euclidean distance metric as shown in Eq. 56 is used to compute the medoids. The pseudo-code for this algorithm is shown in alg. 5. The "min_cluster_size" hyperparameter controls the minimum number of datapoints to be contained in a cluster. Increasing this value would mean clusters with more number of elements. But this would also mean that groups smaller than this size would be treated as outliers. Thus increasing this value could lead to significant loss of information about the features of different objects in the cluster. To circumvent this issue, the value of this parameter was kept at its default value which is 2. This is the reason why HDBSCAN algorithm returns a significantly high number of clusters. Thus, the proposal of multi-level HDBSCAN algorithm was deemed necessary for this use case. The HDBSCAN algorithm performs the DBSCAN algorithm for different epsilon values and hence provides a more robust clustering algorithm. Moreover, the optimum value of epsilon is case-dependent and no single value is suitable for clustering different datasets. This also means that expert domain knowledge is essential for determining the optimum epsilon value and is difficult to obtain in this use case. The HDBSCAN algorithm incorporates the result of clustering for different epsilon value and finds the clustering solution that provides the optimum stability over all epsilon. The HDBSCAN algorithm can opt for different algorithms like K-Dimensional (KD)-trees, Ball Tree or brute-force algorithm.

KD-tree algorithm The KD-tree is a data structure that is used for the purpose of effectively organising and searching for points in multi-dimensional spaces. Specifically, KD-trees excel at solving the nearest neighbor search problem, which involves finding the closest data points to a given query point. The structure of a KD-Tree resembles a binary tree, where each node can have up to two children. It operates by recursively dividing data points along different dimensions, creating a hierarchical arrangement that facilitates efficient search and retrieval as shown in Fig. 4.7.1. At each level, the tree selects a dimension and splits the data points into two groups based on their values in that dimension. This process continues until all data points are assigned to leaf nodes. One of the key advantages of KD-Trees lies in their ability

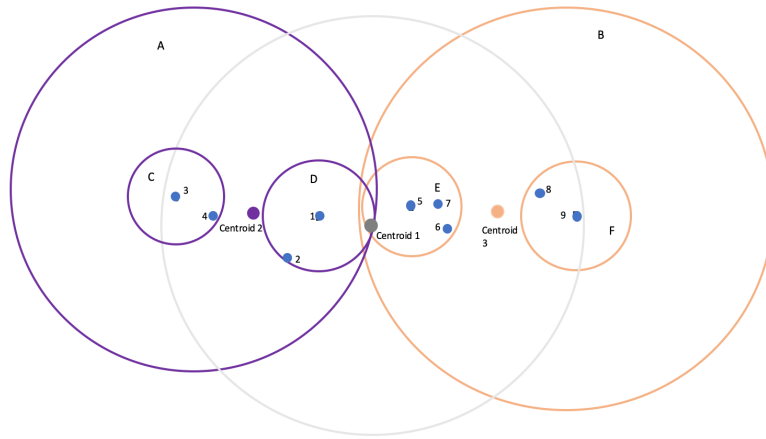


Figure 4.7.2: A demonstration of the Ball-tree algorithm with 9 datapoints having 2 dimensions x and y . [70]

to handle high-dimensional data in static dataset—a common challenge in machine learning applications like computer vision, natural language processing, and bioinformatics. High-dimensional data often suffers from the “curse of dimensionality,” where the search space grows exponentially with the number of dimensions, making nearest neighbor queries time-consuming. As the ABC[39] dataset is an immensely high dimensional dataset, the KD-Tree algorithm was used in this use case. KD-Trees address this by narrowing down the search space at each tree level, resulting in faster queries. Despite their advantages, KD-Trees face limitations and challenges. One concern arises when the number of dimensions increases, particularly with non-uniformly distributed data points. In such cases, the tree may become unbalanced, resulting in inefficient search times. But since the datapoints were reduced to 2-dimensional datapoint in this case, the HDBSCAN algorithm doesn’t result to inefficient search times in this case. Also KD-trees are sensitive to outliers. The steps for KD-tree algorithm are:[22]

1. **Dimension Selection:** A dimension is chosen along which the data is split.
2. **Sorting:** The data points are sorted and arranged.
3. **Median Point:** The median point (root of the current subtree) is selected, splitting the data into two subsets.
4. **Recursive Process:** The process is repeated for each subset, with a new dimension selected at each level.

Ball-Tree algorithm The ball tree algorithm is designed for organizing and efficiently querying multidimensional data in computational geometry and machine learning. Formally, subsets of points are enclosed within hyperspheres, and the structure is constructed as a binary tree. Each non-leaf node represents a hypersphere containing a subset of the data, and each leaf node corresponds to a small subset of points. The partitioning process involves choosing a ‘pivotal’ point within the subset and constructing a hypersphere centered at this point to enclose the data as shown in Fig. 4.7.2. Fast nearest neighbor searches are facilitated by this hierarchical arrangement, allowing for the rapid elimination of entire subtrees during the search process. Ball trees are particularly useful in scenarios with high-dimensional data, offering improved efficiency over exhaustive search methods in applications such as clustering, classification, and nearest neighbor queries. The steps for ball-tree algorithm are:[56]

1. **Point Selection :** A random data point is chosen as the center of the hypersphere.
2. **Radius Calculation:** The radius of the hypersphere is calculated using distance metrics such as Euclidean distance or Manhattan distance.

3. **Datapoint Assignment:** Data points are assigned to the current node.
4. **Recursive Process:** The process is repeated by selecting new centers and radii.

Thus, the memory consumption for the ball-tree algorithm is way more in comparison to the KD-tree algorithm. Furthermore, it fails to provide good results in low-dimensional data. It results to inefficient search time for low dimensional queries. This KD-tree is suitable for low dimensional static use cases like image recognition and nearest neighbor search in low to medium dimensional dataset. Whereas, the ball tree algorithm is suitable for data clustering in high dimensional spaces and nearest neighbor search in high dimensional datasets.

Brute Force Algorithm A brute force algorithm is a straightforward and exhaustive search strategy that systematically examines all possible options until it finds the solution to a problem. It's a general approach used for problem-solving when the problem size allows for detailed investigation. In the brute force method, it is iterated through each data point and the density is calculated by considering the distances to other points. However, due to their high time complexity, brute force methods are inefficient for large-scale problems.

4.8 Important aspects of the Studies

4.8.1 Focus and Restrictions

4.9 Suitable Algorithm and Adjustments

Algorithm 1 Pseudo-code for point-level clustering

Input: $\{\mathcal{P}\}$: is a set of 3D point clouds with N number of points in it; K : number of epochs.

Output: backbone f_ϕ .

```

1: for  $i$  in range  $(0, K)$  do
2:    $\mathcal{L} = 0$ 
3:   for  $\mathcal{P} \in \{\mathcal{P}\}$  do
4:     # Computes Class probability
5:      $S = \text{softmax}(\phi_\alpha(f_\phi(\mathcal{P})))$ 
6:     # Computes cluster representatives of partitions
7:      $C = \left\{ \frac{1}{\sum_{i=1}^N s_{ij}} \sum_{i=1}^N s_{ij} \mathbf{p}_i \right\}_{j=1}^N$ 
8:     # Computes mean-squared error
9:      $D = \left\{ \|\mathbf{p}_i - c_j\|_2^2 \right\}_{i,j}^{N,J}$ 
10:    # Computes posterior probability matrix
11:     $\gamma = \text{sinkhorn}(\text{stopgrad}(D), 1e-3, 20)$ 
12:    # Computes Loss Function
13:     $\mathcal{L} = \varepsilon(\gamma, S) + \eta \mathcal{L}_{\text{orth}}(C)$ 
14:  end for
15:  # Updates backbone, projector and predictor
16:   $f_\phi, \phi_\alpha \leftarrow \text{optimize}(\frac{\mathcal{L}}{N})$ 
17: end for
18: return  $f_\phi$ 

```

Algorithm 2 Pseudo-code for instance-level clustering**Input:** $\{\mathcal{P}\}$: is a set of 3D point clouds with N number of points in it; K : number of epochs.**Output:** backbone f_ϕ .

```

1: for  $i$  in range  $(0, K)$  do
2:    $\mathcal{L} = 0$ 
3:   for  $\mathcal{P} \in \{\mathcal{P}\}$  do
4:     # Generate random augmentations
5:      $\mathcal{P}^a = \text{aug}(\mathcal{P})$ 
6:      $\mathcal{P}^b = \text{aug}(\mathcal{P})$ 
7:     # Compute projections
8:      $z^a = g(\rho(f_\phi(\mathcal{P}^a)))$ 
9:      $z^b = g(\rho(f_\phi(\mathcal{P}^b)))$ 
10:    # Compute predictions
11:     $q^a, q^b = q(z^a), q(z^b)$ 
12:    # Calculate loss
13:     $\mathcal{L} += \mathcal{D}(q^a, \text{stopgrad}(z^b)) + \mathcal{D}(q^b, \text{stopgrad}(z^a))$ 
14:   end for
15:   # Updates backbone, projector and predictor
16:    $f_\phi, g, q \leftarrow \text{optimize}(\frac{\mathcal{L}}{N})$ 
17: end for
18: return  $f_\phi$ 

```

Algorithm 3 Pseudo-code for Sinkhorn-Knopp algorithmdef sinkhorn($D, \epsilon, \text{niters} = 3$)

```

1:  $\Gamma = \exp(D/\epsilon)^T$ 
2:  $\gamma/ = \text{sum}(\Gamma)$ 
3:  $N, J = \Gamma.\text{shape}$ 
4:  $u, r, c = \text{zeros}(N), \text{ones}(J)/J, \text{ones}(N)/N$ 
5: for  $\text{inrange}(0, \text{niters})$  do
6:    $u = \text{sum}(\Gamma, \text{dim} = 1)$ 
7:    $\Gamma^* = (r/u).\text{unsqueeze}(1)$ 
8:    $\Gamma^* = (c/\text{sum}(\Gamma, \text{dim} = 0)).\text{unsqueeze}(0)$ 
9: end for
10: return  $\Gamma$ 

```

Algorithm 4 Pseudo-code for hierarchical clustering**Input:** $\{\mathcal{E}\}$: is the set of feature representations for all objects in the dataset, K is the number of permissible clusters.**Output:** $\{\mathcal{P}_R\}$ is the set of representative members of the dataset.

```

1:  $\mathcal{E}_R \leftarrow \{\mathcal{E}\}$ 
2:  $\text{classifier}, \text{predictions}, \text{num\_clusters} = \text{HDBSCAN}(\mathcal{E}_R)$ 
3:  $\text{count} \leftarrow 0$ 
4:  $C \leftarrow \text{predictions}$ 
5:  $\text{cluster\_members} = \text{get\_cluster\_members}(C, \{\mathcal{E}\})$ 
6:  $\text{clusters} = \text{list}(\text{unique}(C))$ 
7: while  $\text{True}$  do
8:    $\mathcal{E}_R, C, \text{cluster\_members}, \text{num\_clusters}, \text{classifier} = \text{get\_hier\_clusters}(\mathcal{E}, \mathcal{E}_R, C, \text{cluster\_members}, \text{count})$ 
9:    $\text{count} += 1$ 
10:   $\text{total\_num\_clusters} = \text{len}(\text{list}(\text{unique}(C)))$ 
11:  if  $\text{total\_num\_clusters} \leq K$  or  $\text{n\_clusters}$  is 0 then
12:    break
13:  end if
14: end while
15:  $\text{medoid\_indices} = \text{get\_medoid\_indices}(\mathcal{E}_R, \mathcal{E})$ 
16:  $\{\mathcal{P}_R\} = \{\mathcal{P}_i, \mathcal{P}_i \in \{\mathcal{P}\} \text{ and } i \in \text{medoid\_indices}\}$ 
17: return  $\{\mathcal{P}_R\}$ 

```

Algorithm 5 Pseudo-code for multi-level HDBSCAN clustering

Input: $\{\mathcal{P}\}$: is a set of 3D point clouds with N number of points in it, $\{\mathcal{E}\}$: is the set of feature representations for all objects in the dataset, $\{\mathcal{E}_R\}$ is the set of feature representations for relevant points objects in the dataset, C is the cluster membership of all the points clouds in the dataset, $clusters$ is the list of all clusters, $clusters_members$ is the list of all the point clouds in the dataset, grouped by their cluster membership, $count$ is the current level of multi-level HDBSCAN

Output: return $\mathcal{E}_R, C, cluster_members, num_clusters, classifier$

```

1:  $classifier, predictions, num\_clusters = HDBSCAN(\mathcal{E}_R)$ 
2:  $medoids = classifier.medoids$ 
3:  $cluster\_pointer \leftarrow num\_clusters$ 
4: Assign Empty list to  $outlier\_list$ 
5: for  $\forall obj\_idx, object \in enumerate(\mathcal{E}_R)$  do
6:   if  $prediction[obj\_idx] < 0$  then
7:      $AddItem(outlier\_list, obj\_idx)$ 
8:      $C[cluster\_members[obj\_idx]] = cluster\_pointer$ 
9:   else
10:     $C[cluster\_members[obj\_idx]] = prediction[obj\_idx]$ 
11:   end if
12: end for
13:  $\{O\} = \{\mathcal{E}_i, \mathcal{P}_i \in \{\mathcal{E}_R\} \text{ and } i \in outlier\_list\}$ 
14:  $cluster\_members = get\_cluster\_members(C, \{\mathcal{E}\})$ 
15:  $medoid\_indices = get\_medoid\_indices(medoids, \mathcal{E}_R)$ 
16:  $\mathcal{E}_R = \{\mathcal{E}_i, \mathcal{E}_i \in \{\mathcal{E}_R\} \text{ and } i \in medoid\_indices\}$ 
17: if  $count \neq 0$  then
18:    $\mathcal{E}_R = concatenate(\mathcal{E}_R, O)$ 
19: end if
20: if  $count$  is 0 then
21:    $num\_clusters = len(list(unique(C)))$ 
22: end if
23: return  $\mathcal{E}_R, C, cluster\_members, num\_clusters, classifier$ 

```

4.10 Whatever new Concept we propose**4.11** Source and Target Objects**4.12** Data Generation**4.13** Important Aspects

5 Application of algorithms

6 Experiments

The two different backbone architecture for the generation of the latent space representation of the objects and the performance of the different clustering algorithms are evaluated by hyperparameter optimization of the different parameters along the entire pipeline of the procedure is evaluated in this section.

6.1 Baseline algorithm

The already existing PointNet autoencoder currently used at IPA is used as a baseline for evaluating the performance of the proposed algorithms in this thesis. The dimension of the output of the latent space representation of the baseline algorithm is 64. As mentioned in the restrictions in Section. 4.8.1, usually a two-step dimensionality reduction is performed before the datasets are grouped into clusters for the two networks evaluated in this thesis because the dimension of the output of the latent space representations is too high. This is done to suppress some noise and also to escalate the procedure by faster calculation of the pairwise distance between the datapoints. But since an output dimension of 64 is not very high, two-step dimensionality reduction is not necessary. However, to make the baseline algorithm comparable with the proposed networks, one-step dimensionality reduction is performed in some cases.

6.1.1 Basic Study without dimensionality reduction

Table 3 shows the results of the baseline algorithm when no dimensionality reduction technique is used. It shows the results of the basic algorithm for the clustering of 1000 datapoints. The unit of time across all the tables in this section is in seconds unless otherwise mentioned. The four clustering algorithms HDBSCAN, Kmeans clustering, spectral clustering and agglomerative clustering are evaluated. The number of clusters generated by the HDBSCAN algorithm cannot be fixed by the user and hence it is determined by the algorithm automatically. Thus, the number of clusters after the first level of the multi-level HDBSCAN algorithm as proposed in Section. 4.7 is 232 in this case. As mentioned in Section 4.8.1 the number of clusters for the rest of the clustering algorithms is set to 175 throughout all the experiments in this section. The DBCV score varies in the range $[-1,1]$ with more positive results denoting better clustering of the datapoints. The reason why HDBSCAN algorithm shows better results as compared to other clustering algorithms is HDBSCAN algorithm is a density based clustering algorithm. DBCV being a density based clustering validation index is likely to give better results for density based clustering algorithms which is proven to be true in this experiment. Furthermore, theoretically HDBSCAN is better capable of handling noises and outliers in the dataset which can also contribute to the better results as exhibited in this experiment.

include experiment on multi-level HDBSCAN

No. of datapoints	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
1000	HDBSCAN	232	0.182	-0.189	297.031
	Kmeans	175	3.960	-0.332	237.640
	Spectral	175	2.396	-0.339	161.604
	Agglomerative	175	0.115	-0.327	157.584

Table 3: Results of the baseline algorithm on 1000 datapoints and without any dimensionality reduction technique. The unit of time is in seconds.

6.1.2 Basic Study with t-SNE

Table 4 shows the results of the baseline algorithm when the t-SNE dimensionality reduction is applied. The number of components for t-SNE could be 1, 2 or 3 as per the available scikit-learn implementation [61]. It was chosen to be 2 in these set of experimentation for better and easier visualization. The DBCV score

caries in the range $[-1,1]$ with more positive results denoting better clustering of the datapoints. The results are seen to be significantly better as compared to Table 3 when dimensionality reduction technique was not applied. This is because the t-SNE algorithm tends to keep datapoints that are distant from one another in the higher dimensional space also distant datapoints even in the lower-dimensional space. This gives rise to more dense and distinct clusters thus improving the DBCV score. As opposed to the result in Tab.3, where the DBCV score was negative, meaning the datapoints were mostly wrongly classification, in this table it can be seen that the datapoints are mostly correctly classified. It is also to be note that the number of clusters generated by the HDBSCAN algorithm is 286 which is a bit more as compared to the previous case. This further hints to the direction that as a result of using t-SNE, more dense and non-overlapping clusters are formed in the dataset.

No. of data-points	t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
1000	2	3.030	HDBSCAN	286	0.110	0.0975	275.039
	2	2.777	Kmeans	175	0.191	-0.052	94.004
	2	3.318	Spectral	175	3.604	-0.064	126.854
	2	3.190	Agglomerative	175	0.087	-0.076	113.937

Table 4: Results of the baseline algorithm on 1000 datapoints after applying t-SNE dimensionality reduction. The unit of time is in seconds.

6.1.3 Basic Study with increasing datapoints

As mentioned in Section 4.8.1, it is extremely inefficient to compute the DBCV for all the datapoints in the ABC datasets [39] as once, it was necessary to verify that the results obtained for a smaller subset of the dataset can be interpolated for the entire dataset. In other words, the results obtained for a smaller subset of the dataset generalises the behavior of the entire dataset well. Table 5 shows the results of the baseline algorithm on 2000 datapoints without applying dimensionality reduction and table 6 shows the results of the baseline algorithm on 2000 datapoints when t-SNE dimensionality reduction is applied. It was observed that roughly 10% of the datapoints were classified as outliers by the HDBSCAN algorithm for all the experiments on 1000 and 2000 datapoints.

No. of datapoints	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2000	HDBSCAN	436	0.444	-0.234	2115.777
	Kmeans	175	0.793	-0.462	764.222
	Spectral	175	20.607	-0.617	749.487
	Agglomerative	175	0.205	-0.502	756.480

Table 5: Results of the baseline algorithm on 2000 datapoints and without any dimensionality reduction technique. The unit of time is in seconds.

No. of data-points	t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2000	2	6.689	HDBSCAN	626	0.237	0.172	4497.151
	2	7.450	Kmeans	175	0.155	-0.226	932.509
	2	157.017	Spectral	175	91.684	-0.125	755.647
	2		Agglomerative				

Table 6: Results of the baseline algorithm on 2000 datapoints after applying t-SNE dimensionality reduction. The unit of time is in seconds.

6.2 Conclu approach with PointNet as Backbone

As mentioned in Section 4, two feature representation learning backbones were used for the experiments. Experiments with the PointNet autoencoder as the backbone is recorded here. The performance of all the clustering algorithm has been evaluated. Different hyperparameter optimization has been performed which will be elaborated gradually through this subsection.

6.2.1 Evaluation of increasing datapoints

As mentioned in Section 4.8.1, it is extremely inefficient to compute the DBCV for all the datapoints in the ABC datasets [39] as once, it was necessary to verify that the results obtained for a smaller subset of the dataset can be interpolated for the entire dataset. In other words, the results obtained for a smaller subset of the dataset generalises the behavior of the entire dataset well. Table 7 shows the results of the ConClu approach on 1000 datapoints and table 8 and 9 shows the results of the ConClu approach on 2000 datapoints and 5000 datapoints respectively. It was observed that roughly 10% of the datapoints were classified as outliers by the HDBSCAN algorithm for all the experiments on 1000, 2000 and 5000 datapoints. The dimension of the latent space representations for table 7, 8, and 9 is 1024.

No. of datapoints	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
1000	HDBSCAN	288	0.228	0.137	437.956
	Agglomerative	175	0.063	-0.057	191.887
	Kmeans	175	7.763	-0.120	192.580
	Spectral	175	28.831	-0.051	186.879

Table 7: Results of the ConClu approach on 1000 datapoints. The unit of time is in seconds.

It can be noticed that the DBCV score for the HDBSCAN algorithm increases consistently from 1000 datapoints to 2000 datapoints and so on. This could be attributed to the fact that more datapoints would quite naturally give rise to denser clusters and more dense the inside of the cluster is as compared to its outside, the better is the DBCV score. This further reassures the fact that denser clusters would mean more number of elements in individual clusters. In other words, the representative member of the cluster would represent a more diverse amount of other cluster members.

No. of datapoints	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2000	HDBSCAN	605	1.030	0.159	8278.394
	Agglomerative	175	0.196	-0.212	786.866
	Kmeans	175	6.322	-0.209	1079.195
	Spectral	175	80.008	-0.145	1187.477

Table 8: Results of the ConClu approach on 2000 datapoints. The unit of time is in seconds.

Also it is noteworthy how the time to calculate the DBCV score doesn't increase linearly on increasing the number of datapoints. Rather it increases in an exponential rate. This shows how it would take immensely long to compute the DBCV score of the entire dataset in the presence of 1 million datapoints in it. But since the results obtained on a smaller subset of the dataset can be interpolated to the entire dataset, evaluation hyperparameter optimization for the rest of the parameters are performed on 1000 datapoints. However, after deciding the final values for all the hyperparameters, the cluster representatives are generated taking into account all the datapoints under consideration as that no longer involves calculating the DBCV score anymore for evaluation.

No. of datapoints	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
5000	HDBSCAN	1.586	1.904	0.210	165884.854
	Agglomerative	175	1.345	-0.380	4945.410
	Kmeans	175	9.142	-0.427	8289.148
	Spectral	175	641.189	-0.359	5069.388

Table 9: Results of the ConClu approach on 5000 datapoints. The unit of time is in seconds.

6.2.2 Evaluation of different dimensionality reduction techniques

As mentioned in section 4.8.1, it is essential to perform dimensionality reduction before clustering the objects in the dataset into groups. Two different dimensionality techniques were evaluated for this purpose, the PCA method and the t-SNE method. Table 10 shows the results on the ConClu approach for 1000 datapoints and applying PCA method as the dimensionality reduction technique in both steps. As the DBCV score depends on the number of nearest neighbor raised to the power of the negative of 1 divided by the number of features in the latent space representation as shown in Eq. 14, it is important to determine the to the maximal possible dimension of the representation after the second step of dimensionality reduction. Experiments were performed and the value was found to be 154 in this use case. Also keeping the first step of dimensionality reduction to be PCA, experiments were also conducted with t-SNE method as the second step. Table 11 shows the results on the ConClu approach for 1000 datapoints under such settings. The dimension of the latent space representations in both the cases is set to 1024. it is to be noted that the results for all the clustering algorithms have better performance when t-SNE is used as the dimensionality reduction technique in the second step as compared to PCA. It can be attributed to the fact that the t-SNE algorithm preserves the spatial relationship between the datapoints. In other words, if datapoints are close to one another in a higher dimensional space, t-SNE tends to preserve this spatial closeness between points even in the lower dimensional space.

PCA dim	PCA time	PCA dim	PCA time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
512	32.921	154	11.249	HDBSCAN	169	0.459	-0.410	234.172
	41.230	154	14.891	Agglomerative	175	0.216	-0.151	209.954
	38.408	154	13.032	Kmeans	175	7.001	-0.184	212.482
	44.171	154	13.897	Spectral	175	46.703	-0.481	221.064

Table 10: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints with PCA as dimensionality reduction technique in both steps. The unit of time is in seconds.

Also it is to be noted that the HDBSCAN algorithm performs significantly bad in presence of PCA as compared to t-SNE as the second step of dimensionality reduction. This is also because of the property of t-SNE mentioned above which gives rise to denser clusters as compared to PCA.

PCA dim	PCA time	t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
512	32.530	2	143.024	HDBSCAN	288	0.228	0.137	437.956
	38.832	2	141.566	Agglomerative	175	0.063	-0.057	191.887
	39.326	2	162.957	Kmeans	175	7.763	-0.120	192.580
	71.552	2	177.427	Spectral	175	28.831	-0.051	186.879

Table 11: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints with PCA as dimensionality reduction technique in the first step and t-SNE as the second. The unit of time is in seconds.

The effect of two step dimensionality reduction was also compared against only one step dimensionality reduction as shown in table 12. A PCA followed by a t-SNE shows better results as compared to directly applying t-SNE just once. This is because of the fact that a multi-step dimensionality reduction helps in reducing the noises during the procedure. Thus for the rest of the experiments it was decided to perform the dimensionality reduction with PCA having number of components to be 512 in the first step followed by applying the t-SNE method with 2 components as the next step.

No. of data-points	t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBC score	DBC time
1000	2	6.769	HDBSCAN	314	0.189	-0.120	337.313
	2	4.872	Kmeans	175	0.337	-0.132	131.622
	2	5.831	Spectral	175	7.592	-0.122	130.678
	2	4.478	Agglomerative	175	0.033	-0.085	128.358

Table 12: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints with only t-SNE as dimensionality reduction technique as the only step. The unit of time is in seconds.

6.2.3 Evaluation of dimension of feature space representations

Latent space representations are lower dimensional representations of the dataset. It is expensive to work with very high dimensional data. Thus the dimension of the latent space representations is immensely important. It is extremely necessary that the original high dimensional data is compressed to a lower dimensional latent space without severe loss of information in the process. Since the original dataset had the point clouds with 2048, for the sake of completeness of the experimentation, the evaluation has been performed for all dimensions on a logarithmic scale of 2 from 2048 to down until 4. Table 13 shows the experiments when the dimension of the latent space representations was set to be 2048. As mentioned in Section 4.8.1, because of the very high number of features in the output of the network, a two-step dimensionality reduction was performed to suppress noise during the process of dimensionality reduction and also to facilitate faster calculation of the pairwise distance between the points. In line with the results of the previous experiments, the HDBSCAN clustering algorithm produces the best results. The learning rate was rate set to 0.001 with a decay factor of 0.1 throughout all the experiments in this subsection. A early stopping criteria was used to naturally terminate the training of the models on convergence. When the validation loss no long reduced for atleast 5 epochs, then the training was terminated. The number of epochs was set to a very high number of 200 for all the experiments so that the training is only stopped when early stopping criteria is triggered on model convergence.

PCA dim	PCA time	t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBC score	DBC time
512	18.507	2	3.594	HDBSCAN	289	0.127	0.189	243.297
	15.525	2	3.368	Agglomerative	175	0.036	-0.013	109.668
	16.595	2	3.709	Kmeans	175	0.222	-0.040	102.518
	18.593	2	3.918	Spectral	175	9.257	0.020	101.925

Table 13: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 2048. The unit of time is in seconds.

But larger dimensions of the latent space representations mean longer training time. When it was set to 2048, the model converged after the 50th epoch according to the early stopping criteria mentioned above with a total training time of 137.4 hours which is quite high. So to evaluate the performance of the model with fewer dimensions of the latent space representation further experiments were conducted. Table 11 on page 56 and tab. 14 shows the experiments when the dimension of the latent space representations was set to be 1024 and 512 respectively. The number of semantic subgroups or parts in the objects was set to 64 in line with [50] to find the optimum number of dimensions of the latent space representations. Experiments on the number of semantic subgroups has been performed later on.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	145.809	HDBSCAN	307	0.187	0.186	489.133
	6.575	Kmeans	175	0.532	-0.023	129.956
	4.989	Spectral	175	9.226	0.024	131.239
	6.823	Agglomerative	175	0.053	0.010	131.956

Table 14: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512. The unit of time is in seconds.

The training time for the model with 1024 dimensions was 73 hours for 65 epochs. The training time for the model with 512 dimensions was 64 hours for 54 epochs. The performance for the model with dimension 512 is comparable to the model with dimension 2048 for a much lower training time. If a good enough result could be achieved with a significant decrease in the training time, it would mean a more cost efficient model.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	5.901	HDBSCAN	293	0.725	0.164	718.808
	6.206	Kmeans	175	0.471 secs	-0.078	136.173
	6.106	Spectral	175	11.077	0.003	133.593
	5.539	Agglomerative	175	0.031	-0.020	132.193

Table 15: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 256. The unit of time is in seconds.

It is to be noted here that the performance of the HDBSCAN algorithm was worse for dimension 1024 as compared to both dimensions 2048 and 512. To achieve a conclusive result as to whether the performance of the HDBSCAN algorithm on the model with dimension 1024 is an outlier, further experiments were conducted to train models with fewer dimensions.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.036	HDBSCAN	307	0.121	0.174	354.460
	3.689	Kmeans	175	5.255	-0.037	177.251
	3.295	Spectral	175	5.511	0.024	116.696
	3.531	Agglomerative	175	0.057	-0.033	94.799

Table 16: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 128. The unit of time is in seconds.

Table 15 and tab. 16 are the experiments for the model with dimensions 256 and 128 respectively with rest of the parameters being same as mentioned above. The training time for the model with dimension 256 is 42 hours for 50 epochs. Whereas, the training time for the model with dimension 256 is 32 hours for 40 epochs. It was observed that there wasn't a significant change in the training time as compared to training time of the model with dimension of 512 to 256 as it was observed when the dimension changed from 2048 to 512. In the latter, when the dimension changed by a factor of 4, the training time reduced by almost a factor of 2. Whereas, when the dimension changed from 512 to 128, there was a change in the dimension by a factor of 4, but the training time did not reduce comparably. Also, the performance of the HDBSCAN algorithm was better with the model of dimension 512 by 2% and 1% as compared to models with dimension 256 and 128 respectively. Until now, the number of semantic subgroups in the objects were set to 64. In other words, each dimension of the feature representation of the point cloud is to belong to 1 of the 64 semantic subgroups or parts. Thus when the dimension of the latent space representation of the objects is atmost 64, it is not realistic to have 64 semantic subgroups. So to train the models with dimensnions of latent space representation to be less than equal to 64, the number of semantic subgroups is to chosen to be half of the number of dimensions. So the model of dimension 64 has K set to be 32, the model of dimensions 32 has K set to be 16 and so on. Thus, table 17 and tab. 18 are the experiments for the model with dimensions 256 and 128 respectively with rest of the parameters being same as mentioned above. The training time for the model with 64 dimensions was 45 hours for 48 epochs. The training time for the model with 32 dimensions was 23 hours for 39 epochs.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.059	HDBSCAN	286	0.538	0.148	639.097
	3.086	Kmeans	175	3.143	-0.068	145.711
	3.189	Spectral	175	8.760	-0.020	152.836
	3.914	Agglomerative	175	0.032	-0.017	157.152

Table 17: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 64. The unit of time is in seconds.

As expected, the performance of these models have decreased by 4% and 3% respectively as compared to

the model with dimensions 512. When the number of dimensions have decreased by a factor of 16, the amount of training time has only reduced by a factor of 3 when the dimensions have changed from 512 to 32. The learnign rate for all the experiments in this subsection was set to 0.001 with a decay factor of 0.1. A batch size of 128 was chosen for these experiments as that was the maximum possible batch size that could be executed without running into out of memory issues in the given hardware setup.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBC score	DBC time
2	2.748	HDBSCAN	273	0.222	0.158	562.223
	2.917	Kmeans	175	1.434	-0.021	141.890
	2.985	Spectral	175	6.190	0.016	107.808
	2.501	Agglomerative	175	0.031	0.023	93.092

Table 18: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 32. The unit of time is in seconds.

Experiments were also conducted for training the models with dimensions 16, 8 and 4 for the completeness of the experiments. Since the dimensions of the latent space representations are to be categorized into subgroups, the minimum number of possible subgroups is 2. Therefore, a model with the dimension of the latent space representations as 2 is not realistic. So the minimum number of dimensions of the latent space representations is 4.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBC score	DBC time
2	2.673	HDBSCAN	287	0.419	0.168	621.601
	3.240	Kmeans	175	2.276 secs	-0.023	147.403
	3.005	Spectral	175	6.064	0.027	109.908
	3.469 secs	Agglomerative	175	0.031	0.005	92.398

Table 19: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 16. The unit of time is in seconds.

Table 19, 20 and tab. 21 are the experiments for the model with dimensions 16, 8 and 4 respectively with rest of the parameters being same as mentioned above.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	2.851	HDBSCAN	290	0.187	0.135	551.979
	2.529	Kmeans	175	7.996	-0.028	146.129
	2.693	Spectral	175	3.949	-0.026	115.756
	2.714	Agglomerative	175	0.033	-0.020	93.977

Table 20: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 8. The unit of time is in seconds.

The training time for the model with 16 dimensions was 16 hours for 29 epochs. The number of subgroups for this model was set to be 8 as explained earlier. The training time for the model with 8 dimensions was 12 hours for 28 epochs. The number of semantic groups was set to be 4. The training time for the model with 4 dimensions was 13 hours for 31 epochs. The number of semantic groups was set to be 2. The performance is seen to decline rapidly as the number of dimensions decrease. This could also be attributed to the fact that models with such low dimensions aren't powerful enough to capture the correct representation of such large datasets accurately. Thus, weighing the performance of the models, their validation loss and the training time, the model with dimensions 512 was seen to have the best performance with a practical training time to carry on experiments for the rest of the parameters. An overview of the training and validation loss of the models is shown in Fig. 6.2.1 The validation loss for models with dimensions 4 and 8 are quite high as compared to the rest of the models. This further proves that the models with such low dimensions aren't complex enough to accurately represent the objects in the dataset.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	2.876	HDBSCAN	296	0.550	0.101	526.278
	3.102	Kmeans	175	0.555	-0.102	193.242
	2.432	Spectral	175	1.952	-0.071	92.449
	2.517	Agglomerative	175	0.030	-0.116	92.347

Table 21: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 4. The unit of time is in seconds.

6.2.4 Evaluation of the number of semantic subgroups

Once the number of dimensions of the latent space representations was set to 512, next the optimum number of semantic subgroups was evaluated. Since the dataset consists of a large variety of objects, it is not possible to determine a fixed number of categories the objects are to be partitioned into. Thus the evaluation of the number of subcategories was essential so that the ultimate goal of achieving a set of representative members of the dataset which generalises the entire dataset quite well is achieved.

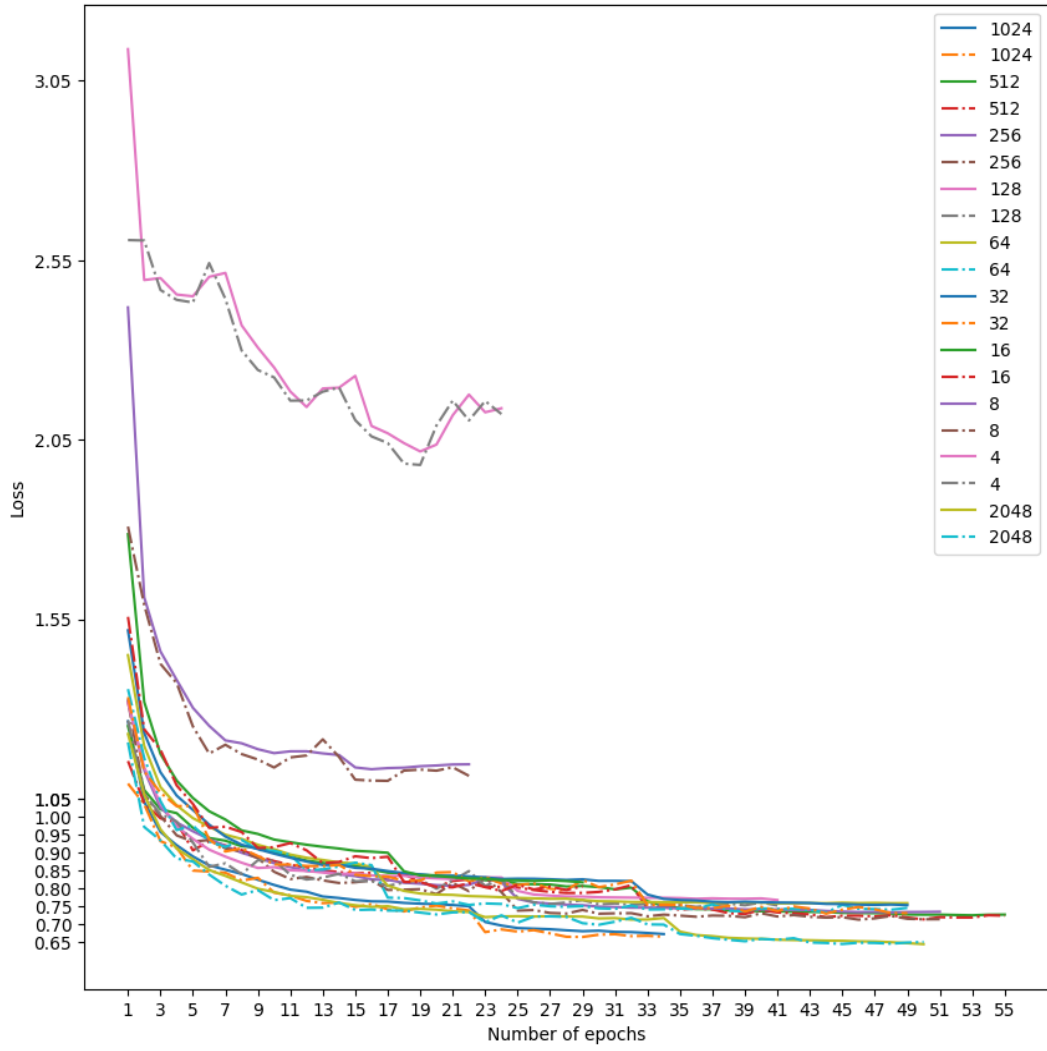


Figure 6.2.1: Overview of the validation loss of the models trained with different dimensions. The y-axis represents the loss. The x-axis represents the number of number of epochs. The solid lines represent the mean training loss after each epoch. The dash dot line represents the mean validation loss after each epoch. The numbers in the legend represent the dimension of the latent space representations of the models.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.555	HDBSCAN	279	0.620	0.185	689.203
	3.731	Kmeans	175	0.176	-0.029	101.171
	3.349	Spectral	175	7.221	0.042	100.748
	3.663	Agglomerative	175	0.031	-0.007	102.203

Table 22: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512 and the number of semantic subgroups is 32. The unit of time is in seconds.

Table 14 on Page 58 already showed the performance of the model when the dimension of the latent space representation was 512 and the number of semantic subgroups is 64. Table 23 records the performance of the model when the number of semantic subgroups is set to 32. The rest of the parameters for the experiments remains constant through this subsection.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.555	HDBSCAN	279	0.620	0.185	689.203
	3.731	Kmeans	175	0.176	-0.029	101.171
	3.349	Spectral	175	7.221	0.042	100.748
	3.663	Agglomerative	175	0.031	-0.007	102.203

Table 23: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512 and the number of semantic subgroups is 32. The unit of time is in seconds.

It is seen that the performance of the model with $k = 32$ is comparable to the model with $k = 64$. The DBCV score for the model with $k = 32$ is 0.185 while that of the model with $k = 64$ is 0.186. In other words, even with fewour semantic subgroups the model could achieve comparable results. Moreover, the model for $k = 32$ required a training time of 28 hours for 39 epochs. Whereas, the model for $k = 64$ required a training time of 64 hours for 54 epochs. Thus, reducing the value of k by a factor of 2 also led t the decrease in the training time by a factor of 2.29. This also re-ensures the ultimate goal of the task. Fewer subcategories in the objects means the model will learn more "general" features of the objects and not the extremely intricate and instance-specific object features. This would help in objects of the dataset which actually capture the overall variance in the dataset.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.797	HDBSCAN	291	0.140	0.182	244.248
	3.557	Kmeans	175	0.257	-0.50	100.699
	3.480	Spectral	175	6.331	0.030	103.005
	3.731	Agglomerative	175	0.033	-0.007	104.203

Table 24: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512 and the number of semantic subgroups is 16. The unit of time is in seconds.

Table 24 and tab. 25 shows the performance of the model when the dimension of the latent space representation was 512 and the number of semantic subgroups is 16 and 8 respectively. The dimension of the latent space representations of all these models are 512. a batch size of 128 was used to train the models as mentioned earlier.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.018	HDBSCAN	291	0.140	0.139	235.023
	3.091	Kmeans	175	0.191	-0.092	98.415
	3.101	Spectral	175	6.810	-0.027	96.898
	3.177	Agglomerative	175	0.038	-0.030	97.490

Table 25: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512 and the number of semantic subgroups is 8. The unit of time is in seconds.

The model for $k = 16$ required a training time of 37 hours for 63 epochs until convergence. Whereas, the model for $k = 8$ required a training time of 31 hours for 40 epochs. It is seen that the model with $k = 16$ gives almost comparable results to that of the model with $k = 32$. But it is observed that the model with $k = 16$ takes longer training time for convergence as compared to the model with $k = 32$. It is further noted that the performance of the model with $k = 8$ has already degraded by a factor of 4.3% as compared to that of the model with $k = 8$.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.259	HDBSCAN	301	0.115	0.186	332.309
	4.356	Kmeans	175	0.172	-0.049	205.339
	3.344	Spectral	175	11.716	0.024	166.501
	5.815	Agglomerative	175	0.049	-0.041	170.280

Table 26: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512 and the number of semantic subgroups is 128. The unit of time is in seconds.

Table 26 shows the performance of the model when the dimension of the latent space representation was 512 and the number of semantic subgroups is 128. The model required a training time of 58 hours for 36 epochs until convergence it reached. It can be seen that the performance for the model with $k = 128$ is comparable to the model with $k = 32$. Also, the training time for the model with $k = 32$ is much less compared to that of the model with $k = 128$.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.295	HDBSCAN	296	0.122	0.103	269.228
	3.577	Kmeans	175	0.308	-0.045	139.550
	3.751	Spectral	175	11.623	-0.004	167.611
	5.013	Agglomerative	175	0.032	-0.005	169.509

Table 27: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512 and the number of semantic subgroups is 4. The unit of time is in seconds.

Table 27, and tab. 28 shows the performance of the model when the dimension of the latent space representation was 512 and the number of semantic subgroups is 4, and 2 respectively. The training time for the model with $k = 4$ is 17 hours for 23 epochs. Similarly, the training time for the model with $k = 2$ is 5 hours for 6 epochs. It can be observed that the performance of the models decline significantly when the number of semantic subgroups is too low. Thus, it can be concluded that the performance of the models increases with the increase in the number of semantic subgroups and then attains a saturation in the performance. In other words, the number of semantic subgroups doesn't effect the performance of the models as long as they are enough parts the objects are partitioned into.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	4.469	HDBSCAN	289	0.126	0.085	320.217
	3.364	Kmeans	175	5.662	-0.118	155.136
	3.349	Spectral	175	5.662	-0.089	115.978
	3.029	Agglomerative	175	0.038	-0.100	98.683

Table 28: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512 and the number of semantic subgroups is 2. The unit of time is in seconds.

An overview of the training and validation loss of the models is shown in Fig. 6.2.2. It can be observed that the early stopping criteria is triggered quite early for the models with $k = 2$ and $k = 4$. Thus, the training is stopped before the model reaches convergence. This justifies the reason behind the degradation of the performance of the models when the value of k is too low. Thus, weighing the performance of clustering algorithm on the trained models, the validation loss and the training time, the model with $k = 32$ seems to be a perfect trade-off with a realistic training time and a decent performance in clustering the objects of the objects. So, the model with dimension of the latent space representations as 512 and k as 32 is chosen for the hyperparameter tuning for the rest of the experiments.

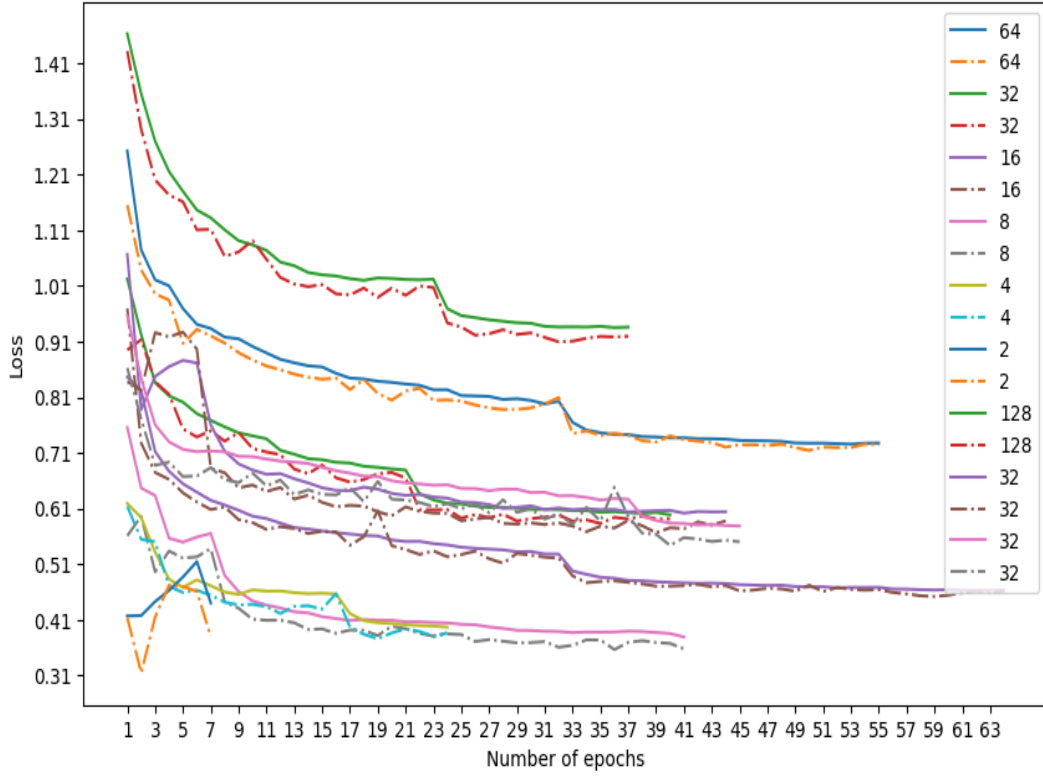


Figure 6.2.2: Overview of the validation loss of the models trained with different values of semantic subgroups when dimension of the latent space representation was fixed to 512. The y-axis represents the loss. The x-axis represents the number of number of epochs. The solid lines represent the mean training loss after each epoch. The dash dot line represents the mean validation loss after each epoch. The numbers in the legend represent number of semantic subgroups in the objects.

6.2.5 Evaluation of the batch size for training

One of the most crucial hyperparameters to adjust in deep learning methods is batch size. Attempting to optimize convex functions involves an intrinsic trade-off between the advantages of larger and smaller batch sizes. One way to ensure convergence to the global optima of the objective function is to use a batch size equal to the entire dataset. But doing so comes at the expense of a slower empirical convergence to that optimal state. Also, it is practically infeasible to load the entire dataset like the ABC dataset [39] in this use-case all at once. However, it has been demonstrated empirically that employing lower batch sizes leads to a faster convergence to "good" solutions. Smaller batch sizes enable the model to "start learning before having to see all the data," which provides an easy explanation for this. A smaller batch size has the drawback of not guaranteeing that the model will eventually come to the global optima. It will oscillate between the global optima, remaining outside a certain ϵ -ball of the optima, where ϵ is determined by the batch size to dataset ratio. Therefore, as mentioned in section 4.8.1, the largest batch size that could be fit in the GPU with the higher capacity was observed to be 256. Thus, to facilitate the training of multiple models for experimentation in the limited timeframe parallelly, the performance of the models with batch sizes 64, 128 and 256 were evaluated.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	5.855	HDBSCAN	291	0.178	0.154	382.814
	6.034	Kmeans	175	0.384	-0.040	146.189
	5.365	Spectral	175	13.660	0.026	145.207
	5.513	Agglomerative	175	0.040	-0.011	167.093

Table 29: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512, the number of semantic subgroups is 32 and the batch size for training the model is 256. The unit of time is in seconds.

The dimension of the latent space representations of the all the models in this subsection is set to 512 and $k = 32$ as discussed above. The learning rate is 0.001 and the decay factor for the learning rate is 0.1 as mentioned earlier. Table 23 on page 63 already shows the performance of the model when batch size is 128. Table 29 shows the performance of the model when the batch size is 256. The training time for this model was 40 hours for 43 epochs. Quite naturally, the model took longer to converge for a larger batch size. But performance of the model did not seem to improve as expected as compared to the model with batch size 128. The DBCV of the HDBSCAN algorithm was 0.185 for the earlier model, whereas for the model with batch size it appears to suffer a downfall in performance by 3.1%.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	5.454	HDBSCAN	300	1.080	0.177	730.026
	6.732	Kmeans	175	7.606	-0.056	602.923
	6.326	Spectral	175	51.657	0.008	492.017
	6.129	Agglomerative	175	0.489	-0.007	481.838

Table 30: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512, the number of semantic subgroups is 32 and the batch size for training the model is 64. The unit of time is in seconds.

On the other hand, table 30 shows the performance of the model when the batch size is 64. The training time for this model was 36 hours for 44 epochs. The performance of the models is seen to improve from batch size 64 to 128 as expected. Fig. 6.2.3 gives an overview of the training and validation loss of the models with changing batch size. It can be observed that the validation loss of the models are quite comparable on convergence which can justify the reason behind no significant improvement of the models with different batch sizes.

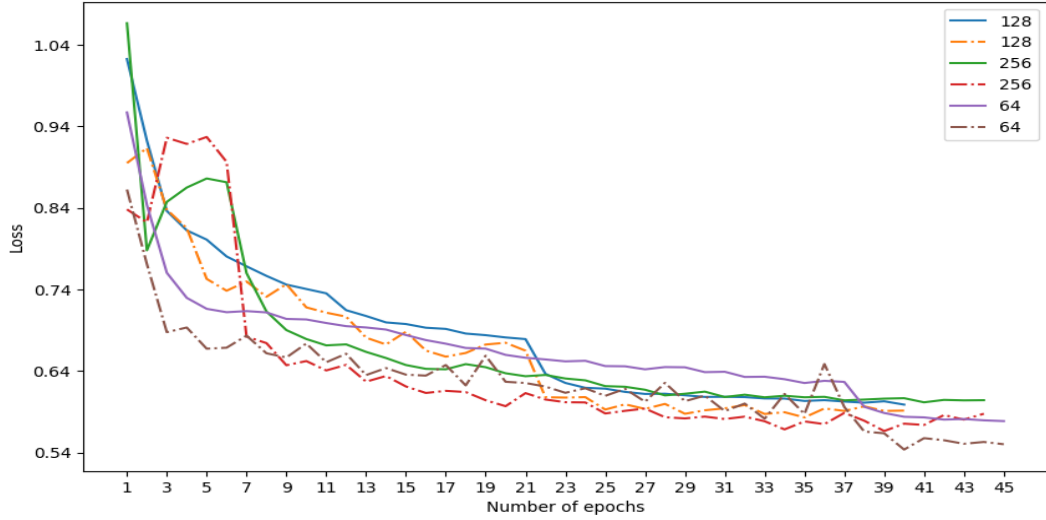


Figure 6.2.3: Overview of the validation loss of the models trained with different batch sizes when dimension of the latent space representation was fixed to 512 and the number of semantic subgroups to 32. The y-axis represents the loss. The x-axis represents the number of number of epochs. The solid lines represent the mean training loss after each epoch. The dash dot line represents the mean validation loss after each epoch. The numbers in the legend represent the batch size used for training the models.

6.2.6 Evaluation of the effect of early stopping

Determining an appropriate finite number of training epochs is a major challenge while develop machine learning models. If a model has too less epochs, then it would result to an underfit model whereas if the number of epochs is too large, then the model tends to memorize the training dataset and doesn't generalize well for unseen data. Therefore, the method of termination of the training when the model no longer improves it's performance on the validation dataset is known as the early stopping mechanism. The patience parameter plays a very vital role in this mechanism. The model does not immediately terminate the training as soon as there is an increase in the validation loss. Rather it waits for a number of consecutive epochs defined by the patience parameter. And if the validation loss no longer decreases as compared to the least recorded validation loss, the training is then terminated. A patience of 3, 5, 7 and 10 have been evaluated in this subsection. The learning rate of 0.001 and a decay factor of 0.1 as mentioned earlier was used throughout the experiments recorded below.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBC score	DBC time
2	3.643	HDBSCAN	289	0.112	0.132	260.046
	3.614	Kmedoids	175	0.101	-0.054	115.333
	3.535	Spectral	175	5.567	-0.008	114.561
	3.765	Agglomerative	175	0.299	-0.033	106.243

Table 31: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the patience for early stopping is 3, dimension of the latent space representations is 512, the number of semantic subgroups is 32, and the batch size for training the model is 128. The unit of time is in seconds.

Table 31 shows the performance of the model with patience 3. The model took a training time of 10 hours for 10 epochs until convergence. Table 23 on page 63 already shows the performance of the model when then the patience parameter was set to 5. It required a training time of 28 hours for 39 epochs. As expected the model with less patience has lower training time compared to that with a high patience. But it is that the performance of the model with less patience is worse as compared to that with higher patience. There is a 5% decline in the quality of the performance.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.731	HDBSCAN	298	0.126	0.199	293.794
	3.257	Kmedoids	175	0.134	-0.037	116
	3.340	Spectral	175	9.391	0.011	94.393
	2.788	Agglomerative	175	0.032	-0.018	93.149

Table 32: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the patience for early stopping is 7, dimension of the latent space representations is 512, the number of semantic subgroups is 32, and the batch size for training the model is 128. The unit of time is in seconds.

Table 32 and tab. 33 shows the performance of the models with patience 7 and 10 respectively. The former model takes a training time of 46 hours for 49 epochs. As expected, the model took more number of epochs until convergence as compared to the model with patience 5. Also, the results for this model outform the performance shown by the model with patience 3 and 5 as expected. The DBCV score attained by the HDBSCAN algorithm is 0.199 by this model as compared to the next best model with patience 5 which has the DBCV score of 0.185. The model with patience 10 requires a training time of 30 hours for 42 epochs. But the performance of the model is seen to drop a bit as compared to the model with patience 7.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.338	HDBSCAN	308	0.126	0.190	330.257
	3.191	Kmedoids	175	0.288	-0.051	166.315
	3.072	Spectral	175	7.785	0.013	156.930
	3.441	Agglomerative	175	0.032	-0.041	159.887

Table 33: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the patience for early stopping is 10, dimension of the latent space representations is 512, the number of semantic subgroups is 32, and the batch size for training the model is 128. The unit of time is in seconds.

Fig. 6.2.3 gives an overview of the training and validation loss of the models for different values of the patience parameter. As expected, the model with patience 10 has the least validation loss on convergence. Thus weighing the training time of the models and their performance for the clustering task, the model with patience 7 is observed to have the best results.

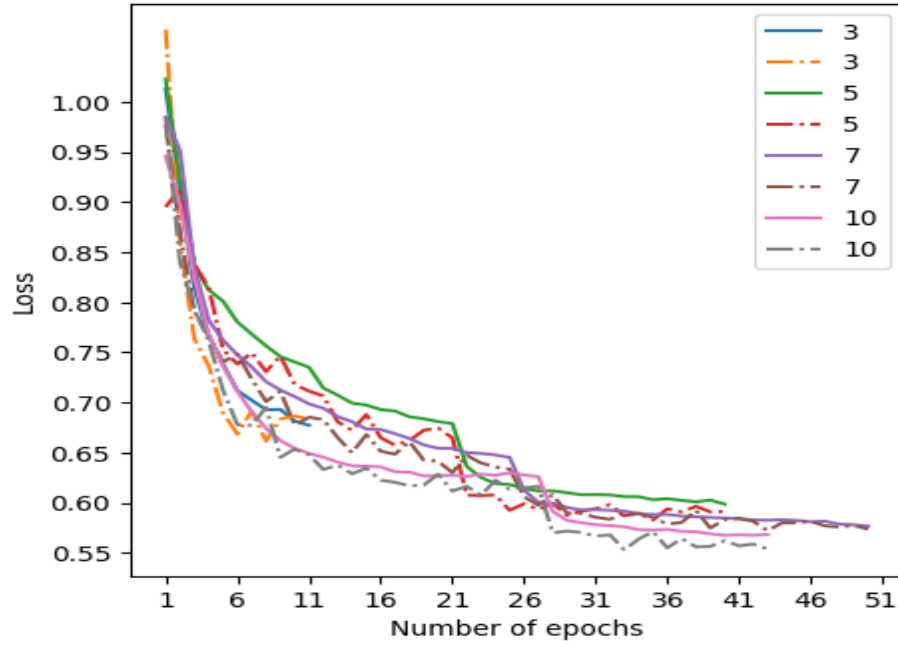


Figure 6.2.4: Overview of the validation loss of the models trained with different values of the patience parameter in early stopping when dimension of the latent space representation was fixed to 512, the number of semantix subgroups is 32 and batch size is 128. The y-axis represents the loss. The x-axis represents the number of number of epochs. The solid lines represent the mean training loss after each epoch. The dash dot line represents the mean validation loss after each epoch. The numbers in the legend represent the value of the patience parameter.

6.2.7 Evaluation of the effect of learning rate

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCv score	DBCv time
2	3.417	HDBSCAN	285	0.109	0.128	315.829
	3.455	Kmedoids	175	0.246	-0.038	147.585
	3.270	Spectral	175	6.347	-0.057	156.651
	3.844	Agglomerative	175	0.037	-0.083	162.778

Table 34: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the learning rat is 0.01. The unit of time is in seconds.

The learning rate is one of the crucial hyperparameters when building a machine learning model. When the model weights are updated, it regulates how much the model is to be altered after calculating the estimated error. Carefully selecteing the learning rate is extremely necessary because if te learning rate is too small, then the training process will be extremely low and it can get stuck at a local optima. Whereas, if the learning rate s too high, then the model can oscillate around the global optima without raching a convergence. Table 18 already shows the performance of the model when the learning rate was set ot 0.001. Table 34. The

model required a training time of 28 hours for 40 epochs. It took almost the same number of epochs to converge as did the model with learning rate 0.001. Furthermore, it can be observed that the performance of the model deteriorated when the learning rate was changed to 0.01 from 0.001. There was a 6% degradation in performance when the learning rate was increased by a factor of 10.

6.2.8 Evaluation of the effect of decay factor of the learning rate

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.351	HDBSCAN	303	0.113	0.212	334.613
	3.734	Kmedoids	175	0.273	-0.033	164.001
	3.192	Spectral	175	6.865	0.030	109.385
	3.722	Agglomerative	175	0.043	-0.001	92.632

Table 35: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the learning rate is 0.001 and the decay factor is 0.5. The unit of time is in seconds.

Keeping the learning rate and the rest of the parameters constant, experiments were performed to evaluate the effect of the decay factor of the learning rate on the performance of the model. The decay factor determines the value of the new learning rate as shown in Eq. 58 when the metrices for the performance of the model stops improving.

$$\text{new learning rate} = \text{decay factor} \times \text{current learning rate} \quad (58)$$

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.265	HDBSCAN	287	0.114	0.184	225.236
	3.298	Kmedoids	175	0.144	0.010	95.764
	4.107	Spectral	175	14.298	0.036	116.901
	3.297	Agglomerative	175	0.033	0.017	96.614

Table 36: Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the learning rate is 0.001 and the decay factor is 0.9. The unit of time is in seconds.

Table 18 already shows the performance of the model when the learning rate was set to 0.001 and the decay factor was 0.1. Table 35 shows the performance of the model when the learning rate remained constant at 0.001 but the decay factor was change to 0.5. The performance of this model is seen to improve as compared to model with decay factor 0.1. There was almost a 3% increase in the performance of the model. The decay factor was further improved and tab. 36 records the performance of the model when the decay factor is 0.9. All the remaining parameters of the model remained constant. The model took a training time of 20

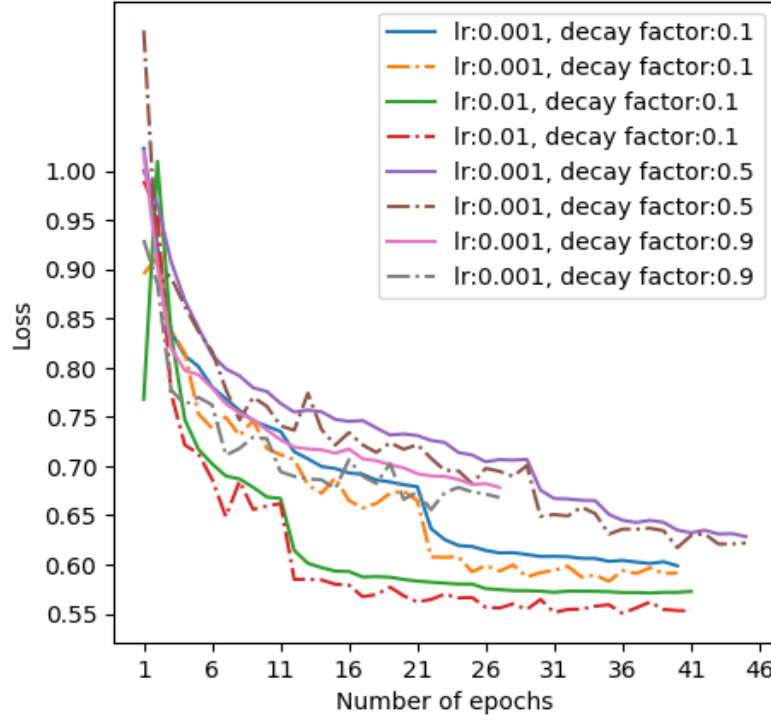


Figure 6.2.5: Overview of the validation loss of the models trained with different values of learning rate and decay factor of the learning rate when dimension of the latent space representation was fixed to 512, the number of semantix subgroups is 32 and batch size is 128. The y-axis represents the loss. The x-axis represents the number of number of epochs. The solid lines represent the mean training loss after each epoch. The dash dot line represents the mean validation loss after each epoch. The numbers in the legend represent the learning rate and decay factor of the learning rate.

hours for 26 epochs until convergence. A decrease in performance of the model was observed as compared to the model with decay rate 0.5.

Fig. 6.2.5 gives an overview of the training and validation loss of the models for learning rate and decay factor of the learning rate used during training the models. Even with a lower higher learning rate, the model takes almost the same number of epoch until convergence. But it can be noticed that the model reaches a comparable valiation loss quite early on as compared to the model with a lower learning rate. The model with learning rate 0.001 as decay factor 0.9 takes the least training time.

6.3 Conclu approach with DGCNN as Backbone

As mentioned in Section 4, two feature representation learning backbones were used for the experiments. Experiments with the DGCNN as the backbone is recorded here. The performance of all the clustering algorithm has been evaluated.

6.3.1 Evaluation of dimension of feature space representations

As discussed earlier, the dimension of the latent space representation is an extremely crucial parameter while training the network. Since the original dataset had the point clouds with 2048, for the sake of completeness of the experimentation, the evaluation has been performed for all dimensions on a logarithmic scale of 2 from 1024 to down until 4. Table 37 shows the experiments when the dimension of the latent space representations was set to be 1024. For all the models in this subsection, the number of nearest neighbors required for a DGCNN graph was set to 20 as per [50]. It was observed that training the models with the DGCNN backbone took extremely large training time. The model with the dimension of latent

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.022 secs	HDBSCAN	295	0.121	0.112	228.925
	3.519	Kmedoids	175	0.151	-0.067	92.043
	3.212	Spectral	175	6.712	-0.038	92.080
	3.683	Agglomerative	175	0.037	-0.089	124.850

Table 37: Results of the Conclu approach with DGCNN backbone on 1000 datapoints when the dimension of the latent space representation is 1024. The unit of time is in seconds.

space representation of 1024 took 168 hours for 37 epochs until convergence. This is more than twice the time required for the model with PointNet autoencoder as backbone, which required 73 hours for 65 epochs. This is because of the dynamic graph creation step involved for this method in each epoch before every convolutional layer. Also the performance of this model is a little worse as compared to the model with PointNet autoencoder as the backbone with same dimension for the latent space representation, the performance of which is recorded in tab. 7 on Page 54.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.214	HDBSCAN	299	0.125	0.129	327.926
	3.739	Kmedoids	175	0.248	-0.060	162.210
	3.212	Spectral	175	6.712	-0.038	92.080
	3.683	Agglomerative	175	0.037	-0.089	124.850

Table 38: Results of the Conclu approach with DGCNN backbone on 1000 datapoints when the dimension of the latent space representation is 512. The unit of time is in seconds.

Table 38 records the performance of the model when the dimension of the latent space representation is 512. All other parameters of the model remains constant. This model takes a training time of 231 hours for 60 epochs. This is four times the time required by the model when PointNet autoencoder as the backbone. Even with such high training time, the performance of the model is seen to deteriorate by atleast 3% in case of HDBSCAN algorithm and atmost 5% in case of agglomerative clustering algorithm. Table. 39 records the performance of the model when the dimension was further reduced to 256. On reducing the dimension by a factor of 2, the model training time required by the model also decreased by a factor of 2. Now the model requires a training time of 120 hours for 31 epochs. But unlike the model with the PointNet autoencoder as backbone, this model had a degradation in the performance when the dimension was reduced to 256. There was a 3% decrease in performance for the HDBSCAN algorithm, and roughly 5% degradation for the KMedoids algorithm and spectral clustering algorithm. However, the performance of the agglomerative clustering algorithm remained almost similar.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBC score	DBC time
2	3.374	HDBSCAN	278	0.112	0.098	240.169
	3.673	Kmedoids	175	0.171	-0.151	103.629
	3.579	Spectral	175	6.933	-0.078	105.700
	3.722	Agglomerative	175	0.044	-0.001	92.631

Table 39: Results of the Conclu approach with DGCNN backbone on 1000 datapoints when the dimension of the latent space representation is 256. The unit of time is in seconds.

Table 40 records the performance of the model when the dimension of the latent space representation was further reduced to 128. The learning rate for all the models in this subsection is set to be 0.001 with a decay factor of 0.1. It was also noticed that the models in with the DGCNN backbone has a higher Graphics Processing Unit (GPU) consumption. So, a batch size of a maximum of 32 could be used according to the hardware settings available at IPA as mentioned in sec. 4.8.1. The model required a training time of 91 hours for 33 epochs until convergence. The performance of the HDBSCAN algorithm remained almost constant whereas the performance of the Kmedoids algorithm improved by 2% as compared to the model with dimension 256. The performance of the spectral clustering algorithm degraded by 2% whereas the performance of the agglomerative clustering. Furthermore, the performance of all the clustering algorithm deteriorated for this model as compared to its other counterpart with PointNet autoencoder backbone.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBC score	DBC time
2	3.498	HDBSCAN	289	0.118	0.103	250.010
	3.839	Kmedoids	175	0.131	-0.116	106.311
	3.559	Spectral	175	6.662	-0.108	103.972
	3.590	Agglomerative	175	0.045	-0.099	104.450

Table 40: Results of the Conclu approach with DGCNN backbone on 1000 datapoints when the dimension of the latent space representation is 128. The unit of time is in seconds.

For the purpose of completeness of the evaluation, the dimension of the model was further reduced to 64, 32, 16, 8 and 4. Table 41 and tab. 42 are the models with dimension 64 and 32 respectively. As mentioned earlier, early stopping criteria was used during the training of the models to prevent overfitting. A patience of 5 was used for all the models in this subsection. Similar to the setup of the Conclu approach with PointNet autoencoder as backbone. A very high number of epochs was set for the training so that the training always terminates only when the early stopping criteria is triggered. Thus, the number of epochs was set to 200. The model with dimension 64 took a training time of

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.430	HDBSCAN	289	0.113	0.083	253.259
	3.572	Kmedoids	175	0.137	-0.074	107.411
	3.384	Spectral	175	6.879	-0.090	105.169
	3.298	Agglomerative	175	0.030	-0.104	104.248

Table 41: Results of the Conclu approach with DGCNN backbone on 1000 datapoints when the dimension of the latent space representation is 64. The unit of time is in seconds.

t-SNE dim	t-SNE time	Clustering Algo.	No. of Clusters	Clustering time	DBCV score	DBCV time
2	3.390	HDBSCAN	284	0.179	0.088	308.699
	4.151	Kmedoids	175	0.304	-0.098	178.198
	4.199	Spectral	175	34.680	-0.065	185.643
	3.067	Agglomerative	175	0.032	-0.110	99.183

Table 42: Results of the Conclu approach with DGCNN backbone on 1000 datapoints when the dimension of the latent space representation is 32. The unit of time is in seconds.

7 Implementation in a Bin Picking Cell

8 Conclusions

8.1 Discussions

8.2 Limitations

8.3 Future Scope

9 Summary

List of Acronyms

AI	Artificial Intelligence
CAD	Computer-Aided Design
CNN	Convolutional Neural Network
DBCV	Density Based Clustering Validation
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
DGCNN	Dynamic Graph Convolutional Neural Network
DSC	Density Sparseness of a Cluster
DSPC	Density Separation of a Pair of Clusters
EM	Expectation Maximization
FCN	Fully Connected Network
GPU	Graphics Processing Unit
HDBSCAN	Hierarchical Density-Based Spatial Clustering of Applications with Noise
IPA	Fraunhofer Institute for Manufacturing Engineering and Automation
KD	K-Dimensional
KL	Kullback–Leibler divergence
k-NN	k-Nearest Neighbor
LDA	Linear Discriminant Analysis
LiDAR	Light Detection and Ranging
LIN	linear
ML	Machine Learning
MLP	Multi Layer Perceptron
MRD	Mutual Reachability Distance
MST	Minimum Spanning Trees
NDA	Normal Discriminant Analysis
DFA	Discriminant Function Analysis
PAM	Partitioning Around Medoids
PTP	point-to-point
PCA	Principal Component Analysis
ReLU	Rectified Linear Unit
RGB	Red Green Blue
RL	Reinforcement Learning
RNN	Recurrent Neural Network

stopgrad stop-gradient

SVM Support Vector Machines

TCP Tool Center Point

t-SNE t-distributed Stochastic Neighbor Embedding

VAE Variational Autoencoders

List of Figures

1.2.1 Some random objects from the ABC dataset.[39]	2
1.3.1 Overview of the entire pipeline for development of source library.	3
3.1.1 Vacuum Grippers[63]	7
3.1.2 Mechanical Grippers[23]	7
3.1.3 Magnetic Grippers[71]	7
3.1.4 Adaptive Grippers[53]	7
3.1.5 Soft Grippers[55]	8
3.1.6 Hybrid Grippers with soft and rigid structures[60]	8
3.1.7 Overview of model-based bin picking approach.[75]	9
3.1.8 Important positions the workspace in a bin-picking application. W is the world coordinate system with it's origin at the centre of the base of the robot. O is the object coordinate system with it's origin at the center of the object. G is the gripper coordinate system with its origin at the center of the gripper. $\mathbf{W}_{P_O}$ is the object's position in relation to the world coordinate system. $\mathbf{W}_{P_G}$ is the end-effector's position in relation to the world coordinate system. $\mathbf{G}_{P_O}$ is the position of the object in relation to the position of the gripper.[8]	11
3.2.1 Architecture of an undercomplete autoencoder.[51]	13
3.2.2 Architecture of a single layer sparse encoder. The hidden nodes in bright yellow are active, while the ones in light yellow are set to zero, hence inactive[3]	13
3.2.3 Difference between a vanilla autoencoder and a VAE.[79]	13
3.2.4 A generic siamese network.[1]	14
3.2.5 Before (left) and after (right) minimizing triplet loss function[78]	14
3.2.6 Minibatches in offline triplet mining. The green boxes are positive datapoints, the blue boxes are anchor datapoints and the red boxes are negative datapoints.	15
3.2.7 Triplets constructed only within the minibatch. There are P object classes in the minibatch. The green boxes are positive datapoints, the blue box is the anchor datapoint and the red boxes are negative datapoints.	15
3.3.1 DBSCAN can find non-linearly separable non-convex clusters.[16]	16
3.3.2 Difference between K-Means and K-Medoids.[20]	17
3.3.3 A dendrogram in hierarchical clustering.[29]	17
3.3.4 A comparison between the results of K-means clustering and Spectral clustering. The image in the left shows the points that need to be clustered. [57]	19
3.5.1 Model trained for a task in the similar domain is re-used for a new task.[30]	25
3.5.2 A model trained on a different task is adapted to be used for a new task.[89]	25
3.5.3 Overview of the different types of transfer learning.[58]	27
3.8.1 The effect of the number of model parameters on the model performance.[88]	28
3.8.2 Projection of the data before and after applying LDA[88]	29
3.8.3 Projection of the data from higher dimensional space to lower dimensional space on applying PCA[64]	30
3.8.4 The percentage of explained variance for each principal component	31
4.1.1 The Conclu model pipeline including the two modules: point -level clustering and instance-level contrasting.[50]	32
4.2.1 The PointNet Architecture.[65]	33
4.2.2 Critical points set and the original shape.[65]	34
4.2.3 The DGCNN Architecture.[84]	36
4.2.4 The Computation of edge feature e_{ij} from a pair of points p_i and p_j . Here, h_θ is a fully connected layer and the weights associated with it are the trainable parameters θ . [84]	37
4.2.5 The EdgeConv operation. The output of EdgeConv is computed by aggregating the features associated with all the edges that originate from each connected vertex.[84]	37
4.3.1 The architecture of the point-level clustering based unsupervised feature learning.[50]	39
4.3.2 Architecture of class probability prediction.[50]	40
4.3.3 Architecture of label reassigning [50]	40
4.4.1 The architecture of the instance-level Contrasting.[50]	41

4.6.1 The original data in 2-D space	43
4.6.2 Data projected to 1-D space	43
4.6.3 A random dataset created in 1D space.	44
4.6.4 Visualization of the dataset in 1D space after applying t-SNE.	44
4.7.1 A demonstration of the KD-tree algorithm with 7 datapoints having 2 dimensions x and y.[70]	46
4.7.2 A demonstration of the Ball-tree algorithm with 9 datapoints having 2 dimensions x and y.[70]	47
6.2.1 Overview of the validation loss of the models trained with different dimensions. The y-axis represents the loss. The x-axis represents the number of number of epochs. The solid lines represent the mean training loss after each epoch. The dash dot line represents the mean validation loss after each epoch. The numbers in the legend represent the dimension of the latent space representations of the models.	62
6.2.2 Overview of the validation loss of the models trained with different values of semantic subgroups when dimension of the latent space representation was fixed to 512. The y-axis represents the loss. The x-axis represents the number of number of epochs. The solid lines represent the mean training loss after each epoch. The dash dot line represents the mean validation loss after each epoch. The numbers in the legend represent number of semantic subgroups in the objects.	66
6.2.3 Overview of the validation loss of the models trained with different batch sizes when dimension of the latent space representation was fixed to 512 and the number of semantic subgroups to 32. The y-axis represents the loss. The x-axis represents the number of number of epochs. The solid lines represent the mean training loss after each epoch. The dash dot line represents the mean validation loss after each epoch. The numbers in the legend represent the batch size used for training the models.	68
6.2.4 Overview of the validation loss of the models trained with different values of the patience parameter in early stopping when dimension of the latent space representation was fixed to 512, the number of semantix subgroups is 32 and batch size is 128. The y-axis represents the loss. The x-axis represents the number of number of epochs. The solid lines represent the mean training loss after each epoch. The dash dot line represents the mean validation loss after each epoch. The numbers in the legend represent the value of the patience parameter.	70
6.2.5 Overview of the validation loss of the models trained with different values of learning rate and decay factor of the learning rate when dimension of the latent space representation was fixed to 512, the number of semantix subgroups is 32 and batch size is 128. The y-axis represents the loss. The x-axis represents the number of number of epochs. The solid lines represent the mean training loss after each epoch. The dash dot line represents the mean validation loss after each epoch. The numbers in the legend represent the learning rate and decay factor of the learning rate.	72

List of Tables

1	Overview of the approximate time taken for the creation of the source library	3
2	Different settings of transfer learning [58]	26
3	Results of the baseline algorithm on 1000 datapoints and without any dimensionality reduction technique. The unit of time is in seconds.	52
4	Results of the baseline algorithm on 1000 datapoints after applying t-SNE dimensionality reduction. The unit of time is in seconds.	53
5	Results of the baseline algorithm on 2000 datapoints and without any dimensionality reduction technique. The unit of time is in seconds.	53
6	Results of the baseline algorithm on 2000 datapoints after applying t-SNE dimensionality reduction. The unit of time is in seconds.	54
7	Results of the ConClu approach on 1000 datapoints. The unit of time is in seconds.	54
8	Results of the ConClu approach on 2000 datapoints. The unit of time is in seconds.	55
9	Results of the ConClu approach on 5000 datapoints. The unit of time is in seconds.	55
10	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints with PCA as dimensionality reduction technique in both steps. The unit of time is in seconds.	56
11	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints with PCA as dimensionality reduction technique in the first step and t-SNE as the second. The unit of time is in seconds.	56
12	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints with only t-SNE as dimensionality reduction technique as the only step. The unit of time is in seconds.	57
13	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 2048. The unit of time is in seconds.	57
14	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512. The unit of time is in seconds.	58
15	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 256. The unit of time is in seconds.	58
16	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 128. The unit of time is in seconds.	59
17	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 64. The unit of time is in seconds.	59
18	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 32. The unit of time is in seconds.	60
19	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 16. The unit of time is in seconds.	60
20	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 8. The unit of time is in seconds.	61
21	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 4. The unit of time is in seconds.	61
22	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512 and the number of semantic subgroups is 32. The unit of time is in seconds.	62
23	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512 and the number of semantic subgroups is 32. The unit of time is in seconds.	63
24	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512 and the number of semantic subgroups is 16. The unit of time is in seconds.	63
25	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512 and the number of semantic subgroups is 8. The unit of time is in seconds.	64

26	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512 and the number of semantic subgroups is 128. The unit of time is in seconds.	64
27	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512 and the number of semantic subgroups is 4. The unit of time is in seconds.	65
28	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512 and the number of semantic subgroups is 2. The unit of time is in seconds.	65
29	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512, the number of semantic subgroups is 32 and the batch size for training the model is 256. The unit of time is in seconds.	67
30	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the dimension of the latent space representations is 512, the number of semantic subgroups is 32 and the batch size for training the model is 64. The unit of time is in seconds.	67
31	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the patience for early stopping is 3, dimension of the latent space representations is 512, the number of semantic subgroups is 32, and the batch size for training the model is 128. The unit of time is in seconds.	68
32	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the patience for early stopping is 7, dimension of the latent space representations is 512, the number of semantic subgroups is 32, and the batch size for training the model is 128. The unit of time is in seconds.	69
33	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the patience for early stopping is 10, dimension of the latent space representations is 512, the number of semantic subgroups is 32, and the batch size for training the model is 128. The unit of time is in seconds.	69
34	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the learning rat is 0.01. The unit of time is in seconds.	70
35	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the learning rat is 0.001 and the decay factor is 0.5. The unit of time is in seconds.	71
36	Results of the Conclu approach with PointNet autonocoder on 1000 datapoints when the learning rat is 0.001 and the decay factor is 0.9. The unit of time is in seconds.	71
37	Results of the Conclu approach with DGCNN backbone on 1000 datapoints when the dimension of the latent space representation is 1024. The unit of time is in seconds.	73
38	Results of the Conclu approach with DGCNN backbone on 1000 datapoints when the dimension of the latent space representation is 512. The unit of time is in seconds.	73
39	Results of the Conclu approach with DGCNN backbone on 1000 datapoints when the dimension of the latent space representation is 256. The unit of time is in seconds.	74
40	Results of the Conclu approach with DGCNN backbone on 1000 datapoints when the dimension of the latent space representation is 128. The unit of time is in seconds.	74
41	Results of the Conclu approach with DGCNN backbone on 1000 datapoints when the dimension of the latent space representation is 64. The unit of time is in seconds.	75
42	Results of the Conclu approach with DGCNN backbone on 1000 datapoints when the dimension of the latent space representation is 32. The unit of time is in seconds.	75

List of Symbols

Bibliography

- [1] *A Comprehensive Guide to Siamese Neural Networks*. <https://medium.com/@rinkinag24/a-comprehensive-guide-to-siamese-neural-networks-3358658c0513>.
- [2] Matan Atzmon, Haggai Maron, and Yaron Lipman. “Point convolutional neural networks by extension operators”. In: *arXiv preprint arXiv:1803.10091* (2018).
- [3] *Autoencoder*. <https://en.wikipedia.org/wiki/Autoencoder>.
- [4] Vassileios Balntas et al. “Learning local feature descriptors with triplets and shallow convolutional neural networks.” In: *Bmvc*. Vol. 1. 2. 2016, p. 3.
- [5] R Bellman. “Adaptive control processes: a guided tour. Princeton University Press, 1961”. In: *PUBLICATIONS 46 Signatureofthecandidate withReg* ().
- [6] Konstantinos Bousmalis et al. “Using simulation and domain adaptation to improve efficiency of deep robotic grasping”. In: *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE. 2018, pp. 4243–4250.
- [7] Jane Bromley et al. “Signature verification using a siamese time delay neural network”. In: *Advances in neural information processing systems* 6 (1993).
- [8] Dirk Buchholz. *Bin-picking: new approaches for a classical problem*. Vol. 44. Springer, 2015.
- [9] *Calinski-Harabasz Index*. https://en.wikipedia.org/wiki/Calinski-Harabasz_index.
- [10] Ricardo JGB Campello, Davoud Moulavi, and Joerg Sander. “A simpler and more accurate AUTO-HDS framework for clustering and visualization of biological data”. In: *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 9.6 (2012), pp. 1850–1852.
- [11] Xinlei Chen and Kaiming He. “Exploring simple siamese representation learning”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2021, pp. 15750–15758.
- [12] *Convolutional Neural Networks, Explained*. <https://towardsdatascience.com/convolutional-neural-networks-explained-9cc5188c4939>.
- [13] Marco Cuturi. “Sinkhorn distances: Lightspeed computation of optimal transport”. In: *Advances in neural information processing systems* 26 (2013).
- [14] Wenyuan Dai et al. “Self-taught clustering”. In: *Proceedings of the 25th international conference on Machine learning*. 2008, pp. 200–207.
- [15] Hal Daume III and Daniel Marcu. “Domain adaptation for statistical classifiers”. In: *Journal of artificial Intelligence research* 26 (2006), pp. 101–126.
- [16] *DBSCAN Wikipedia*. <https://en.wikipedia.org/wiki/DBSCAN>.
- [17] Lih-Yuan Deng, Max Garzon, and Nirman Kumar. “What is dimensionality reduction (dr)?” In: *Dimensionality Reduction in Data Science*. Springer, 2022, pp. 67–77.
- [18] Dandan Ding et al. “Point cloud upsampling via perturbation learning”. In: *IEEE Transactions on Circuits and Systems for Video Technology* 31.12 (2021), pp. 4661–4672.
- [19] Lixin Duan, Dong Xu, and Ivor Tsang. “Learning with augmented features for heterogeneous domain adaptation”. In: *arXiv preprint arXiv:1206.4660* (2012).
- [20] Alireza Entezami, Hassan Sarmadi, and Behzad Saeedi Razavi. “An innovative hybrid strategy for structural health monitoring by modal flexibility and clustering methods”. In: *Journal of Civil Structural Health Monitoring* 10.5 (2020), pp. 845–859.
- [21] Martin Ester et al. “A density-based algorithm for discovering clusters in large spatial databases with noise”. In: *kdd*. Vol. 96. 34. 1996, pp. 226–231.
- [22] Jerome H Friedman, Jon Louis Bentley, and Raphael A Finkel. *An algorithm for finding best matches in logarithmic time*. Department of Computer Science, Stanford University, 1975.
- [23] *Fundamentals of Pneumatic Grippers for Industrial Applications*. <https://www.digikey.co.uk/en/articles/fundamentals-of-pneumatic-grippers-for-industrial-applications>.

- [24] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [25] Jean-Bastien Grill et al. “Bootstrap your own latent-a new approach to self-supervised learning”. In: *Advances in neural information processing systems* 33 (2020), pp. 21271–21284.
- [26] John A Hartigan. *Clustering algorithms*. John Wiley & Sons, Inc., 1975.
- [27] Kaiming He et al. “Deep residual learning for image recognition”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.
- [28] Alexander Hermans, Lucas Beyer, and Bastian Leibe. “In defense of the triplet loss for person re-identification”. In: *arXiv preprint arXiv:1703.07737* (2017).
- [29] *Hierarchical Clustering*. <https://harshsharma1091996.medium.com/hierarchical-clustering-996745fe656b>.
- [30] Asmaul Hosna et al. “Transfer learning: a friendly introduction”. In: *Journal of Big Data* 9.1 (2022), p. 102.
- [31] *Intuitively Understanding Variational Autoencoders*. <https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf>.
- [32] Anil K Jain and Richard C Dubes. *Algorithms for clustering data*. Prentice-Hall, Inc., 1988.
- [33] Eric Jang et al. “Grasp2vec: Learning object representations from self-supervised grasping”. In: *arXiv preprint arXiv:1811.06964* (2018).
- [34] Xin Jin and Jiawei Han. “K-Means Clustering”. In: *Encyclopedia of Machine Learning*. Ed. by Claude Sammut and Geoffrey I. Webb. Boston, MA: Springer US, 2010, pp. 563–564. ISBN: 978-0-387-30164-8. DOI: 10.1007/978-0-387-30164-8_425. URL: https://doi.org/10.1007/978-0-387-30164-8_425.
- [35] Xin Jin and Jiawei Han. “K-Medoids Clustering”. In: *Encyclopedia of Machine Learning*. Ed. by Claude Sammut and Geoffrey I. Webb. Boston, MA: Springer US, 2010, pp. 564–565. ISBN: 978-0-387-30164-8. DOI: 10.1007/978-0-387-30164-8_426. URL: https://doi.org/10.1007/978-0-387-30164-8_426.
- [36] Diederik P Kingma, Max Welling, et al. “An introduction to variational autoencoders”. In: *Foundations and Trends® in Machine Learning* 12.4 (2019), pp. 307–392.
- [37] Kilian Kleeberger et al. “Precise Object Placement with Pose Distance Estimations for Different Objects and Grippers”. In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2021, pp. 4639–4646.
- [38] Gregory Koch, Richard Zemel, Ruslan Salakhutdinov, et al. “Siamese neural networks for one-shot image recognition”. In: *ICML deep learning workshop*. Vol. 2. 1. Lille. 2015.
- [39] Sebastian Koch et al. “ABC: A Big CAD Model Dataset For Geometric Deep Learning”. In: *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2019.
- [40] Brian Kulis, Kate Saenko, and Trevor Darrell. “What you saw is not what you get: Domain adaptation using asymmetric kernel transforms”. In: *CVPR 2011*. IEEE. 2011, pp. 1785–1792.
- [41] Solomon Kullback and Richard A Leibler. “On information and sufficiency”. In: *The annals of mathematical statistics* 22.1 (1951), pp. 79–86.
- [42] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
- [43] Levi Lelis and Jörg Sander. “Semi-supervised density-based clustering”. In: *2009 Ninth IEEE International Conference on Data Mining*. IEEE. 2009, pp. 842–847.
- [44] Sergey Levine et al. “Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection”. In: *The International journal of robotics research* 37.4-5 (2018), pp. 421–436.
- [45] Yangyan Li et al. “Pointcnn: Convolution on x-transformed points”. In: *Advances in neural information processing systems* 31 (2018).

- [46] Huei-Yung Lin, Shih-Cheng Liang, and Yu-Kai Chen. “Robotic grasping with multi-view image acquisition and model-based pose estimation”. In: *IEEE Sensors Journal* 21.10 (2020), pp. 11870–11878.
- [47] Tong Liu et al. “Exploring transfer learning to reduce training overhead of hpc data in machine learning”. In: *2019 IEEE International Conference on Networking, Architecture and Storage (NAS)*. IEEE. 2019, pp. 1–7.
- [48] Yongcheng Liu et al. “Relation-shape convolutional neural network for point cloud analysis”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 8895–8904.
- [49] Claudia Malzer and Marcus Baum. “A hybrid approach to hierarchical density-based cluster selection”. In: *2020 IEEE international conference on multisensor fusion and integration for intelligent systems (MFI)*. IEEE. 2020, pp. 223–228.
- [50] Guofeng Mei et al. “Unsupervised learning on 3d point clouds by clustering and contrasting”. In: *arXiv preprint arXiv:2202.02543* (2022).
- [51] Sumit Misra et al. “An autoencoder based model for detecting fraudulent credit card transaction”. In: *Procedia Computer Science* 167 (2020), pp. 254–262.
- [52] Davoud Moulavi et al. “Density-based clustering validation”. In: *Proceedings of the 2014 SIAM international conference on data mining*. SIAM. 2014, pp. 839–847.
- [53] *New Electric Gripper Plus Kit for Universal Robots*. <https://blog.robotiq.com/bid/73065/New-Electric-Gripper-Plus-Kit-for-Universal-Robots>.
- [54] Andrew Ng et al. “Sparse autoencoder”. In: *CS294A Lecture notes* 72.2011 (2011), pp. 1–19.
- [55] *NUS engineers bring a soft touch to commercial robotics*. <https://news.nus.edu.sg/nus-engineers-bring-a-soft-touch-to-commercial-robotics/>.
- [56] Stephen M Omohundro. “Five balltree construction algorithms”. In: (1989).
- [57] William R Palmer and Tian Zheng. “Spectral clustering for directed networks”. In: *Complex Networks & Their Applications IX: Volume 1, Proceedings of the Ninth International Conference on Complex Networks and Their Applications COMPLEX NETWORKS 2020*. Springer. 2021, pp. 87–99.
- [58] Sinno Jialin Pan and Qiang Yang. “A survey on transfer learning”. In: *IEEE Transactions on knowledge and data engineering* 22.10 (2009), pp. 1345–1359.
- [59] Wei-ke Pan et al. “Transfer learning to predict missing ratings via heterogeneous user feedbacks”. In: *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence, Barcelona, Catalonia, Spain*. 2011, p. 2318.
- [60] Wookeun Park, Seongmin Seo, and Joonbum Bae. “A hybrid gripper with soft material and rigid structures”. In: *IEEE Robotics and Automation Letters* 4.1 (2018), pp. 65–72.
- [61] F. Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [62] Gabriel Peyré, Marco Cuturi, et al. “Computational optimal transport: With applications to data science”. In: *Foundations and Trends® in Machine Learning* 11.5-6 (2019), pp. 355–607.
- [63] *Piab broadens scope of Kenos KVG vacuum grippers*. <https://www.therobotreport.com/piab-kenos-kvg-vacuum-grippers/>.
- [64] *Principal Component Analysis (PCA) Transformation*. <https://www.biorender.com/template/principal-component-analysis-pca-transformation>.
- [65] Charles R Qi et al. “Pointnet: Deep learning on point sets for 3d classification and segmentation”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 652–660.
- [66] Charles Ruizhongtai Qi et al. “Pointnet++: Deep hierarchical feature learning on point sets in a metric space”. In: *Advances in neural information processing systems* 30 (2017).

- [67] Rajat Raina et al. “Self-taught learning: transfer learning from unlabeled data”. In: *Proceedings of the 24th international conference on Machine learning*. 2007, pp. 759–766.
- [68] Salah Rifai et al. “Contractive auto-encoders: Explicit invariance during feature extraction”. In: *Proceedings of the 28th international conference on machine learning*. 2011, pp. 833–840.
- [69] *scikit-learn-extra API KMedoids*. https://scikit-learn-extra.readthedocs.io/en/stable/generated/sklearn_extra.cluster.KMedoids.html.
- [70] *Search and Insertion in K Dimensional tree*. <https://www.geeksforgeeks.org/search-and-insertion-in-k-dimensional-tree/>.
- [71] *SGM-SV MAGNETIC GRIPPER*. <https://www.universal-robots.com/fi/plus/products/j-schmalz/sgm-sv-magnetic-gripper/>.
- [72] Hidetoshi Shimodaira. “Improving predictive inference under covariate shift by weighting the log-likelihood function”. In: *Journal of statistical planning and inference* 90.2 (2000), pp. 227–244.
- [73] Karen Simonyan and Andrew Zisserman. “Very deep convolutional networks for large-scale image recognition”. In: *arXiv preprint arXiv:1409.1556* (2014).
- [74] Fei Song et al. “Layer-wise geometry aggregation framework for lossless lidar point cloud compression”. In: *IEEE Transactions on Circuits and Systems for Video Technology* 31.12 (2021), pp. 4603–4616.
- [75] Felix Spenrath. “Heuristic search method for efficient planning for gripping disorderly stored workpieces with industrial robots”. In: (2022).
- [76] Tiecheng Sun et al. “Quadratic terms based point-to-surface 3D representation for deep learning of point cloud”. In: *IEEE Transactions on Circuits and Systems for Video Technology* 32.5 (2021), pp. 2705–2718.
- [77] Christian Szegedy et al. “Going deeper with convolutions”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2015, pp. 1–9.
- [78] *Triplet Loss and Siamese Neural Networks*. <https://medium.com/@enoshshr/triplet-loss-and-siamese-neural-networks-5d363fdeba9b>.
- [79] *Understanding Variational Autoencoders(VAEs)*. <https://towardsdatascience.com/understanding-variational-autoencoders-vaes-f70510919f73>.
- [80] Alberto A Vergani and Elisabetta Binaghi. “A soft davies-bouldin separation measure”. In: *2018 IEEE international conference on fuzzy systems (FUZZ-IEEE)*. IEEE. 2018, pp. 1–8.
- [81] Pascal Vincent et al. “Extracting and composing robust features with denoising autoencoders”. In: *Proceedings of the 25th international conference on Machine learning*. 2008, pp. 1096–1103.
- [82] Oriol Vinyals, Samy Bengio, and Manjunath Kudlur. “Order matters: Sequence to sequence for sets”. In: *arXiv preprint arXiv:1511.06391* (2015).
- [83] Ulrike Von Luxburg. “A tutorial on spectral clustering”. In: *Statistics and computing* 17 (2007), pp. 395–416.
- [84] Yue Wang et al. “Dynamic graph cnn for learning on point clouds”. In: *ACM Transactions on Graphics (tog)* 38.5 (2019), pp. 1–12.
- [85] Zheng Wang, Yangqiu Song, and Changshui Zhang. “Transferred dimensionality reduction”. In: *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2008, Antwerp, Belgium, September 15-19, 2008, Proceedings, Part II* 19. Springer. 2008, pp. 550–565.
- [86] Kilian Q Weinberger and Lawrence K Saul. “Distance metric learning for large margin nearest neighbor classification.” In: *Journal of machine learning research* 10.2 (2009).
- [87] Karl Weiss, Taghi M Khoshgoftaar, and DingDing Wang. “A survey of transfer learning”. In: *Journal of Big data* 3.1 (2016), pp. 1–40.
- [88] *What is dimensionality reduction?* <https://www.ibm.com/topics/dimensionality-reduction>.

- [89] *What Is Transfer Learning? Exploring the Popular Deep Learning Approach*. <https://builtin.com/data-science/transfer-learning>.
- [90] Wenxuan Wu, Zhongang Qi, and Li Fuxin. “Pointconv: Deep convolutional networks on 3d point clouds”. In: *Proceedings of the IEEE/CVF Conference on computer vision and pattern recognition*. 2019, pp. 9621–9630.
- [91] Mutian Xu et al. “Paconv: Position adaptive convolution with dynamic kernel assembling on point clouds”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2021, pp. 3173–3182.
- [92] Bianca Zadrozny. “Learning and evaluating classifiers under sample selection bias”. In: *Proceedings of the twenty-first international conference on Machine learning*. 2004, p. 114.
- [93] Manzil Zaheer et al. “Deep sets”. In: *Advances in neural information processing systems* 30 (2017).
- [94] Lichen Zhao et al. “Transformer3D-Det: Improving 3D object detection by vote refinement”. In: *IEEE Transactions on Circuits and Systems for Video Technology* 31.12 (2021), pp. 4735–4746.
- [95] Haoran Zhou et al. “Adaptive graph convolution for point cloud analysis”. In: *Proceedings of the IEEE/CVF international conference on computer vision*. 2021, pp. 4965–4974.
- [96] Yin Zhu et al. “Heterogeneous transfer learning for image classification”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 25. 1. 2011, pp. 1304–1309.